

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2017

Tập luận văn ứng viên đội tuyển quốc gia IOI Trung Quốc

Tháng 4 năm 2017

Nguồn gốc: Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2017

IOI China candidate team papers / National training team 2017 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2017. Khác với một số tập trước đó, lớp văn bản của tập này trích xuất khá tốt bằng pdftotext. Bản LaTeX này chuyển ngữ phần nhận diện tập sách, toàn bộ mục lục, bài đầu tiên của Mao Xiao về công thức truy hồi của dãy số, và bài thứ hai của Yang Jiaqi về matching trên đồ thị tổng quát bằng đại số tuyến tính, bài thứ ba của Yuan Yutao về bài toán tính tổng đa thức, bài thứ tư của Zhong Zhixian về bài toán tập độc lập trong thi tin học, và bài thứ năm của Chen Junkun về *Đồ thị con kỳ diệu*.

1 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2017*. Trang bìa ghi thời điểm phát hành là tháng 4 năm 2017. Tập PDF gồm 204 trang, được tạo bằng LaTeX với gói hyperref.

2 Mục lục đã dịch

Trang	Bài viết	Tác giả
1	Một số nghiên cứu về công thức truy hồi của dãy số	Mao Xiao
12	Matching trên đồ thị tổng quát dựa trên đại số tuyến tính	Yang Jiaqi
27	Tính tổng đa thức	Yuan Yutao
32	Bàn về bài toán tập độc lập trong thi tin học	Zhong Zhixian
50	Báo cáo ra đề và mở rộng của <i>Đồ thị con kỳ diệu</i>	Chen Junkun
75	Tìm hiểu bài toán bao đóng bắc cầu động	Sun Yaofeng
90	Báo cáo ra đề <i>A + B Problem</i>	Wang Leping
99	Bước đầu tìm hiểu thuật toán chia khối kích thước phi chuẩn	Xu Mingkuan
122	Cây palindrome và ứng dụng	Weng Wentao
139	Báo cáo ra đề <i>Cây đen trắng</i>	Yan Shuyi
144	Báo cáo ra đề <i>Đa giác đều</i>	Yang Jingqin
155	Bàn về lời giải tuyến tính cho quy hoạch động có đơn điệu quyết định	Feng Zhe
165	Báo cáo ra đề <i>Cây đoạn bị thao túng</i>	Shen Rui
179	Bước đầu về logic máy tính và nghệ thuật: mô hình cường độ nốt khi chơi piano dựa trên logic	Zhao Shengyu
190	Báo cáo ra đề <i>Tái cấu trúc bộ gen</i>	Hong Huadun

3 Một số nghiên cứu về công thức truy hồi của dãy số

Mao Xiao, Trường Trung học Yali

Tóm tắt

Bài viết này là luận văn đội tuyển quốc gia tin học Trung Quốc năm 2017 của tác giả. Trong bài, tác giả trước hết giới thiệu đa thức truy hồi, sau đó giới thiệu thuật toán Berlekamp–Massey, rồi trình bày triển vọng ứng dụng của chúng trong thi tin học, gồm đếm công thức truy hồi và tìm đa thức đặc trưng của một số ma trận thưa.

Dẫn nhập

Đa thức truy hồi là một thuật ngữ do tác giả đặt ra. Trong thi tin học, công thức truy hồi thường được cho tường minh, còn nhiệm vụ của thí sinh thường là: biết một số hạng đầu của dãy và một công thức truy hồi nào đó, hãy tính một số hạng của dãy. Tuy nhiên, bài viết này nghiên cứu những công thức truy hồi ẩn. Vì vậy tác giả đưa ra khái niệm hoàn toàn mới là đa thức truy hồi. Đây là một khái niệm rất mạnh: nó không chỉ có các ứng dụng như đếm công thức truy hồi, mà còn giúp ta học và hiểu thuật toán Berlekamp–Massey.

Berlekamp–Massey là một thuật toán xuất sắc nhưng bị lạnh nhạt. Trong thi tin học, Berlekamp–Massey không mấy phổ biến; ngay cả những người biết nó cũng thường chỉ xem nó như một đường tắt. Ngay cả trên phạm vi thế giới, tôi cũng chưa nghe nói thuật toán này từng xuất hiện như thuật toán chuẩn của một bài nào. Tuy vậy, nó có khá nhiều ứng dụng.

Do đó, bài viết này sẽ trình bày hai nội dung ấy và giới thiệu một số ứng dụng của chúng.

3.1 Đa thức truy hồi

Trong mục này, tác giả sẽ đưa ra nhiều định nghĩa mới, đồng thời giới thiệu nhiều kết luận hữu ích.

3.1.1 Bậc nhỏ nhất

Định nghĩa 1.1. Với dãy $\{s_0, s_1, \dots, s_{l-1}\}$, ta đặt đa thức tương ứng của dãy là

$$S(x) = \sum_{k=0}^{l-1} s_k x^k.$$

Định nghĩa 1.2. Với một đa thức khác không A , **bậc** của nó được định nghĩa là bậc của hạng tử cao nhất, ký hiệu $\deg(A)$. Riêng đa thức không có bậc là vô cùng nhỏ.

Định nghĩa 1.3. Trên một trường nào đó, với dãy $\{r_0, r_1, \dots, r_{m-1}\}$ và dãy $\{a_0, a_1, \dots, a_{n-1}\}$, nếu r không toàn bằng không và với mọi $0 \leq p \leq n - m$ ta có

$$\sum_{k=0}^{m-1} a_{p+k} \times r_{m-k-1} = 0, \quad (1.1)$$

thì gọi r là một **công thức truy hồi**¹ của dãy a .

Nếu $r_0 = 1$, ta gọi r là **công thức truy tiến**. Nếu r là công thức truy tiến có độ dài ngắn nhất trong mọi công thức truy tiến của dãy a , ta gọi nó là **công thức truy tiến ngắn nhất** của dãy a .

¹Một số nơi gọi khái niệm này là công thức truy tiến. Tác giả không tìm được khác biệt giữa hai thuật ngữ, nên trong bài chọn thuật ngữ ít dùng hơn để định nghĩa riêng, còn thuật ngữ kia giữ nghĩa thường gặp.

Định nghĩa 1.4. Với dãy a và đa thức khác không $R(x)$, luôn tồn tại duy nhất công thức truy hồi ngắn nhất r sao cho đa thức tương ứng của nó là $R(x)$. Ta gọi r là **công thức truy hồi ngắn nhất tương ứng** của $R(x)$. Độ dài của r trừ một được gọi là **bậc nhỏ nhất** của đa thức $R(x)$, ký hiệu d_R hoặc $d_{R(x)}$. Đa thức $R(x)$ được gọi là **đa thức truy hồi bậc** d_R của a .

Chứng minh. Cho một đa thức khác không $R(x)$ và một dãy a độ dài n . Dãy gồm các hệ số từ hạng tử bậc 0 đến hạng tử bậc $\max(\deg(R), n)$ của nó chắc chắn là một công thức truy hồi của a , và đa thức tương ứng là $R(x)$, nên r tồn tại.

Nếu bậc nhỏ nhất của đa thức khác không $R(x)$ là d , thì r nhất định là dãy gồm các hệ số từ bậc 0 đến bậc d của nó, nên r là duy nhất. ■

3.1.2 Các phép toán trên đa thức truy hồi

Sau khi có đa thức truy hồi, ta có thể nhận được một số kết luận hữu ích. Từ các kết luận này, ta có thể làm nhiều việc, và chúng cũng giúp ích cho thuật toán Berlekamp–Massey.

Bổ đề 1.1. Đa thức $R(x)$ có bậc nhỏ nhất d đối với dãy a khi và chỉ khi d không nhỏ hơn bậc của $R(x)$, và các hệ số từ bậc d đến bậc $n - 1$ của $R(x) \times A(x)$ đều bằng không.

Chứng minh. Ta nhận thấy công thức (1.1) có dạng tích chập; từ đó dễ suy ra bổ đề. ■

Định lý 1.1. Giả sử $F(x), G(x)$ là hai đa thức có bậc nhỏ nhất không vượt quá d đối với dãy a . Khi đó $F(x) + G(x)$ cũng có bậc nhỏ nhất không vượt quá d đối với dãy a .

Chứng minh. Theo Bổ đề 1.1, các hệ số từ bậc d đến bậc $n - 1$ của $F(x) \times A(x)$ và $G(x) \times A(x)$ đều bằng không. Vì vậy các hệ số từ bậc d đến bậc $n - 1$ của $(F(x) + G(x)) \times A(x)$ đều bằng không; đồng thời bậc của $F(x) + G(x)$ hiển nhiên không vượt quá d . Do đó $F(x) + G(x)$ có bậc nhỏ nhất không vượt quá d đối với dãy a . ■

Bổ đề 1.2. Giả sử $F(x)$ có bậc nhỏ nhất d đối với dãy a . Khi đó với $k \in \mathbb{N}$ và $k \geq 0$, $x^k F(x)$ có bậc nhỏ nhất không vượt quá $d + k$ đối với dãy a .

Chứng minh. Theo Bổ đề 1.1, các hệ số từ bậc d đến bậc $n - 1$ của $F(x) \times A(x)$ đều bằng không. Vì vậy các hệ số từ bậc $d + k$ đến bậc $n + k - 1$ của $x^k F(x) \times A(x)$ đều bằng không; đồng thời bậc của $x^k F(x) \times A(x)$ hiển nhiên không vượt quá $d + k$. Do đó $x^k F(x)$ có bậc nhỏ nhất không vượt quá $d + k$ đối với dãy a . ■

Định lý 1.2. Giả sử $F(x)$ có bậc nhỏ nhất d_F đối với dãy a . Khi đó $F(x) \times G(x)$ có bậc nhỏ nhất không vượt quá $d_F + \deg(G)$ đối với dãy a .

Chứng minh. Suy ra từ Định lý 1.1 và Bổ đề 1.2. ■

3.1.3 Cơ sở

Định nghĩa 1.5. Với dãy a và dãy đa thức $\{\text{Base}_0(x), \text{Base}_1(x), \text{Base}_2(x), \dots\}$, nếu với mọi k , $\text{Base}_k(x)$ là đa thức có bậc nhỏ nhất nhỏ nhất trong mọi đa thức có hạng tử thấp nhất là x^k , thì gọi dãy đa thức Base là **một cơ sở** của dãy a .

Ta nhận thấy một cơ sở của một dãy thật ra chính là cơ sở của không gian tuyến tính do nó sinh ra. Ta cũng có thể nhận thấy tọa độ của một đa thức theo một cơ sở liên hệ chặt chẽ với bậc nhỏ nhất của nó.

Định lý 1.3. Nếu Base là cơ sở của dãy a , thì mọi đa thức có bậc nhỏ nhất d đều có thể được biểu diễn tuyến tính một cách duy nhất bởi một số $\text{Base}_k(x)$ có bậc nhỏ nhất không vượt quá d ; hơn nữa trong biểu diễn đó nhất định tồn tại ít nhất một $\text{Base}_k(x)$ có bậc nhỏ nhất đúng bằng d .

Chứng minh. Giả sử $R(x)$ là một đa thức có bậc nhỏ nhất d . Ta xét cách phân rã nó.

Nếu $R(x)$ bằng không thì không cần phân rã. Ngược lại, giả sử hạng tử thấp nhất của nó là x^k , hệ số là C_r , và hệ số tương ứng của $\text{Base}_k(x)$ là C_{Base} . Khi đó hạng tử thấp nhất của $R(x) - C_{\text{Base}}^{-1} C_r \text{Base}_k(x)$ ít nhất là x^{k+1} . Vì $\text{Base}_k(x)$ là một đa thức có bậc nhỏ nhất nhỏ nhất trong các đa thức có hạng tử thấp nhất x^k , bậc nhỏ nhất của nó hiển nhiên không vượt quá d . Theo Định lý 1.1, $R(x) - C_{\text{Base}}^{-1} C_r \text{Base}_k(x)$ cũng có bậc nhỏ nhất không vượt quá d . Ta phân rã biểu thức này theo cách đệ quy. Cuối cùng ta thu được một biểu diễn tuyến tính thỏa điều kiện.

Vì các $\text{Base}_k(x)$ là một cơ sở của không gian tuyến tính mà chúng sinh ra, cách biểu diễn hiển nhiên là duy nhất.

Nếu trong biểu diễn, mọi $\text{Base}_k(x)$ đều có bậc nhỏ nhất nhỏ hơn d , thì hoặc đa thức này là đa thức không, hoặc theo Bổ đề 1.1 bậc nhỏ nhất của nó sẽ nhỏ hơn d , mâu thuẫn. ■

3.2 Thuật toán Berlekamp–Massey

Mục này giới thiệu thuật toán Berlekamp–Massey². Tuy các tài liệu như Wikipedia [1] đều đã giới thiệu quy trình của thuật toán Berlekamp–Massey, tác giả không tìm được bất kỳ bản quy trình thuật toán tiếng Trung nào. Đồng thời, tác giả cũng muốn dùng ngôn ngữ của bài viết này để trình bày lại quy trình của thuật toán.

3.2.1 Giới thiệu thuật toán

Thuật toán Berlekamp–Massey có thể tìm một cơ sở Base cho dãy a độ dài n trên một trường bất kỳ bằng $\mathcal{O}(n^2)$ phép toán và không cần thêm không gian đáng kể.

3.2.2 Quy trình thuật toán

Theo định nghĩa, mọi đa thức có hạng tử thấp nhất x^k , với $k \geq n$, đều có bậc nhỏ nhất đúng bằng bậc của nó. Vì vậy với $\text{Base}_k(x)$, ta chỉ cần chọn bất kỳ Cx^k nào với $C \neq 0$. Không mất tính tổng quát, giả sử mọi đa thức tìm được đều có hệ số của hạng tử thấp nhất bằng 1, khi đó ở đây ta đặt $C = 1$.

Giả sử ta đã tìm được một đa thức có bậc nhỏ nhất nhỏ nhất trong các đa thức từ $\text{Base}_{k+1}(x)$ đến $\text{Base}_n(x)$. Xét cách tìm $\text{Base}_k(x)$.

Ta xét $x \text{Base}_k(x)$. Theo Định lý 1.3, ta chỉ cần làm cho bậc nhỏ nhất lớn nhất trong biểu diễn tuyến tính của nó càng nhỏ càng tốt.

²Wikipedia tiếng Trung phiên âm thuật toán này là Berlekamp–Massey.

Trước hết, vì hạng tử thấp nhất của $x \text{Base}_k(x)$ là x^{k+1} , trong biểu diễn nhất định có hạng $\text{Base}_{k+1}(x)$. Xét các hạng còn lại: rõ ràng bậc nhỏ nhất lớn nhất trong số chúng phải càng nhỏ càng tốt.

Đặt $\text{res}(x) = x \text{Base}_k(x) - \text{Base}_{k+1}(x)$. Ta tính $\text{pro}(x) = \text{res}(x) \times A(x)$. Theo Bổ đề 1.1, để biết các hệ số từ bậc $k+1$ đến bậc $n-1$ của $\text{pro}(x)$ đều bằng không.

Xét hạng tử bậc n . Nếu nó bằng không, ta đã biết $x \text{Base}_k(x)$; ngược lại giả sử nó là u . Xét $\text{Base}_l(x)$ với $l > k+1$. Nếu hệ số bậc n của $\text{Base}_l(x) \times A(x)$ khác không, giả sử hệ số bậc n là v . Để biết các hệ số từ bậc $k+1$ đến bậc n của $\text{res}(x) - uv^{-1} \text{Base}_l(x)$ đều bằng không. Vì bậc nhỏ nhất lớn nhất phải càng nhỏ càng tốt, nếu tồn tại đa thức thỏa điều kiện thì $\text{Base}_l(x)$ nhất định là đa thức có bậc nhỏ nhất nhỏ nhất trong mọi đa thức thỏa điều kiện. Lấy $\text{Base}_l(x)$ như vậy, ta có thể tìm được $x \text{Base}_k(x)$. Nếu không tồn tại $\text{Base}_l(x)$ như vậy, thì chỉ có thể là bậc nhỏ nhất của $x \text{Base}_k(x)$ đã vượt quá n . Khi đó ta chỉ cần tùy ý chọn một đa thức có hạng tử thấp nhất là hạng bậc x^{k+1} và hệ số bằng 1.

Khi đã biết $x \text{Base}_k(x)$, hiển nhiên ta cũng biết $\text{Base}_k(x)$.

Dùng phương pháp này, mỗi $P_k(x)$ đều có thể được tìm trong không quá $\mathcal{O}(n)$ phép toán, vì vậy tổng số phép toán là $\mathcal{O}(n)$, còn số Base cần tính chỉ là $\mathcal{O}(n)$. Do đó tổng số phép toán là $\mathcal{O}(n^2)$.

Rõ ràng ngoài việc lưu $A(x)$ và Base, thuật toán này không cần không gian phụ đáng kể.

3.2.3 Mã giả

Theo mô tả trên, thuật toán có thể được cài đặt bằng phương pháp sau.

Thuật toán 1 Berlekamp–Massey Algorithm I

Require: $a, n = |a|$

Ensure: Base

```

1:  $\text{Base}_k = x^k$  ( $k \geq n$ )
2:  $l \leftarrow n + 1$ 
3:  $v \leftarrow 1$ 
4: for  $k \leftarrow n - 1$  to 1 do
5:    $u \leftarrow [x^n] \text{Base}_{k+1}(x) \times A(x)$ 
6:    $\text{Base}_k(x) \leftarrow (\text{Base}_{k+1}(x) - uv^{-1} \text{Base}_l(x)) \div x$ 
7:   if  $(u \neq 0) \wedge (d_{\text{Base}_{k+1}} < d_{\text{Base}_l})$  then
8:      $l \leftarrow k + 1$ 
9:      $v \leftarrow u$ 
10:  end if
11: end for
```

Theo mô tả thuật toán, giá trị ban đầu của l và v không quan trọng; chỉ cần giá trị ban đầu thỏa $l \geq n+1$ là có thể bảo đảm đáp án đúng. Tuy nhiên, $l = n+1, v = 1$ là giá trị khởi tạo tham khảo trên Wikipedia tiếng Anh, nên tác giả xem nó như một phương án đã thành thông lệ.

Tuy nhiên, vì phép toán đa thức khá phiền phức, khi cài đặt thật sự ta thường không dùng đa thức một cách tường minh.

Ký hiệu r_k là công thức truy hồi ngắn nhất tương ứng với $\text{Base}_{n-k+1} \div x^{n-k+1}$. Khi đó ta dễ thấy r_k là công thức truy tiến ngắn nhất của tiền tố độ dài k của dãy a . Vì vậy quá trình trên dễ dàng được đổi thành quá trình tính r .

Ta xem dãy như một mảng có độ dài thay đổi, chỉ số bắt đầu từ 0. Thuật toán mới như sau.

Thuật toán 2 Berlekamp–Massey Algorithm II

Require: $a, n = |a|$

Ensure: r_k ($0 \leq k \leq n$)

```
1:  $r_0 = \{1\}$ 
2:  $\text{last} \leftarrow \{1\}$ 
3:  $v \leftarrow 1$ 
4:  $s \leftarrow 1$ 
5: for  $k \leftarrow 1$  to  $n$  do
6:    $r_k \leftarrow r_{k-1}$ 
7:    $u \leftarrow \sum_{i=0}^{|r_k|-1} a_{k-i-1} \times r_i$ 
8:   if ( $u \neq 0$ ) then
9:     if  $|r_k| < |\text{last}| + s$  then
10:       $r_k.\text{append}(\{0\} \times (|\text{last}| + s - |r_k|))$ 
11:      for  $i \leftarrow 0$  to  $|\text{last}| - 1$  do
12:         $r_k[i + s] \leftarrow r_k[i + s] - uv^{-1} \text{last}[i]$ 
13:      end for
14:       $\text{last} \leftarrow r_{k+1}$ 
15:       $v \leftarrow u$ 
16:       $s \leftarrow 0$ 
17:    else
18:      for  $i \leftarrow 0$  to  $|\text{last}| - 1$  do
19:         $r_k[i + s] \leftarrow r_k[i + s] - uv^{-1} \text{last}[i]$ 
20:      end for
21:    end if
22:  end if
23:   $s \leftarrow s + 1$ 
24: end for
```

Ngoài ra, nếu ta chỉ cần tìm công thức truy tiến ngắn nhất của cả dãy, thì ở mọi thời điểm ta thật ra chỉ cần lưu một r_k và một last . Nếu không tính kích thước từng phần tử, độ phức tạp không gian có thể giảm xuống $\mathcal{O}(n)$.

3.3 Đếm công thức truy hồi

3.3.1 Đa thức truy hồi nhỏ nhất và đa thức truy hồi thứ nhì không tầm thường

Định nghĩa 3.1. Trong mọi $\text{Base}_k(x)$ đã tìm được, một đa thức có bậc nhỏ nhất nhỏ nhất được gọi là **đa thức truy hồi nhỏ nhất**, ký hiệu $M(x)$. Nếu có nhiều đa thức như vậy thì chọn tùy ý; bậc nhỏ nhất của nó được ký hiệu d_M .

Trong mọi $\text{Base}_k(x)$ đã tìm được, nếu tồn tại một đa thức không thể biểu diễn dưới dạng $x^k M(x)$, với $k \in \mathbb{N}, k \geq 0$, thì đa thức có bậc nhỏ nhất nhỏ nhất trong số đó được gọi là **đa thức truy hồi thứ nhì không tầm thường**, ký hiệu $S(x)$. Nếu có nhiều đa thức như vậy thì chọn tùy ý; nếu không tồn tại thì đặt $S(x) = \text{Base}_{n+1}(x)$. Bậc nhỏ nhất của nó được ký hiệu d_S .

3.3.2 Biểu diễn tuyến tính

Định nghĩa 3.2. Với các đa thức $A(x), B(x), C(x)$, nếu tồn tại duy nhất đa thức $F_B(x)$ có bậc không vượt quá $d_A - d_B$ và đa thức $F_C(x)$ có bậc không vượt quá $d_A - d_C$ sao cho

$$A(x) = F_B(x)B(x) + F_C(x)C(x),$$

thì gọi $A(x)$ là **có thể biểu diễn tuyến tính** bởi $B(x), C(x)$.

Bổ đề 3.1. Với các đa thức $A(x), B(x), C(x), D(x), E(x)$, nếu $A(x)$ có thể biểu diễn tuyến tính bởi $B(x), C(x)$, và $B(x), C(x)$ đều có thể biểu diễn tuyến tính bởi $D(x), E(x)$, thì $A(x)$ cũng có thể biểu diễn tuyến tính bởi $D(x), E(x)$.

Chứng minh. Đặt

$$\begin{aligned} A(x) &= F_B(x)B(x) + F_C(x)C(x), \\ B(x) &= F_{BD}(x)D(x) + F_{BE}(x)E(x), \\ C(x) &= F_{CD}(x)D(x) + F_{CE}(x)E(x). \end{aligned}$$

Dễ biết

$$\begin{aligned} A(x) &= (F_B(x)F_{BD}(x) + F_C(x)F_{CD}(x))D(x) \\ &\quad + (F_D(x)F_{BE}(x) + F_C(x)F_{CE}(x))E(x). \end{aligned}$$

Vì

$$\deg(F_B(x)) \leq d_A - d_B, \quad \deg(F_{BD}(x)) \leq d_B - d_D,$$

nên

$$\deg(F_B(x)F_{BD}(x)) = \deg(F_B(x)) + \deg(F_{BD}(x)) \leq d_A - d_D.$$

Tương tự,

$$\deg(F_C(x)F_{CD}(x)) \leq d_A - d_D.$$

Do đó

$$\deg(F_B(x)F_{BD}(x) + F_C(x)F_{CD}(x)) \leq d_A - d_D.$$

Tương tự,

$$\deg(F_B(x)F_{BE}(x) + F_C(x)F_{CE}(x)) \leq d_A - d_E.$$

Quá trình này bảo đảm tính duy nhất, nên bổ đề được chứng minh. ■

Định lý 3.1. Sau khi tính xong $\text{Base}_k(x)$, nó là đa thức nhỏ nhất hoặc là đa thức thứ nhì không tầm thường.

Tạm thời ta chưa chứng minh định lý này; trước hết giới thiệu một bổ đề mới, rồi chứng minh cả hai cùng lúc.

Bổ đề 3.2. Mọi $\text{Base}_k(x)$ đều có thể được biểu diễn tuyến tính bởi $S(x)$ và $M(x)$.

Chứng minh. Xét quy trình của thuật toán Berlekamp–Massey.

Khi $k = n$, hiển nhiên Định lý 3.1 và Bổ đề 3.2 đều đúng.

Giả sử sau khi tính xong $\text{Base}_{k+1}(x)$, cả hai đều đúng.

Xét quá trình tính $x \text{Base}_k(x)$. Cuối cùng nó nhất định là $\text{Base}_{k+1}(x)$ cộng với một hằng số nhân với một $\text{Base}_l(x)$ nào đó. Theo mô tả thuật toán, ta dễ nhận thấy $\text{Base}_{k+1}(x)$ và hạng còn lại đều là một trong $M(x)$ và $S(x)$.

Vì vậy bậc nhỏ nhất của $x \text{Base}_k(x)$ bằng $\max(d_M, d_S)$, còn bậc nhỏ nhất của $\text{Base}_k(x)$ phải trừ thêm một. Do đó nó nhất định là đa thức nhỏ nhất hoặc đa thức thứ nhì tầm thường. Như vậy ta đã chứng minh Định lý 3.1.

Hiển nhiên với chính nó, Bổ đề 3.2 đúng. Tuy nhiên vì $M(x), S(x)$ đã thay đổi, ta còn phải xét $\text{Base}_l(x)$ với $l > k$.

Xét sự thay đổi của $M(x)$ và $S(x)$. $M(x)$ hoặc không đổi, hoặc trở thành $S(x)$ mới.

Xét $S(x)$. Theo quy trình thuật toán, ta có

$$x \text{Base}_k(x) = C_M M(x) + C_S S(x).$$

Do đó

$$S(x) = x \text{Base}_k(x) - C_M M(x).$$

Vì $d_S > d_{\text{Base}_k}$ và $d_S \geq d_M$, $S(x)$ trước khi thay đổi có thể được biểu diễn tuyến tính bởi $M(x)$ và $S(x)$ sau khi thay đổi.

Như vậy, $M(x)$ và $S(x)$ trước khi thay đổi đều có thể được biểu diễn tuyến tính bởi $M(x)$ và $S(x)$ sau khi thay đổi. Theo Bổ đề 3.1, $\text{Base}_l(x)$ với $l > k$ vẫn có thể được biểu diễn tuyến tính bởi $S(x)$ và $M(x)$.

Đồng thời, với trường hợp $k > n$, vì $\text{Base}_k(x) = x^{k-n} \text{Base}_n(x)$, bổ đề hiển nhiên đúng.

Tương tự, quá trình này giữ tính duy nhất, nên bổ đề được chứng minh. ■

Định lý 3.2. Mọi đa thức đều có thể được biểu diễn tuyến tính bởi $S(x)$ và $M(x)$.

Chứng minh. Dễ suy ra từ Định lý 1.3 và Bổ đề 3.2. ■

3.3.3 Định lý đếm công thức truy hồi

Định lý 3.3 (Định lý đếm đa thức truy hồi). Giả sử kích thước của trường là q . Khi đó số đa thức truy hồi bậc k của một dãy là

$$\text{num}_k = q^{\max(k-d_M-1,0)+\max(k-d_S-1,0)} (q^{[k \geq d_M]+[k \geq d_S]} - 1).$$

Trong công thức này, $[x]$ bằng 1 khi x đúng, và bằng 0 nếu không.

Chứng minh. Dễ suy ra từ Định lý 3.2. ■

Định lý 3.4 (Định lý đếm công thức truy hồi). Số công thức truy hồi của một dãy là

$$\sum_{k=1}^n (n - k + 1) \text{num}_k.$$

Chứng minh. Một đa thức truy hồi bậc x tương ứng đúng một công thức truy hồi có độ dài $x, x+1, \dots, n$, tổng cộng $n - x + 1$ công thức, nên có công thức trên. ■

3.4 Đa thức đặc trưng của ma trận thưa đặc biệt

Thuật toán Berlekamp–Massey không chỉ có tác dụng rất mạnh trong lĩnh vực công thức truy hồi; nó cũng có thể giúp ích cho các bài toán liên quan đến ma trận.

Vì vậy, chương này sẽ giới thiệu cách dùng thuật toán đó để tính đa thức đặc trưng của ma trận thưa đặc biệt.

3.4.1 Điều kiện quan trọng

Khi mỗi trị riêng chỉ tương ứng với một khối Jordan³, đa thức đặc trưng chính là đa thức tối tiểu. Đây là điều kiện quan trọng để thuật toán này hoạt động.

³Nói cách khác, chiều cao của mỗi vector đều bằng bội số của nó.

3.4.2 Quá trình thực hiện

Theo định lý Cayley–Hamilton, đa thức đặc trưng của một ma trận $n \times n$ A là đa thức triệt tiêu của nó; tức là nếu đa thức đặc trưng là p thì $p(A) = 0$.

Nhớ lại định nghĩa công thức truy hồi, ta thấy nếu chọn ngẫu nhiên một vector v , trước hết hiển nhiên $p(Av) = 0$. Nếu kích thước của trường số đủ lớn, thì với xác suất rất cao, dãy vector $\{v, Av, A^2v, \dots\}$ tồn tại một công thức truy tiến ngắn nhất duy nhất r có độ dài n . Khi đó $R(x)$ chính là đa thức đặc trưng.

Nếu A là ma trận thưa và số phần tử là e , thì từ $A^k v$ có thể tính $A^{k+1}v$ trong $\mathcal{O}(e)$ phép toán.

Khi xét dùng thuật toán Berlekamp–Massey, ta có hai khó khăn.

Khó khăn thứ nhất là xử lý nhanh dãy vector. Ta có thể thiết kế một hàm băm, trong đó hàm băm là một biến đổi tuyến tính, biến mọi vector thành một số; như vậy ta có thể xử lý dãy vector.

Khó khăn thứ hai là xử lý một dãy vô hạn. Tuy dãy này dài vô hạn, ta có thể nhận thấy trong trường hợp ngẫu nhiên, độ dài công thức truy tiến ngắn nhất của tiền tố độ dài $2n$ xấp xỉ n . Vì vậy ta có thể lấy một tiền tố đủ dài để tính. Một cách tốt hơn là xét Thuật toán 2: thật ra ta có thể liên tục tăng tiền tố cho đến khi độ dài công thức truy tiến ngắn nhất đạt n .

Như vậy, ta thu được một thuật toán Monte Carlo có thể tính đa thức đặc trưng của ma trận thưa đặc biệt bằng $\mathcal{O}(n(n + e))$ phép toán và không gian tuyến tính theo kích thước ma trận.

3.4.3 Định thức của ma trận thưa

Xét cách tính định thức $\det(A)$. Hiển nhiên nếu ma trận này thỏa điều kiện quan trọng ở Mục 4.1, ta có thể tìm đa thức đặc trưng, rồi lấy hạng tử tự do nhân với $(-1)^n$.

Nếu ma trận này không thỏa điều kiện đó, ta có thể lấy ngẫu nhiên một ma trận đường chéo B ; số phép toán là $\mathcal{O}(ne)$. Nếu trường số đủ lớn, thì AB có xác suất rất cao thỏa điều kiện này. Vì $\det(AB) = \det(A) \det(B)$, ta chỉ cần tính $\det(AB)$, rồi chia cho $\det(B)$. Vì B là ma trận đường chéo, $\det(B)$ chính là tích các phần tử trên đường chéo của B .

Như vậy, ta thu được một thuật toán Monte Carlo có thể tính định thức của ma trận thưa bằng $\mathcal{O}(n(n + e))$ phép toán và không gian tuyến tính theo kích thước ma trận.

3.4.4 Đếm cây khung của đồ thị thưa

Hiển nhiên, có thể dùng định lý ma trận-cây Kirchhoff để tính số cây khung. Nếu đồ thị đủ thưa, thì định thức của ma trận Laplace hiển nhiên có thể được tính bằng phương pháp ở Mục 4.3.

Độ phức tạp thời gian của thuật toán này là $\mathcal{O}(n(n + m))$, độ phức tạp không gian là $\mathcal{O}(n + m)$, rất tốt.

3.5 Tổng kết

Toàn bài đều xoay quanh công thức truy hồi. Phần thứ nhất trình bày đa thức truy hồi, phần thứ hai nghiên cứu thuật toán Berlekamp–Massey, còn phần thứ ba và thứ tư chủ yếu giới thiệu một số ứng dụng của chúng.

Tôi hy vọng bài viết này có thể đóng vai trò gợi mở. Đây vẫn chỉ là một nghiên cứu chưa chín muồi về công thức truy hồi, và các ứng dụng được giới thiệu cũng khá cơ bản. Khi khoa học kỹ thuật không ngừng tiến bộ, nhất định còn nhiều ứng dụng thú vị hơn đang chờ chúng ta khám phá. Vì vậy, mong rằng bài viết này có thể gợi ý cho độc giả và dẫn ra một số ứng dụng hấp dẫn hơn.

Lời cảm ơn

1. Cảm ơn bạn Du Yuhao. Lần đầu tiên tôi nghe nói về thuật toán này là từ bạn ấy, và sự xuất hiện của một phần nội dung trong bài viết này không thể tách rời sự giúp đỡ của bạn ấy.
2. Cảm ơn các bạn Du Yuhao, Yang Jiaqi và Weng Wentao đã sửa lỗi.
3. Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi.

Tài liệu tham khảo

[1] Wikipedia.

https://en.wikipedia.org/wiki/Berlekamp%E2%80%93Massey_algorithm

4 Matching trên đồ thị tổng quát dựa trên đại số tuyến tính

Yang Jiaqi, Trường Trung học Tưởng niệm Trung Sơn, thành phố Trung Sơn

Tóm tắt

Bài toán matching trên đồ thị là một bài toán kinh điển của lý thuyết đồ thị, có vị trí quan trọng cả trong học thuật lẫn trong thi thuật toán. Bài viết này chủ yếu giới thiệu một thuật toán matching trên đồ thị tổng quát dựa trên đại số tuyến tính, đồng thời đưa ra một số ứng dụng đơn giản của nó. Thuật toán này rất khác với các phương pháp tổ hợp truyền thống để giải bài toán này; hy vọng bản thân thuật toán và những tư tưởng dùng trong quá trình thu được thuật toán có thể gợi mở cho độc giả.

Dẫn nhập

Bài toán matching trên đồ thị có thể được chia theo tính chất của đồ thị thành hai loại: matching trên đồ thị hai phía và matching trên đồ thị tổng quát. Trong đó, matching hai phía đã xuất hiện nhiều năm trong giới thi thuật toán và là một điểm kiến thức thường gặp. Vì thuật toán của nó tương đối đơn giản, nó đã được phổ cập rất tốt: hiện nay phần lớn thí sinh đều đã nắm vững và có thể vận dụng thành thạo thuật toán matching hai phía.

Ngược lại, matching trên đồ thị tổng quát là một điểm kiến thức mới nổi. Vài năm gần đây, nó đã xuất hiện trong bài tập đội tuyển, kỳ thi mùa đông và các kỳ thi giao lưu đội tuyển. Nhưng các kỳ thi ấy đều ở cấp trên NOI; trong các kỳ thi dưới NOI, đề về matching tổng quát còn hiếm gặp, và hiện nay số thí sinh nắm được thuật toán matching tổng quát cũng tương đối ít. Tác giả cho rằng một nguyên nhân quan trọng khiến thuật toán này chưa phổ biến là thuật toán cây hoa, cách thường dùng hiện nay để giải loại bài toán này, khá phức tạp và có ngưỡng học tập cao. Vì vậy bài viết này sẽ giới thiệu một thuật toán matching dễ nhớ, dễ cài đặt, với hy vọng thúc đẩy việc phổ cập thuật toán matching trên đồ thị tổng quát.

Như tiêu đề bài viết đã nói, thuật toán được giới thiệu ở đây là một phương pháp đại số. Dù trong thi thuật toán, phương pháp tổ hợp thường gặp hơn, phương pháp đại số cũng là một công cụ mạnh để giải các bài toán đồ thị. Ví dụ nổi tiếng nhất có lẽ là định lý Matrix-Tree để đếm cây khung. Phương pháp đại số đem lại một góc nhìn hoàn toàn mới khi giải quyết vấn đề; hy vọng bài viết này có thể giúp độc giả cảm nhận sức hấp dẫn riêng của phương pháp đại số.

Bài viết trước hết sẽ bắt đầu từ bài toán matching hoàn hảo, giải bài toán quyết định tương ứng, sau đó thu được một thuật toán xây dựng phương án matching hoàn hảo và từng bước tối ưu nó bằng một số kiến thức đại số tuyến tính. Tiếp theo, bài viết chuyển bài toán matching cực đại về matching hoàn hảo, rồi cuối cùng trình bày một số ứng dụng và mở rộng của thuật toán này.

4.1 Kiến thức chuẩn bị

4.1.1 Lý thuyết đồ thị

Nếu không nói gì thêm, các đồ thị trong bài đều là đồ thị vô hướng.

Ta dùng $G = (V, E)$ để biểu diễn một đồ thị, trong đó V là tập đỉnh và E là tập cạnh. Ta thường dùng n để biểu diễn kích thước $|V|$ của tập đỉnh, dùng m để biểu diễn kích thước $|E|$ của tập cạnh, và thường xem $V = \{v_1, v_2, \dots, v_n\}$.

Khi không gây nhập nhằng, ta dùng uv để biểu diễn cạnh (u, v) .

Một **matching** M của đồ thị $G = (V, E)$ là một tập con của tập cạnh E , thỏa mãn hai cạnh bất kỳ trong M không có đỉnh chung. Nếu đỉnh v là đầu mút của một cạnh nào đó trong M , ta nói v nằm trong matching M . Giá trị lớn nhất của kích thước mọi matching của đồ thị G được ký hiệu là $\nu(G)$; nếu $|M| = \nu(G)$, ta gọi M là một **matching cực đại** của G . Đặc biệt, nếu mọi đỉnh của G đều nằm trong matching M , ta gọi M là một **matching hoàn hảo** của G . Rõ ràng G chỉ có thể có matching hoàn hảo khi $|V|$ là số chẵn.

Với một đồ thị $G = (V, E)$ và một tập đỉnh $U \subseteq V$, ta ký hiệu đồ thị thu được bằng cách xóa mọi đỉnh trong U khỏi G là $G - U$, tức là

$$G - U = (V \setminus U, E \cap \{uv \mid u, v \in V \setminus U\});$$

đồ thị con cảm sinh của G theo U được ký hiệu là $G[U]$, tức là

$$G[U] = G - (V \setminus U).$$

4.1.2 Đại số tuyến tính

Với một ma trận A có m hàng và n cột, ta đặt tập chỉ số hàng là $\{1, \dots, m\}$, tập chỉ số cột là $\{1, \dots, n\}$. Phần tử ở hàng i , cột j của ma trận A được ký hiệu là $A_{i,j}$.

Với một ma trận $m \times n$ A và các tập bất kỳ $I \subseteq \{1, \dots, m\}, J \subseteq \{1, \dots, n\}$, ta ký hiệu ma trận con thu được bằng cách giữ lại mọi hàng trong I và mọi cột trong J là $A_{I,J}$. Ta viết tắt $A_{I,\{1,\dots,n\}}$ thành $A_{I,\cdot}$, và $A_{\{1,\dots,m\},J}$ thành $A_{\cdot,J}$.

Tương tự, ta ký hiệu $A^{I,J}$ là ma trận con thu được bằng cách xóa khỏi A mọi hàng trong I và mọi cột trong J .

Với một ma trận vuông $n \times n$ A , định thức của nó được ký hiệu là $\det A$, và

$$\det A = \sum_{\pi \in \Sigma_n} (-1)^{\text{sgn } \pi} \prod_{k=1}^n A_{k,\pi(k)}.$$

Trong đó Σ_n biểu diễn tập mọi hoán vị của $\{1, \dots, n\}$, còn $\text{sgn } \pi$ biểu diễn dấu của hoán vị π . Nếu số cặp nghịch thế của dãy $\{\pi(1), \dots, \pi(n)\}$ là x , thì $\text{sgn } \pi = (-1)^x$.

Cho m vector n chiều $\{v_1, \dots, v_m\}$. Nếu tồn tại $\lambda_1, \lambda_2, \dots, \lambda_m$ sao cho

$$v = \sum_{i=1}^m \lambda_i v_i,$$

thì gọi v là một tổ hợp tuyến tính của $\{v_1, \dots, v_m\}$. Nếu không vector nào trong m vector này có thể được biểu diễn thành tổ hợp tuyến tính của các vector còn lại, ta nói m vector này độc lập tuyến tính; ngược lại, chúng phụ thuộc tuyến tính.

Với một ma trận A , nếu xem mỗi hàng của nó là một vector, thì số lượng cực đại các vector độc lập tuyến tính có thể chọn ra được gọi là **hạng hàng**; nếu xem mỗi cột là một vector, số lượng cực đại các

vector cột độc lập tuyến tính được gọi là **hạng cột**. Hạng hàng và hạng cột của một ma trận luôn bằng nhau, nên ta gọi chung là **hạng** của ma trận A , ký hiệu $\text{rank } A$.

Cuối cùng, ta nêu một kết luận mà không chứng minh.

Định lý 1.1. Bốn bài toán sau có cùng độ phức tạp:

1. tính định thức của ma trận;
2. nhân hai ma trận;
3. tìm ma trận nghịch đảo;
4. khử Gauss trên ma trận, tức phân rã LU.

Ta đặt độ phức tạp của cả bốn bài toán này là $\mathcal{O}(n^\omega)^4$.

4.2 Sự tồn tại của matching hoàn hảo

4.2.1 Ma trận Tutte

Định nghĩa 2.1 (Ma trận Tutte). Với một đồ thị vô hướng $G = (V, E)$, định nghĩa ma trận Tutte của G là một ma trận $n \times n$ $\tilde{A}(G)$, trong đó

$$\tilde{A}(G)_{i,j} = \begin{cases} x_{i,j}, & \text{nếu } v_i v_j \in E \text{ và } i < j, \\ -x_{j,i}, & \text{nếu } v_i v_j \in E \text{ và } i > j, \\ 0, & \text{trong các trường hợp khác.} \end{cases}$$

Ở đây, $x_{i,j}$ là một biến duy nhất gắn với cạnh $v_i v_j$.

Ta xét ví dụ sau.



Hình 1: Đồ thị G và ma trận Tutte của nó

Nửa trái của Hình 1 cho một đồ thị G , còn nửa phải cho ma trận Tutte $\tilde{A}(G)$ tương ứng với G . Có thể thấy mỗi cạnh của G đều tương ứng với một biến, nên $\tilde{A}(G)$ dùng tổng cộng $|E|$ biến.

4.2.2 Định lý Tutte

Ma trận Tutte có liên hệ mật thiết với sự tồn tại của matching hoàn hảo trong đồ thị. Thật vậy, ta có định lý sau.

Định lý 2.1 (Tutte). Đồ thị G có matching hoàn hảo khi và chỉ khi $\det \tilde{A}(G) \neq 0$.

Để chứng minh định lý này, trước hết ta cần dùng khái niệm “phủ chu trình chẵn” để đặc trưng một đồ thị có matching hoàn hảo.

⁴Trong học thuật, người ta thường cho rằng $2 \leq \omega < 2.373$.

Định nghĩa 2.2. Một **phủ chu trình** của đồ thị $G = (V, E)$ là việc dùng một số chu trình để phủ mọi đỉnh của G , sao cho mỗi đỉnh của G nằm đúng trong một chu trình.

Nói một cách hình thức, mỗi phủ chu trình của G tương ứng với một hoán vị π từ 1 đến n , thỏa mãn $v_i v_{\pi(i)} \in E$.

Nếu mọi chu trình trong phủ chu trình này đều có độ dài chẵn, ta gọi nó là một **phủ chu trình chẵn**; ngược lại, ta gọi nó là một **phủ chu trình lẻ**.

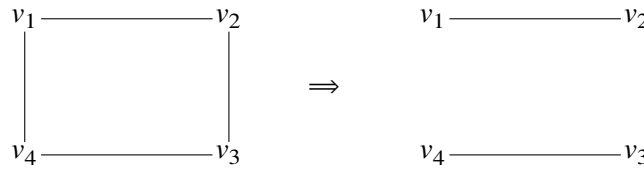
Chú ý rằng Định nghĩa 2.2 không yêu cầu các cạnh $v_i v_{\pi(i)}$ đôi một khác nhau, nghĩa là trong một phủ chu trình có thể xuất hiện cạnh lặp.

Bổ đề 2.2. Đồ thị $G = (V, E)$ có matching hoàn hảo khi và chỉ khi G có một phủ chu trình chẵn.

Chứng minh. Nếu G có một matching hoàn hảo, ta trực tiếp dùng chu trình độ dài hai do mỗi cặp đỉnh trong matching tạo thành để phủ G .

Nếu G có một phủ chu trình chẵn, thì trong mỗi chu trình chẵn, lấy cách một cạnh một cạnh, ta sẽ thu được một matching hoàn hảo của G . ■

Hình 2 dưới đây minh họa cách thu được một matching hoàn hảo từ một phủ chu trình chẵn của G .



Hình 2: Phủ chu trình chẵn và matching hoàn hảo

Có Bổ đề 2.2, ta có thể chứng minh định lý Tutte, tức Định lý 2.1.

Chứng minh định lý Tutte. Ta có

$$\det \tilde{A}(G) = \sum_{\pi \in \Sigma_n} (-1)^{\text{sgn}(\pi)} \prod_{k=1}^n \tilde{A}(G)_{k, \pi(k)}. \quad (2.1)$$

Các hạng tử khác 0 ở vế phải của đẳng thức có dạng

$$(-1)^{\text{sgn}(\pi)} \prod_{k=1}^n \pm x_{k, \pi(k)},$$

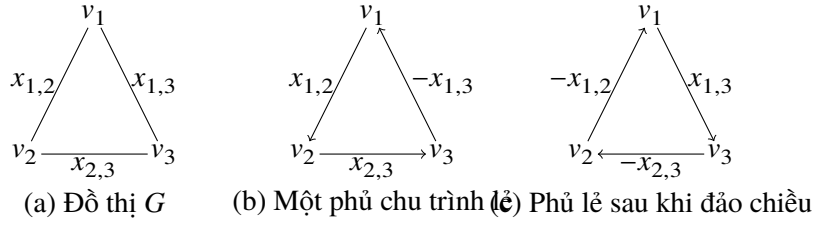
trong đó $v_1 v_{\pi(1)}, \dots, v_n v_{\pi(n)}$ là các cạnh của G . Những cạnh này tạo thành một phủ chu trình của G , tương ứng với phân rã thành chu trình của π .

Tiếp theo, nếu ta đảo chiều một chu trình bất kỳ có độ dài lớn hơn 2 trong π , ta sẽ thu được cùng một phủ chu trình. Bây giờ xét dấu của nó thay đổi ra sao. Gọi π' là hoán vị thu được sau khi đảo chiều một chu trình trong π . Việc đảo chiều một chu trình không làm đổi dấu của hoán vị π , tức $\text{sgn}(\pi) = \text{sgn}(\pi')$, nhưng làm đổi dấu mọi biến $x_{i, \pi(i)}$ trên chu trình đó.

Vì vậy, nếu ta đảo chiều một chu trình chẵn trong π , dấu của hạng tử thu được giống với dấu của hạng tử tương ứng với π ; nếu không, dấu ngược lại.

Do đó mọi phủ chu trình chứa chu trình lẻ đều bị triệt tiêu từng cặp một bởi một hạng tử dương và một hạng tử âm. Vì $\det \tilde{A}(G) \neq 0$, điều này có nghĩa là đồ thị G tồn tại một phủ chu trình chẵn, nên G có matching hoàn hảo. ■

Để hiểu trực quan hơn chứng minh này, ta lấy một chu trình ba đỉnh làm ví dụ để giải thích vì sao phủ chu trình lẻ sẽ không xuất hiện trong $\det \tilde{A}(G)$.



Hình 3: Chu trình ba đỉnh G

Ta lấy một phủ chu trình lẻ của G để phân tích. Như Hình 3(b), giả sử phủ chu trình ta chọn tương ứng với hoán vị $\pi = (2\ 3\ 1)$. Khi đó đóng góp của hạng tử này vào $\det \tilde{A}(G)$ là $\text{sgn}(\pi)x_{1,2}x_{2,3}(-x_{1,3})$. Bây giờ đảo chiều π , ta thu được Hình 3(c), tương ứng với hoán vị $\pi' = \pi^{-1} = (3\ 1\ 2)$, và đóng góp của nó vào $\det \tilde{A}(G)$ là $\text{sgn}(\pi')(-x_{1,2})(-x_{2,3})x_{1,3}$.

Rõ ràng $\text{sgn}(\pi) = \text{sgn}(\pi')$; đồng thời có thể thấy đóng góp của π' chính là đóng góp của π sau khi nhân biến tương ứng với mỗi cạnh trên chu trình với -1 . Vì vậy hai hạng tử này trái dấu nhau, và tổng đóng góp của chúng là 0.

4.2.3 Ngẫu nhiên hóa

Định lý Tutte, tức Định lý 2.1, cho ta một thuật toán quyết định rất tốt để kiểm tra đồ thị G có matching hoàn hảo hay không: chỉ cần xét $\det \tilde{A}(G)$ có bằng không hay không. Nhưng cách làm này có một vấn đề đáng kể: mỗi hạng tử của ma trận Tutte đều là một biến, và tổng số biến lên tới $|E|$, nên việc tính trực tiếp định thức rất khó, thậm chí cần thời gian mũ.

Tuy nhiên, ta không nhất thiết phải tính ra $\det \tilde{A}(G)$; việc ta cần làm chỉ là phán đoán $\det \tilde{A}(G)$ có đồng nhất bằng 0 hay không.

Quyết định một đa thức có đồng nhất bằng 0 hay không là một bài toán kinh điển. Giả sử hiện tại ta có một đa thức n biến $P(x_1, x_2, \dots, x_n)$, và muốn quyết định nó có đồng nhất bằng 0 hay không. Ta nên làm gì?

Trước hết có thể quan sát rằng nếu $P(x_1, x_2, \dots, x_n) = 0$, thì bất kể ta thay giá trị nào vào đa thức P , kết quả thu được chắc chắn đều là 0. Vậy liệu ta có thể chọn n số ngẫu nhiên $r_1, r_2, \dots, r_n$, rồi dựa vào việc $P(r_1, r_2, \dots, r_n)$ có bằng 0 hay không để suy ra $P(x_1, x_2, \dots, x_n)$ có đồng nhất bằng 0 hay không? Bổ đề Schwartz–Zippel dưới đây cho ta biết cách làm trông có vẻ không đáng tin này thực ra có xác suất cao cho kết quả đúng.

Bổ đề 2.3 (Schwartz, Zippel). Với một đa thức n biến bậc d $P(x_1, x_2, \dots, x_n)$ trên trường F , không đồng nhất bằng 0, giả sử $r_1, r_2, \dots, r_n$ là n số ngẫu nhiên trong F được chọn độc lập, khi đó

$$\Pr[P(r_1, r_2, \dots, r_n) = 0] \leq \frac{d}{|F|}.$$

Chứng minh. Ta thử chứng minh định lý này bằng quy nạp.

Xét trường hợp một biến: khi đó P có nhiều nhất d nghiệm, nên xác suất ta vừa đúng chọn vào một nghiệm không vượt quá $d/|F|$.

Bây giờ giả sử định lý đúng cho trường hợp $n - 1$ biến. Ta xét cách phân loại mọi hạng tử của P theo bậc của x_1 :

$$P(x_1, x_2, \dots, x_n) = \sum_{i=0}^d x_1^i P_i(x_2, \dots, x_n).$$

Vì $P \neq 0$, tồn tại một i sao cho $P_i \neq 0$. Xét i lớn nhất thỏa điều kiện đó, khi ấy $\deg P_i \leq d - i$.

Theo giả thiết quy nạp,

$$\Pr[P_i(r_2, \dots, r_n) = 0] \leq \frac{d-i}{|F|}.$$

Nếu $P_i(r_2, \dots, r_n) \neq 0$, thì $P(x_1, r_2, \dots, r_n)$ là một đa thức một biến bậc i , nên

$$\Pr[P(r_1, r_2, \dots, r_n) = 0 \mid P_i(r_2, \dots, r_n) \neq 0] \leq \frac{i}{|F|}.$$

Đặt sự kiện $P(r_1, r_2, \dots, r_n) = 0$ là A , và sự kiện $P_i(r_2, \dots, r_n) = 0$ là B . Khi đó

$$\begin{aligned} \Pr[A] &= \Pr[A \cap B] + \Pr[A \cap B^c] \\ &= \Pr[B] \Pr[A \mid B] + \Pr[B^c] \Pr[A \mid B^c] \\ &\leq \Pr[B] + \Pr[A \mid B^c] \\ &\leq \frac{d-i}{|F|} + \frac{i}{|F|} = \frac{d}{|F|}. \end{aligned}$$

■

Ta biết rằng với một đồ thị G cho trước, bậc của đa thức $\det \tilde{A}(G)$ là cấp $\mathcal{O}(n)$. Vì vậy ta có thể chọn một số nguyên tố p cấp n^2 , rồi trên trường modulo p , tức F_p , gán ngẫu nhiên một giá trị cho biến $x_{i,j}$ tương ứng với mỗi cạnh $v_i v_j$. Sau đó ta kiểm tra⁵ $\det \tilde{A}(G)$ có bằng 0 hay không để phán đoán G có matching hoàn hảo hay không. Theo bổ đề Schwartz–Zippel, xác suất phán đoán sai không vượt quá $\frac{n}{p} = \mathcal{O}(1/n)$; xác suất này chấp nhận được đối với chúng ta. Hơn nữa, ta có thể giảm xác suất sai bằng cách điều chỉnh kích thước của p , hoặc ngẫu nhiên nhiều lần.

Từ đó, ta có thuật toán quyết định sau.

Thuật toán 1 Lovasz’s testing algorithm

```

1: if  $\det \tilde{A}(G) \neq 0$  then
2:   print “YES”
3: else
4:   print “NO”
5: end if
```

Định lý 2.4. Độ phức tạp thời gian của Thuật toán 1 là $\mathcal{O}(n^\omega)$.

4.3 Xây dựng phương án matching hoàn hảo

4.3.1 Một thuật toán vét cạn

Theo Thuật toán 1, hiện tại ta đã có thể quyết định đồ thị G có matching hoàn hảo hay không. Từ đó ta lập tức thu được một thuật toán vét cạn để xây dựng matching: duyệt từng cạnh, xóa hai đầu mút của nó, rồi kiểm tra đồ thị còn lại có matching hoàn hảo hay không.

⁵Từ đây trở đi, khi cần tính định thức, ma trận nghịch đảo, hoặc khử ma trận $\tilde{A}(G)$, ta đều hiểu là sau khi đã gán ngẫu nhiên các biến thì thực hiện phép toán tương ứng trên F_p .

Thuật toán 2 A naive matching algorithm

```

1:  $M \leftarrow \emptyset$ 
2: for  $i \leftarrow 1$  to  $n$  do
3:   for  $j \leftarrow 1$  to  $n$  do
4:     if  $v_i v_j \in E(G)$  and  $\det \tilde{A}(G - \{v_i, v_j\}) \neq 0$  then
5:        $G \leftarrow G - \{v_i, v_j\}$ 
6:        $M \leftarrow M \cup \{v_i v_j\}$ 
7:     end if
8:   end for
9: end for
10: return  $M = 0$ 

```

Thuật toán 2 cần thực hiện $\mathcal{O}(n^2)$ vòng lặp; trong mỗi vòng lặp cần tính định thức của một ma trận vuông cấp $\mathcal{O}(n)$. Do đó:

Định lý 3.1. Độ phức tạp thời gian của Thuật toán 2 là $\mathcal{O}(n^{\omega+2})$.

4.3.2 Một số kiến thức đại số tuyến tính

Nút thắt của Thuật toán 2 nằm ở việc cần phán đoán mỗi cạnh có thể nằm trong matching hoàn hảo hay không. Nếu ta có thể tăng tốc quá trình phán đoán này, ta sẽ giảm được độ phức tạp của thuật toán.

Để đạt mục tiêu đó, trước hết ta cần biết một số kiến thức đại số tuyến tính.

Định nghĩa 3.1. Phần phụ của ma trận A theo hàng i và cột j , ký hiệu $M_{i,j}$, là $\det A^{(i), (j)}$.

Định nghĩa 3.2. Đại số phụ của ma trận A theo hàng i và cột j , ký hiệu $C_{i,j}$, là $(-1)^{i+j} M_{i,j}$.

Ma trận các đại số phụ của A là một ma trận $n \times n$ C , thỏa mãn phần tử ở hàng i , cột j của nó là đại số phụ của A theo hàng i và cột j .

Định nghĩa 3.3. Ma trận phụ hợp $\text{adj } A$ của ma trận A là chuyển vị của ma trận các đại số phụ của A , tức $\text{adj } A = C^T$.

Định lý 3.2 (Khai triển Laplace). Giả sử A là một ma trận $n \times n$. Khi đó với một hàng bất kỳ $i \in \{1, \dots, n\}$, ta có

$$\det A = \sum_{j=1}^n A_{i,j} C_{i,j} = \sum_{j=1}^n A_{i,j} (\text{adj } A)_{j,i}.$$

Tương tự, với một cột bất kỳ $j \in \{1, \dots, n\}$, ta có

$$\det A = \sum_{i=1}^n A_{i,j} (\text{adj } A)_{j,i}.$$

Định lý 3.3. Nếu ma trận A khả nghịch, thì

$$A^{-1} = \text{adj } A / \det A.$$

Chứng minh. Theo Định nghĩa 3.2 và Định nghĩa 3.3, hệ số ở hàng i , cột i của $A \times \text{adj } A$ là $\sum_{j=1}^n A_{i,j} C_{i,j}$. Theo Định lý 3.2, hệ số này bằng $\det A$.

Nếu $i \neq j$, thì hệ số ở hàng i , cột j của $A \times \text{adj } A$ là $\sum_{k=1}^n A_{i,k} C_{j,k}$. Thực ra, điều này tương đương với việc thay các phần tử hàng j của A bằng các phần tử hàng i , rồi lấy định thức. Vì có hai hàng giống nhau, định thức bằng 0.

Vì vậy ta có

$$A \times \text{adj } A = \text{adj } A \times A = \det A \times I_n,$$

tức là

$$A^{-1} = \text{adj } A / \det A.$$

■

4.3.3 Tính chất của ma trận phản đối xứng

Trước hết ta định nghĩa khái niệm ma trận phản đối xứng.

Định nghĩa 3.4. Với một ma trận $n \times n$ A , nếu $A^T = -A$, tức $A_{i,j} = -A_{j,i}$, thì ta gọi A là một ma trận phản đối xứng.

Ma trận Tutte trong Định nghĩa 2.1 chính là một ma trận phản đối xứng. Có thể thấy các thuật toán phía trước của ta đều xoay quanh việc tính định thức của ma trận Tutte; đây chính là lý do ta nghiên cứu tính chất của ma trận phản đối xứng.

Bổ đề 3.4. Với một ma trận phản đối xứng $n \times n$ A , nếu n lẻ thì $\det A = 0$.

Chứng minh. Từ kiến thức về định thức, ta biết $\det A = \det A^T$. Lại vì $A^T = -A$, nên $\det A = \det(-A) = (-1)^n \det A$.

Khi n lẻ, $(-1)^n = -1$, tức $\det A = -\det A$, do đó $\det A = 0$.

■

Định lý 3.5. Với một ma trận phản đối xứng $n \times n$ A và một tập chỉ số hàng $I = \{i_1, i_2, \dots, i_k\}$, nếu $A_{i_1, \cdot}, A_{i_2, \cdot}, \dots, A_{i_k, \cdot}$ là một nhóm độc lập tuyến tính cực đại theo hàng của A , thì $\det A_{I,I} \neq 0$.

Chứng minh. Không mất tính tổng quát, giả sử $I = \{1, 2, \dots, k\}$.

Vì $A_{1, \cdot}, A_{2, \cdot}, \dots, A_{k, \cdot}$ là một nhóm độc lập tuyến tính cực đại theo hàng của A , từ tính phản đối xứng của A , ta có $A_{\cdot, 1}, A_{\cdot, 2}, \dots, A_{\cdot, k}$ là một nhóm độc lập tuyến tính cực đại theo cột của A .

Do đó với mọi $k < i \leq n$, $A_{\cdot, i}$ đều có thể được biểu diễn tuyến tính bởi $A_{\cdot, 1}, A_{\cdot, 2}, \dots, A_{\cdot, k}$. Vì vậy

$$\text{rank } A_{I,I} = \text{rank } A_{I, \cdot} = k,$$

tức $\det A_{I,I} \neq 0$.

■

Hệ quả 3.6. Với một ma trận phản đối xứng A , tồn tại một tập chỉ số hàng $I = \{i_1, i_2, \dots, i_k\}$ sao cho

$$\text{rank } A = \text{rank } A_{I,I} = k.$$

Định lý 3.7. Hạng của một ma trận phản đối xứng là số chẵn.

Chứng minh. Theo Hệ quả 3.6, một ma trận phản đối xứng tồn tại một định thức con chính đầy hạng có hạng bằng hạng của chính nó. Vì định thức con chính của ma trận phản đối xứng cũng là phản đối xứng, kết hợp với Bổ đề 3.4 ta biết hạng của định thức con chính này là số chẵn. Do đó hạng của một ma trận phản đối xứng là số chẵn. ■

Bổ đề 3.8. Với một ma trận phản đối xứng khả nghịch $n \times n$ A , và hai chỉ số bất kỳ $i, j \in \{1, \dots, n\}, i \neq j$, ta có

$$\det A^{(i),\{j\}} \neq 0 \iff \det A^{\{i,j\},\{i,j\}} \neq 0.$$

Chứng minh. Nếu $\det A^{(i),\{j\}} \neq 0$, thì $\text{rank } A^{(i),\{j\}} = n - 1$. Vì $A^{\{i,j\},\{i,j\}}$ thu được từ $A^{(i),\{j\}}$ bằng cách xóa thêm một hàng và một cột, nên $\text{rank } A^{\{i,j\},\{i,j\}} \geq n - 3$. Lại vì $A^{\{i,j\},\{i,j\}}$ là ma trận phản đối xứng, theo Định lý 3.7, hạng của nó là số chẵn, nên $\text{rank } A^{\{i,j\},\{i,j\}}$ nhất định bằng $n - 2$. Do đó $\det A^{\{i,j\},\{i,j\}} \neq 0$.

Ngược lại, nếu $\det A^{\{i,j\},\{i,j\}} \neq 0$, thì $\text{rank } A^{\{i,j\},\{i,j\}} = n - 2$; đồng thời $\text{rank } A^{(i),\{j\}} = n - 2$. Theo Định lý 3.7,

$$\text{rank } A^{(i),\{i,j\}} = \text{rank } A^{(i),\{i\}} = n - 2.$$

Điều này cho thấy cột thứ j của $A^{(i),\emptyset}$ là tổ hợp tuyến tính của các cột khác. Vì vậy $\text{rank } A^{(i),\{j\}} = \text{rank } A^{(i),\emptyset} = n - 1$, suy ra $\det A^{(i),\{j\}} \neq 0$. ■

4.3.4 Thuật toán Rabin–Vazirani

Bây giờ ta thử thu được một thuật toán xây dựng phương án matching tốt hơn Thuật toán 2.

Định lý 3.9 (Rabin, Vazirani). Giả sử $G = (V, E)$ là một đồ thị có matching hoàn hảo, và $\tilde{A} = \tilde{A}(G)$ là ma trận Tutte tương ứng của G . Khi đó

$$(\tilde{A}^{-1})_{i,j} \neq 0 \iff G - \{v_i, v_j\} \text{ có matching hoàn hảo.}$$

Chứng minh. Định lý này có thể được suy ra từ Định lý 3.3, Bổ đề 3.8 và định lý Tutte, tức Định lý 2.1. ■

Dựa trên Định lý 3.9, ta thu được thuật toán sau.

Thuật toán 3 Matching algorithm of Rabin and Vazirani

```

1:  $M \leftarrow \emptyset$ 
2: for  $i \leftarrow 1$  to  $n$  do
3:   if  $v_i$  is not yet matched then
4:     compute  $\tilde{A}(G)^{-1}$ 
5:     find  $j$ , such that  $v_i v_j \in E(G)$  and  $(\tilde{A}(G)^{-1})_{j,i} \neq 0$ 
6:      $G \leftarrow G - \{v_i, v_j\}$ 
7:      $M \leftarrow M \cup \{v_i v_j\}$ 
8:   end if
9: end for
10: return  $M = 0$ 
```

Thuật toán 3 cần thực hiện tổng cộng $\mathcal{O}(n)$ vòng lặp; trong mỗi vòng lặp cần tìm nghịch đảo của một ma trận vuông cấp $\mathcal{O}(n)$. Do đó:

Định lý 3.10. Độ phức tạp của Thuật toán 3 là $\mathcal{O}(n^{\omega+1})$.

4.3.5 Phương pháp xây dựng dựa trên khử Gauss

Nút thắt của Thuật toán 3 nằm ở việc tính nghịch đảo của ma trận Tutte $\tilde{A}(G)$. Thực ra, trong mỗi vòng lặp ta chỉ xóa hai hàng và hai cột của $\tilde{A}(G)$, nhưng lại cần tính lại ma trận nghịch đảo từ đầu. Nếu mỗi lần không cần tính lại nghịch đảo từ đầu, mà có thể dùng một phương pháp hiệu quả để duy trì ma trận nghịch đảo, ta sẽ giảm được độ phức tạp của thuật toán này.

Thuật toán của ta dựa trên định lý sau.

Định lý 3.11. Nếu

$$A = \begin{pmatrix} a_{1,1} & v^T \\ u & B \end{pmatrix}, \quad A^{-1} = \begin{pmatrix} \hat{a}_{1,1} & \hat{v}^T \\ \hat{u} & \hat{B} \end{pmatrix},$$

trong đó $\hat{a}_{1,1} \neq 0$, thì

$$B^{-1} = \hat{B} - \hat{u}\hat{v}^T/\hat{a}_{1,1}.$$

Chứng minh. Vì $AA^{-1} = I$, ta có

$$\begin{pmatrix} a_{1,1}\hat{a}_{1,1} + v^T\hat{u} & a_{1,1}\hat{v}^T + v^T\hat{B} \\ u\hat{a}_{1,1} + B\hat{u} & u\hat{v}^T + B\hat{B} \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & I_{n-1} \end{pmatrix}.$$

Từ đó,

$$\begin{aligned} B(\hat{B} - \hat{u}\hat{v}^T/\hat{a}_{1,1}) &= I_{n-1} - u\hat{v}^T - B\hat{u}\hat{v}^T/\hat{a}_{1,1} \\ &= I_{n-1} - u\hat{v}^T + u\hat{a}_{1,1}\hat{v}^T/\hat{a}_{1,1} \\ &= I_{n-1} - u\hat{v}^T + u\hat{v}^T \\ &= I_{n-1}. \end{aligned}$$

■

Chú ý rằng trong định lý trên, thao tác sửa $\hat{B}$ đúng là một thao tác khử trong khử Gauss. Trong định lý, ta chứng minh với ví dụ khử hàng thứ nhất và cột thứ nhất.

Là một ứng dụng trực tiếp của Định lý 3.11, ta thu được thuật toán sau.

Thuật toán 4 Simple matching algorithm

- 1: $M \leftarrow \emptyset$
- 2: compute $\tilde{A}(G)^{-1}$
- 3: **for** $i \leftarrow 1$ **to** n **do**
- 4: **if** the i -th row is not yet eliminated **then**
- 5: find j such that $v_i v_j \in E(G)$ and $(\tilde{A}(G)^{-1})_{j,i} \neq 0$
- 6: $G \leftarrow G - \{v_i, v_j\}$
- 7: $M \leftarrow M \cup \{v_i v_j\}$
- 8: update $\tilde{A}(G)^{-1}$ by eliminating the i -th row and the j -th column and then the j -th row and the i -th column
- 9: **end if**
- 10: **end for**
- 11: **return** $M = 0$

Thuật toán 4 cần lặp $\mathcal{O}(n)$ lần; trong mỗi vòng lặp cần thực hiện hai thao tác khử. Độ phức tạp thời gian của một thao tác khử là $\mathcal{O}(n^2)$. Vì vậy:

Định lý 3.12. Độ phức tạp thời gian của Thuật toán 4 là $\mathcal{O}(n^3)$.

4.4 Matching cực đại

4.4.1 Kích thước của matching cực đại

Đến đây, mọi thuật toán của ta đều dùng để giải matching hoàn hảo, nhưng trong ứng dụng thực tế, bài toán ta cần giải thường là matching cực đại.

Tương tự ý tưởng giải bài toán matching hoàn hảo phía trước, trước hết ta bắt đầu từ bài toán đơn giản nhất: với một đồ thị cho trước $G = (V, E)$, làm thế nào tìm được kích thước matching cực đại $\nu(G)$ của G ? Với bài toán này, ta có định lý sau.

Định lý 4.1. Kích thước matching cực đại $\nu(G)$ của đồ thị G là

$$\frac{1}{2} \text{rank } \tilde{A}(G).$$

Chứng minh. Đặt $k = \text{rank } \tilde{A}(G)$. Theo Hệ quả 3.6, ta có thể chọn một tập chỉ số hàng $I = \{i_1, i_2, \dots, i_k\}$ của $\tilde{A}(G)$ sao cho

$$\text{rank } \tilde{A}(G) = \text{rank } \tilde{A}(G)_{I,I} = k.$$

Đặt $U = \{v_{i_1}, v_{i_2}, \dots, v_{i_k}\}$. Khi đó $\tilde{A}(G)_{I,I} = \tilde{A}(G[U])$. Vì $\text{rank } \tilde{A}(G[U]) = |U|$, đồ thị $G[U]$ tồn tại matching hoàn hảo; điều này cho thấy $\nu(G) \geq k/2$.

Nếu $\nu(G) > k/2$, ta xét lấy ra mọi đỉnh nằm trong một matching cực đại. Khi đó nhất định tồn tại một tập chỉ số hàng I sao cho $|I| > k$ và $\det \tilde{A}(G)_{I,I} \neq 0$. Điều này có nghĩa là hạng của một ma trận con của $\tilde{A}(G)$ lớn hơn hạng của chính nó, hiển nhiên là không thể.

Tổng hợp lại, $\nu(G) = k/2$. ■

4.4.2 Chuyển hóa thứ nhất

Tiếp theo ta thử chuyển bài toán matching cực đại về bài toán matching hoàn hảo quen thuộc. Nếu với một đồ thị G , ta đã tìm được kích thước matching cực đại $\nu(G)$ của nó, thì ta có thể thêm $n - 2\nu(G)$ đỉnh mới vào G , rồi nối cạnh từ mỗi đỉnh mới đến từng đỉnh trong n đỉnh ban đầu. Đồ thị mới thu được khi đó là một đồ thị có matching hoàn hảo, và matching hoàn hảo của đồ thị mới tương ứng với matching cực đại của đồ thị ban đầu.

Như vậy ta đã giải được bài toán matching cực đại. Chuyển hóa này có tính tổng quát rất tốt, nhưng có một nhược điểm: trong trường hợp xấu nhất, ta có thể phải thêm n đỉnh mới, làm kích thước dữ liệu, tức số đỉnh, tăng gấp đôi, nên hằng số của thuật toán sẽ tăng đáng kể.

4.4.3 Chuyển hóa thứ hai

Ta xét một cách làm khác, cũng chuyển bài toán matching cực đại về bài toán matching hoàn hảo, nhưng lần này thực hiện bằng cách xóa đỉnh. Với một đồ thị $G = (V, E)$ và một matching cực đại M của nó, xét tập đỉnh U gồm mọi đỉnh nằm trong matching M . Rõ ràng $G[U]$ có matching hoàn hảo, và matching hoàn hảo của $G[U]$ chính là matching cực đại của G . Ý tưởng của ta là tìm một tập đỉnh U như vậy, rồi áp dụng thuật toán giải matching hoàn hảo phía trước cho $G[U]$.

Thực ra, quá trình chứng minh Định lý 4.1 đã chỉ đường cho ta. Từ Bổ đề 3.5 và Hệ quả 3.6, ta chỉ cần tìm một nhóm độc lập tuyến tính cực đại theo hàng của $\tilde{A}(G)$. Điều này có thể được tìm bằng một lượt khử Gauss. Vì vậy, ta có thể chuyển bài toán matching cực đại về matching hoàn hảo trong độ phức tạp $\mathcal{O}(n^\omega)$, và nó không làm tăng hằng số của thuật toán.

4.5 Ứng dụng

Thuật toán được trình bày trong bài là thuật toán tổng quát để giải bài toán matching cực đại trên đồ thị tổng quát, nên tất nhiên nó có thể giải mọi bài cần dùng matching tổng quát.

Ngoài ra, nó còn có một số ứng dụng khá đặc sắc. Dưới đây ta xét hai ví dụ.

4.5.1 Vertex Geography⁶

Mô tả bài toán. Alice và Bob chơi một trò chơi trên một đồ thị vô hướng $G = (V, E)$. Hai người luân phiên thao tác, Alice đi trước.

Ban đầu có một quân cờ đặt trên một đỉnh nào đó. Sau đó, trong mỗi lượt, người chơi có thể di chuyển quân cờ này sang một đỉnh kề và chưa từng đi tới trước đó, tính cả điểm xuất phát. Người không thể thao tác sẽ thua. Hỏi những đỉnh nào là đỉnh thắng cho người đi trước?

Giới thiệu thuật toán. Trước hết ta có một kết luận: một đỉnh v_i là trạng thái thắng khi và chỉ khi nó nhất định nằm trong matching cực đại của đồ thị G .⁷

Vì vậy bài toán chuyển thành phán đoán một đỉnh nào đó có nhất định nằm trong matching cực đại của G hay không. Ta làm theo Mục 4.2: thêm $k = n - 2\nu(G)$ đỉnh mới và nối các cạnh tương ứng. Gọi đồ thị mới thu được là G' , các đỉnh mới có chỉ số lần lượt là $n+1, n+2, \dots, n+k$. Sau đó tùy ý chọn một đỉnh mới v_j ($n+1 \leq j \leq n+k$). Khi đó với mỗi đỉnh v_i của đồ thị ban đầu ($1 \leq i \leq n$), nếu $v_i v_j$ có thể xuất hiện trong một matching hoàn hảo của G' , thì v_i không nhất định nằm trong matching cực đại của G .

Theo Định lý 3.9, ta chỉ cần kiểm tra $(\tilde{A}(G')^{-1})_{i,j}$ có bằng không hay không.

Có thể thấy thuật toán matching được giới thiệu trong bài có ưu thế trong các bài toán kiểu “một đỉnh có nhất định nằm trong matching cực đại hay không” và “một cạnh có nhất định nằm trong matching cực đại hay không”. Lý do là thuật toán này tránh được việc phân tích đường tăng, mà chỉ cần kiểm tra một vị trí nào đó của ma trận nghịch đảo có bằng không hay không.

4.5.2 Matching trọng số cực đại

Mô tả bài toán. Matching trọng số cực đại là bài toán trong đó mỗi cạnh $v_i v_j$ của đồ thị G mang một trọng số $w_{i,j}$, và mục tiêu hiện tại của ta là tìm một matching M có tổng trọng số cạnh lớn nhất. Matching cực đại tương đương với matching trọng số cực đại khi mọi trọng số cạnh đều bằng 1.

Giới thiệu thuật toán. Ta có thể nối cạnh trọng số 0 giữa mọi cặp đỉnh; khi n lẻ, thêm một đỉnh mới và nối nó với mọi đỉnh. Vì vậy mỗi matching có trọng số của đồ thị ban đầu sẽ tương ứng với ít nhất một matching hoàn hảo có trọng số của đồ thị mới; và một matching hoàn hảo có trọng số của đồ thị mới tương ứng với một matching có trọng số của đồ thị ban đầu.

Bây giờ ta chuyển thành bài toán tìm matching hoàn hảo có trọng số của đồ thị mới G' . Ta thêm một biến y . Đặt

$$\tilde{A}'(G')_{i,j} = \tilde{A}(G')_{i,j} \times y^{w_{i,j}}.$$

Như vậy, điều ta cần tìm tương đương với bậc cao nhất của y trong $\det \tilde{A}'(G')$; giá trị này chia cho 2 chính là matching trọng số cực đại.

⁶Bài toán kinh điển này có tên tiếng Anh là Undirected Vertex Geography (UVG).

⁷Chứng minh kết luận này không phải trọng tâm của bài viết, nên được lược bỏ ở đây; độc giả quan tâm có thể tham khảo các tài liệu liên quan.

Trước hết, ta gán một giá trị ngẫu nhiên cho mọi $x_{i,j}$. Sau đó giả sử tổng trọng số của mọi cạnh không vượt quá W . Ta có thể lần lượt thay mọi giá trị từ 0 đến W vào y , tính một lần định thức, rồi dùng $W + 1$ kết quả này để nội suy một đa thức theo y . Khi đó bậc cao nhất của đa thức này là thứ ta cần tìm.

Độ phức tạp thời gian của thuật toán này là $\mathcal{O}(Wn^\omega + W^2)$; khi cả W và n đều nhỏ, nó có giá trị nhất định.

4.6 Tổng kết

Đến đây, ta đã có thể tìm kích thước matching cực đại trên đồ thị tổng quát trong độ phức tạp $\mathcal{O}(n^\omega)$, và xây dựng một phương án matching trong độ phức tạp $\mathcal{O}(n^3)$.

Thuật toán được giới thiệu trong bài vẫn còn tiềm năng rất lớn đang chờ chúng ta khám phá, và các ứng dụng được giới thiệu trong bài còn khá cơ bản. Vì vậy tôi hy vọng bài viết này có thể đóng vai trò gợi mở, mong rằng có độc giả sẽ mở rộng thuật toán trong bài, phát triển nó hơn nữa và thu được những ứng dụng thú vị hơn.

Lời cảm ơn

- Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi.
- Cảm ơn thầy Song Xinbo và thầy Zhuo Mingcong đã quan tâm và chỉ dẫn trong nhiều năm.
- Cảm ơn huấn luyện viên đội tuyển quốc gia Zhang Ruizhe và Yu Linyun đã chỉ dẫn.
- Cảm ơn bạn Zhou Zixin đã chia sẻ thuật toán này với tôi.
- Cảm ơn các bạn Gao Wenyuan và Wu Hongxun đã đọc duyệt bài viết này.
- Cảm ơn cha mẹ đã quan tâm và chăm sóc tôi.

Tài liệu tham khảo

- [1] Mucha M, Sankowski P. Maximum matchings via Gaussian elimination[C]//Foundations of Computer Science, 2004. Proceedings. 45th Annual IEEE Symposium on. IEEE, 2004: 248–255.
- [2] Ivan I, Virza M, Yuen H. Algebraic Algorithms for Matching[J]. 2011.
- [3] Cheung H Y. Algebraic algorithms in combinatorial optimization[D]. The Chinese University of Hong Kong, 2011.
- [4] Huang C C, Kavitha T. Efficient algorithms for maximum weight matchings in general graphs with small edge weights[C]//Proceedings of the twenty-third annual ACM-SIAM symposium on Discrete Algorithms. Society for Industrial and Applied Mathematics, 2012: 1400–1412.
- [5] Chen Yinbo. Bàn sơ về thuật toán matching trên đồ thị và ứng dụng của nó[C]//Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2015. 2015.

5 Tính tổng đa thức: báo cáo ra đề

Yuan Yutao, Trường Trung học Yali, thành phố Trường Sa

5.1 Nội dung bài toán

Cho n, m, a, b, c . Định nghĩa $f_0(x) = 1$; với $k > 0$,

$$f_k(x) = \sum_{i=0}^x (ai^2 + bi + c)f_{k-1}(i).$$

Với mọi số nguyên i thỏa mãn $0 \leq i < n$, hãy tính giá trị $f_i(m)$ modulo 1004535809.

5.1.1 Giới hạn dữ liệu

Mã dữ liệu	n	m	a	b	c
0	2000	≤ 2000	0	0	1
1	2000	≤ 2000	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$
2	300	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$
3	1000	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$
4	2000	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$
5	3000	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$
6	50000	$\leq 10^9$	0	0	$\leq 10^9$
7	250000	$\leq 10^9$	0	0	$\leq 10^9$
8	50000	≤ 2000	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$
9	250000	≤ 2000	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$
10	50000	≤ 50000	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$
11	250000	≤ 50000	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$
12	50000	$\leq 10^9$	0	1	0
13	50000	$\leq 10^9$	0	$\leq 10^9$	$\leq 10^9$
14	250000	$\leq 10^9$	0	1	0
15	250000	$\leq 10^9$	0	$\leq 10^9$	$\leq 10^9$
16	50000	$\leq 10^9$	1	0	0
17	50000	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$
18	250000	$\leq 10^9$	1	0	0
19	250000	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$

5.2 Giới thiệu thuật toán

5.2.1 Thuật toán một

Quy hoạch động trực tiếp theo định nghĩa.

Độ phức tạp thời gian là $\mathcal{O}(nm)$. Điểm kỳ vọng: 10–15 điểm.

5.2.2 Thuật toán hai

Tổng tiền tố của một đa thức $f(x)$ vẫn là một đa thức, và có thể được tìm trong $\mathcal{O}(n^2)$ hoặc $\mathcal{O}(n \log n)$ thời gian. Ta lần lượt tìm mọi đa thức $f_i(x)$.

Độ phức tạp thời gian là $\mathcal{O}(n^3)$ hoặc $\mathcal{O}(n^2 \log n)$. Dùng đồng thời với thuật toán một, điểm kỳ vọng là 15–30 điểm.

5.2.3 Thuật toán ba

Bậc của đa thức $f_k(x)$ không vượt quá $3k$, nên chỉ cần truy hồi để tìm $f_k(x)$ với $0 \leq x \leq 3k$, rồi nội suy để tìm đáp án.

Độ phức tạp thời gian là $\mathcal{O}(n^2)$. Dùng đồng thời với thuật toán một, điểm kỳ vọng là 30–35 điểm.

5.2.4 Thuật toán bốn

Khi $a = b = 0$, ta có

$$f_k(x) = \binom{x+k}{k} c^k.$$

Có thể tính bằng truy hồi.

Độ phức tạp thời gian là $\mathcal{O}(n)$. Dùng đồng thời với thuật toán ba, điểm kỳ vọng là 40–45 điểm.

5.2.5 Thuật toán năm

Đặt

$$g_k(x) = \sum_{i=0}^{\infty} f_i(k).$$

Từ định nghĩa, khi $k > 0$ ta có

$$g_k(x) = (ak^2 + bk + c)xg_k(x) + g_{k-1}(x),$$

tức là

$$g_k(x) = \frac{g_{k-1}(x)}{1 - (ak^2 + bk + c)x}.$$

Đồng thời

$$g_0(x) = \frac{1}{1 - cx}.$$

Do đó

$$g_k(x) = \prod_{i=0}^k \frac{1}{1 - (ai^2 + bi + c)x}.$$

Tính mẫu số bằng vét cạn hoặc chia để trị, với độ phức tạp thời gian $\mathcal{O}(m^2)$ hoặc $\mathcal{O}(m \log^2 m)$. Sau đó tìm nghịch đảo đa thức để thu được đáp án.

Độ phức tạp thời gian là $\mathcal{O}(m^2 + n \log n)$ hoặc $\mathcal{O}(m \log^2 m + n \log n)$. Điểm kỳ vọng: 45–60 điểm.

5.2.6 Thuật toán sáu

Đặt

$$E(x) = \prod_{i=0}^k (1 - (ai^2 + bi + c)x) = \sum_{i=0}^{k+1} (-1)^{k-i} e_{k-i} x^k.$$

Đặt

$$p_i = \sum_{j=0}^k (aj^2 + bj + c)^i.$$

Từ đồng nhất thức Newton, ta có

$$e_i = \frac{1}{i} \sum_{j=1}^i (-1)^{j-1} e_{i-j} p_j.$$

Đặt

$$P(x) = \sum_{i=0}^{\infty} (-1)^i p_{i+1}.$$

Khi đó thu được

$$E'(x) = E(x)P(x).$$

Có thể dùng FFT chia để trị để tìm $E(x)$: mỗi lần dùng nửa trước của các hệ số chập với $P(x)$ để cập nhật nửa sau của các hệ số.

Tiếp theo xét cách tìm p_i . Ta chia làm hai trường hợp.

Khi $a = 0, b \neq 0$, đặt $u = \frac{c}{b}$. Khi đó

$$p_i = b^i \sum_{j=0}^k (j+u)^i = b^i \left(\sum_{j=0}^{k+u} j^i - \sum_{j=0}^{u-1} j^i \right).$$

Có thể dùng các số Bernoulli để tính.

Khi $a \neq 0$, đặt $u = \frac{b}{2a}, v = \frac{c}{a}$. Khi đó

$$\begin{aligned} p_i &= a^i \sum_{j=0}^k (j^2 + 2uj + v)^i \\ &= a^i \sum_{j=0}^k ((j+u)^2 + (v-u^2))^i \\ &= a^i \sum_{t=0}^i \binom{i}{t} (v-u^2)^{i-t} \sum_{j=0}^k (j+u)^{2t}. \end{aligned}$$

Có thể tìm $\sum_{j=0}^k (j+u)^{2t}$ tương tự trường hợp trước, rồi cộng để thu được đáp án.

Độ phức tạp thời gian là $\mathcal{O}(n \log^2 n)$. Điểm kỳ vọng: 80 điểm.

5.2.7 Thuật toán bảy

Từ $E'(x) = E(x)P(x)$, $E(0) = 1$, ta giải được

$$E(x) = \exp \left(\int_0^x P(t) dt \right).$$

Vì vậy có thể tìm $E(x)$ bằng một lần lấy exp đa thức.

Độ phức tạp thời gian là $\mathcal{O}(n \log n)$. Điểm kỳ vọng: 90–100 điểm.

5.3 Ý tưởng ra đề

Tính tổng tiền tố của một dãy đa thức là một bài toán rất kinh điển. Nếu liên tục tính tổng tiền tố nhiều lần trên một dãy đa thức, ta có thể dùng tổ hợp để tìm công thức tổng quát. Vì vậy, một mở rộng tự nhiên là thực hiện những phép khác giữa hai lần lấy tổng tiền tố; trong bài này, phép đó chính là nhân với một dãy đa thức khác. Vì tôi không tìm được thuật toán hiệu quả để tìm công thức tổng quát, nên đã thay đổi nội dung truy vấn và thu được bài toán này.

5.4 Tổng kết

Bài này chủ yếu kiểm tra mức độ hiểu biết của thí sinh về hàm sinh, khả năng vận dụng các thuật toán liên quan đến đa thức, cũng như các kiến thức toán học như đồng nhất thức Newton và vi tích phân cơ bản.

Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp cơ hội học tập và trao đổi.

Cảm ơn thầy Wang Xingming và thầy Zhu Quanmin đã dạy dỗ tôi.

Cảm ơn các bạn học đã giúp đỡ tôi.

Tài liệu tham khảo

- [1] Jin Ce. Phép toán hàm sinh và bài toán đếm tổ hợp[C]//Luận văn đội tuyển quốc gia năm 2015. 2015.
- [2] Peng Yuxiang. “Inverse Element of Polynomial”. <http://picks.logdown.com/posts/189620-inverse-element-of-polynomial>
- [3] Peng Yuxiang. “Newton’s Method of Polynomial”. <http://picks.logdown.com/posts/209226-newtons-method-of-polynomial>
- [4] Liu Rujia. *Nhập môn kinh điển về thi thuật toán*. Tsinghua University Press.

6 Bàn về bài toán tập độc lập trong thi tin học

Zhong Zhixian, Trường Trung học số 1 Phúc Châu, tỉnh Phúc Kiến

Tóm tắt

Tìm thuật toán tốt hơn cho các bài toán NP-hard là một nội dung quan trọng của tin học. Trong đó, các bài toán liên quan đến tập độc lập khá thường gặp và cũng nhiều lần xuất hiện trong thi tin học. Bài viết này bắt đầu từ các tính chất của tập độc lập, rồi giới thiệu vài lớp thuật toán tập độc lập dựa trên nhiều tư tưởng khác nhau. Với đồ thị tổng quát, trên cơ sở thuật toán tìm kiếm, bài viết đề xuất thuật toán tối ưu bằng quy hoạch động, đồng thời phân tích và kiểm thử hiệu suất của vài thuật toán trên dữ liệu ngẫu nhiên. Với đồ thị đặc biệt, bài viết giới thiệu hai tư tưởng thuật toán cho đồ thị đặc biệt, đề xuất tư tưởng giải bài toán tập độc lập trên “đồ thị k -xương rồng” và các ứng dụng mở rộng của nó.

6.1 Lời nói đầu

Không ít bài toán liên quan đến tập độc lập, như tập độc lập lớn nhất, tập độc lập có trọng số lớn nhất, đếm tập độc lập, đều là các bài toán NP-hard kinh điển trong lý thuyết đồ thị và xuất hiện rộng rãi trong thi tin học. Tuy nhiên hiệu suất của các thuật toán giải bài toán tập độc lập thường không cao. Vì vậy, tôi đã nghiên cứu sâu hơn lớp bài toán này, với hy vọng dùng được phương pháp hiệu quả hơn để giải chúng.

Nghiên cứu trong bài viết này chia làm hai phần. Phần thứ nhất giới thiệu hai thuật toán giải bài toán tập độc lập trên đồ thị tổng quát: thuật toán tập độc lập dựa trên tìm kiếm tập độc lập cực đại và thuật toán tập độc lập dựa trên quy hoạch động. Phần thứ hai giới thiệu thuật toán tập độc lập cho hai loại đồ thị đặc biệt, lần lượt dựa trên tư tưởng matching đồ thị và tư tưởng chia giai đoạn trên đồ thị.

6.2 Định nghĩa và quy ước

Bài toán tập độc lập có nhiều dạng. Để tiện mô tả, dưới đây đưa ra một số định nghĩa.

Định nghĩa 2.1. Với đồ thị vô hướng $G = (V, E)$ và hai đỉnh $u, v \in V$, nếu $(u, v) \in E$ thì gọi u, v là **kề nhau** (adjacent). **Lân cận** (neighborhood) của đỉnh $v \in V$ được định nghĩa là tập các đỉnh trong V kề với v , ký hiệu $N(v)$. Ngoài ra, $N_G(v)$ biểu thị lân cận của v trong đồ thị G .

Định nghĩa 2.2. **Bậc** (degree) của đỉnh v , ký hiệu $\deg(v)$, được định nghĩa là kích thước của $N(v)$, tức $\deg(v) = |N(v)|$. Ngoài ra, $\deg_G(v)$ biểu thị bậc của v trong đồ thị G .

Định nghĩa 2.3. Một **tập độc lập** (independent set) của đồ thị vô hướng $G = (V, E)$ được định nghĩa là một tập con của V sao cho mọi cặp đỉnh trong tập con đó đều không kề nhau. Nói hình thức, I là một tập độc lập của G khi và chỉ khi $I \subseteq V$ và với mọi $u, v \in I$, $(u, v) \notin E$.

Định nghĩa 2.4. Một **tập độc lập lớn nhất** (maximum independent set) của đồ thị vô hướng $G = (V, E)$ là một tập độc lập I trong G có số đỉnh $|I|$ lớn nhất.

Định nghĩa 2.5. **Số độc lập** (independence number) của đồ thị vô hướng $G = (V, E)$ được định nghĩa là số đỉnh $|I|$ của một tập độc lập lớn nhất I của G , ký hiệu $\alpha(G)$.

Định nghĩa 2.6. **Đồ thị con cảm sinh** (induced subgraph)⁸ của đồ thị vô hướng $G = (V, E)$ trên $S \subseteq V$ được định nghĩa là đồ thị có tập đỉnh là S và tập cạnh gồm các cạnh có cả hai đầu mút nằm trong S , ký hiệu $G[S]$.

Mọi bài toán trong bài này đều lấy bài toán tập độc lập lớn nhất làm ví dụ: với đồ thị vô hướng $G = (V, E)$ cho trước, hãy tìm một tập độc lập lớn nhất I của G .

Để tiện, quy ước n là bậc của đồ thị G (tức số đỉnh), m là số cạnh của G , tức $n = |V|$, $m = |E|$.

6.3 Bài toán tập độc lập trên đồ thị tổng quát

Hiện nay, đa số bài toán liên quan đến tập độc lập trên đồ thị tổng quát không có thuật toán thời gian đa thức, mà chỉ có thể dùng các thuật toán mũ với độ phức tạp tương đối tốt.

Thực ra đã có không ít thuật toán tìm tập độc lập lớn nhất của đồ thị với độ phức tạp lý thuyết rất tốt, có thể nhanh chóng tính tập độc lập lớn nhất của đồ thị vô hướng hàng trăm đỉnh⁹. Nhưng các thuật toán ấy thường quá phức tạp khi cài đặt, khó áp dụng trong thi tin học. Tác giả chọn nghiên cứu một số thuật toán tương đối hiệu quả và dễ cài đặt hơn.

6.3.1 Thuật toán tập độc lập dựa trên tìm kiếm tập độc lập cực đại

Thuật toán tìm kiếm ngây thơ. Thuật toán tìm kiếm ngây thơ nhất rất đơn giản: dùng tìm kiếm theo chiều sâu để liệt kê các tập con $I \subseteq V$, tức theo một thứ tự nào đó liệt kê mỗi đỉnh $v \in V$ có thuộc I hay không; nếu tồn tại $(u, v) \in E$ sao cho $u, v \in I$ thì quay lui. Trong tất cả các tập độc lập I được liệt kê, in ra một tập có $|I|$ lớn nhất. Độ phức tạp của thuật toán là $\mathcal{O}(2^n)$, quá kém hiệu quả.

Riêng với bài toán tập độc lập lớn nhất, có thể thêm vài phép cắt nhánh. Ví dụ:

1. Nếu $\deg(v) = 0$, không có cạnh nào liên thuộc với v , nên luôn có thể cho $v \in I$.
2. Nếu $\deg(v) = 1$, xét đỉnh u duy nhất liên thuộc với v . Nếu $u \notin I$, ta luôn có thể cho $v \in I$; ngược lại, xóa u khỏi I và thêm v vào, kích thước của I không đổi. Vì vậy luôn có thể cho $v \in I$.
3. Khi tìm kiếm, ghi lại giá trị lớn nhất a của kích thước tập độc lập hiện đã tìm được. Ký hiệu P là tập các đỉnh trong $V - I$ không kề với đỉnh nào của I . Khi $|I| + |P| \leq a$, có thể cắt nhánh theo tính tối ưu.

Tuy nhiên sau khi thêm các phép cắt nhánh này, độ phức tạp vẫn còn rất cao và khó chấp nhận.

Tập độc lập cực đại và thuật toán Bron–Kerbosch. Thuật toán tìm kiếm ngây thơ quá kém hiệu quả. Có cách nào tốt để tối ưu không? Ta đưa ra khái niệm tập độc lập cực đại:

⁸Đồ thị con cảm sinh có hai loại: cảm sinh bởi đỉnh và cảm sinh bởi cạnh. Trong bài này đều chỉ loại thứ nhất.

⁹Năm 2001, Robson đề xuất một thuật toán tìm tập độc lập lớn nhất có độ phức tạp xấp xỉ $\mathcal{O}(1.1888^n)$, xem Tài liệu tham khảo [2].

Định nghĩa 3.1. Một **tập độc lập cực đại** (maximal independent set) của đồ thị vô hướng $G = (V, E)$ là một tập độc lập I của G sao cho với mọi đỉnh $v \in V - I$, tập $I + \{v\}$ không phải là tập độc lập.

Thông thường, số tập độc lập cực đại của một đồ thị ít hơn số tập độc lập rất nhiều. Ví dụ, đường đi gồm 50 đỉnh có 32 951 280 099 tập độc lập, nhưng chỉ có 1 221 537 tập độc lập cực đại. Ngoài ra, không ít bài toán tối ưu tổ hợp liên quan đến tập độc lập có thể chỉ xét tập độc lập cực đại; bài toán tập độc lập lớn nhất là một ví dụ như vậy.

Định lý 3.1. Mọi tập độc lập lớn nhất đều là tập độc lập cực đại.

Chứng minh. Giả sử I là một tập độc lập lớn nhất. Với $v \in V - I$ bất kỳ, nếu $I + \{v\}$ là tập độc lập, thì vì $|I + \{v\}| = |I| + 1 > |I|$, $I + \{v\}$ là một tập độc lập lớn hơn I . Nói cách khác, I không phải là tập độc lập lớn nhất, mâu thuẫn với giả thiết. Vì vậy I nhất định là tập độc lập cực đại. ■

Do đó, nếu tìm được mọi tập độc lập cực đại của G , ta sẽ tìm được tập độc lập lớn nhất của G .

Có nhiều thuật toán tìm tập độc lập cực đại; trong đó thuật toán Bron–Kerbosch là một thuật toán cài đặt tương đối gọn. Dưới đây giới thiệu thuật toán Bron–Kerbosch để tìm tập độc lập cực đại.

Thuật toán Bron–Kerbosch có thể tìm mọi tập độc lập cực đại của đồ thị vô hướng bất kỳ G . Tư tưởng cơ bản của nó vẫn là tối ưu tìm kiếm. Mã giả như sau¹⁰:

Thuật toán 1 BronKerbosch(R, P, X)

```

1: if  $P = X = \emptyset$  then
2:   print  $R$ 
3: end if
4: Chọn đỉnh  $u \in P \cup X$  sao cho  $|P \cap (\{u\} \cup N(u))|$  nhỏ nhất
5: for all  $v \in P \cap (\{u\} \cup N(u))$  do
6:   BronKerbosch( $R \cup \{v\}, P - (\{v\} \cup N(v)), X - (\{v\} \cup N(v))$ )
7:    $P \leftarrow P - \{v\}$ 
8:    $X \leftarrow X \cup \{v\}$ 
9: end for
```

Gọi hàm BronKerbosch(R, P, X) sẽ in ra mọi tập độc lập cực đại chứa tất cả các đỉnh trong R , chứa tùy ý nhiều đỉnh trong P , và không chứa đỉnh nào trong X . Gọi BronKerbosch($\emptyset, V, \emptyset$) sẽ thu được mọi tập độc lập cực đại của G .

Khi cài đặt, tập hợp có thể được lưu bằng nén bit: dùng một mảng A gồm $\lceil n/64 \rceil$ số nguyên 64 bit để lưu một tập A' kích thước n , trong đó $x \in A'$ khi và chỉ khi

$$A[\lfloor x/64 \rfloor] \text{ AND } 2^{x \bmod 64} > 0$$

(chỉ số mảng A bắt đầu từ 0). Vì trong bài toán tập độc lập, bậc n của đồ thị sẽ không quá lớn, kích thước mảng nén bit có thể xem xấp xỉ như một hằng số.

Điểm mấu chốt nhất của thuật toán trên là pivoting. Trong quá trình thuật toán có một bước chọn đỉnh $u \in P \cup X$ sao cho $|P \cap (\{u\} \cup N(u))|$ nhỏ nhất; u được gọi là đỉnh pivot. Sau đó liệt kê đỉnh đầu tiên v trong $\{u\} \cup N(u)$ thuộc tập độc lập R . Đó là quá trình pivoting.

Vì sao phải làm như vậy?

Nếu trực tiếp tìm kiếm tập độc lập cực đại, hiệu suất sẽ rất thấp vì thuật toán sẽ duyệt qua rất nhiều tập không cực đại. Chẳng hạn khi G là đồ thị không n đỉnh¹¹, hiển nhiên V là tập độc lập cực đại duy nhất của G . Nhưng khi tìm kiếm ngây thơ liệt kê một đỉnh nào đó không thuộc tập độc lập cực đại, dù

¹⁰Thuật toán Bron–Kerbosch có nhiều cách cài đặt; bài này giới thiệu một trong số đó.

¹¹Đồ thị không được định nghĩa là đồ thị không có cạnh, tức $G = (V, E)$ là đồ thị không khi và chỉ khi $E = \emptyset$.

không thể tìm ra tập độc lập cực đại, thuật toán vẫn sẽ tiếp tục tìm kiếm, lãng phí rất nhiều thời gian. Tính đúng đắn của pivoting dựa trên định lý sau:

Định lý 3.2. Với đồ thị vô hướng $G = (V, E)$ và $v \in V$, mọi tập độc lập cực đại I của G đều thỏa $I \cap (\{v\} \cup N(v)) \neq \emptyset$.

Chứng minh. Giả sử tồn tại tập độc lập cực đại I thỏa $I \cap (\{v\} \cup N(v)) = \emptyset$. Khi đó với mọi $u \in I$, $u \notin \{v\} \cup N(v)$, tức $u \neq v$ và $(u, v) \notin E$.

Vì vậy $I \cup \{v\}$ cũng là một tập độc lập, và $I \subset I \cup \{v\}$. Điều này nói rằng I không cực đại, mâu thuẫn.

■

Như vậy ta đã chứng minh tính đúng đắn của định lý. Dù cách này vẫn sẽ duyệt qua một số tập không cực đại, các tập như thế rõ ràng đã ít hơn rất nhiều.

Số lượng tập độc lập cực đại. Trước đó ta chỉ biết trực giác rằng số tập độc lập cực đại tương đối ít. Ở đây ta đưa ra cận trên cho số lần gọi đệ quy của thuật toán Bron–Kerbosch:

Định lý 3.3. Số lần gọi đệ quy của thuật toán Bron–Kerbosch là $\mathcal{O}(3^{n/3})$.

Chứng minh của định lý này khá phức tạp, bài viết lược bỏ¹².

Từ đó có thể thu được hệ quả:

Định lý 3.4. Số tập độc lập cực đại của đồ thị vô hướng n đỉnh là $\mathcal{O}(3^{n/3})$.

Cận trên này rất dễ đạt tới: chỉ cần dựng $\lfloor n/3 \rfloor$ tam giác độc lập lẫn nhau. Nhưng khi đồ thị được sinh ngẫu nhiên, cận trên này còn rất lỏng. Để minh họa điểm đó, tác giả đã nghiên cứu số tập độc lập cực đại của đồ thị ngẫu nhiên.

Sinh ngẫu nhiên một đồ thị $G = (V, E)$ trong các đồ thị vô hướng đơn n đỉnh có m cạnh. Ký hiệu x là số tập độc lập cực đại của G , tức

$$x = \sum_{S \subseteq V} [S \text{ là tập độc lập cực đại}].$$

Xét cách tính kỳ vọng $\mathbb{E}(x)$. Điều kiện để S là tập độc lập cực đại của G là:

- S là tập độc lập, tức với mọi $u, v \in S$, $(u, v) \notin E$;
- với mọi $v \in V - S$, $S \cup \{v\}$ không phải là tập độc lập, tức mỗi đỉnh trong $V - S$ kề với ít nhất một đỉnh trong S .

Dùng nguyên lý bao hàm–loại trừ, liệt kê k đỉnh trong $V - S$ không kề với đỉnh nào của S . Ký hiệu $i = |S|$. Khi đó $n - i - k$ đỉnh còn lại có thể nối cạnh với các đỉnh trong S , và hai đỉnh bất kỳ trong $V - S$ (tổng cộng $\frac{1}{2}(n - i)(n - i - 1)$ cặp đỉnh) cũng có thể nối cạnh. Vậy số đồ thị G thỏa S là tập độc lập cực đại là

$$\sum_{k=0}^{n-i} (-1)^k \binom{n-i}{k} \binom{(n-i-k)i + \frac{(n-i)(n-i-1)}{2}}{m}.$$

¹²Bạn đọc có hứng thú có thể đọc Tài liệu tham khảo [3].

Do số đồ thị đơn n đỉnh có m cạnh là $\binom{n(n-1)/2}{m}$, ta có

$$\begin{aligned}\mathbb{E}(x) &= \sum_{S \subseteq V} P([S \text{ là tập độc lập cực đại}]) \\ &= \sum_{i=0}^n \sum_{\substack{S \subseteq V \\ |S|=i}} \binom{n(n-1)/2}{m}^{-1} \sum_{k=0}^{n-i} (-1)^k \binom{n-i}{k} \binom{(n-i-k)i + \frac{(n-i)(n-i-1)}{2}}{m} \\ &= \binom{n(n-1)/2}{m}^{-1} \sum_{i=0}^n \binom{n}{i} \sum_{k=0}^{n-i} (-1)^k \binom{n-i}{k} \binom{(n-i-k)i + \frac{(n-i)(n-i-1)}{2}}{m}.\end{aligned}$$

Dưới đây là một số kết quả tính toán, đã làm tròn:

$\mathbb{E}(x)$	$m = n$	$m = \lfloor \sqrt{3}n \rfloor$	$m = 2n$	$m = 3n$	$m = n^2/4$
$n = 20$	84	101	99	81	49
$n = 30$	706	933	909	691	157
$n = 40$	5.95×10^3	8.67×10^3	8.40×10^3	5.88×10^3	403
$n = 50$	5.02×10^4	8.07×10^4	7.76×10^4	5.01×10^4	891
$n = 60$	4.23×10^5	7.51×10^5	7.18×10^5	4.27×10^5	1779
$n = 70$	3.57×10^6	6.99×10^6	6.64×10^6	3.63×10^6	3291
$n = 80$	3.01×10^7	6.51×10^7	6.14×10^7	3.10×10^7	5730
$n = 90$	2.54×10^8	6.06×10^8	5.68×10^8	2.64×10^8	9506
$n = 100$	2.15×10^9	5.64×10^9	5.25×10^9	2.25×10^9	15154

Có thể thấy trong trường hợp ngẫu nhiên, số tập độc lập cực đại nhỏ hơn $3^{n/3}$ rất nhiều. Ngoài ra, khi m gần $\sqrt{3}n$, kỳ vọng $\mathbb{E}(x)$ của số tập độc lập cực đại x là lớn nhất; còn với đồ thị quá dày, số tập độc lập lại khá ít.

Theo kết luận này còn có thể suy ra xác suất $x \geq k \cdot \mathbb{E}(x)$ không vượt quá $1/k$, nên trong đa số trường hợp, số tập độc lập cực đại của đồ thị ngẫu nhiên sẽ không lớn hơn kỳ vọng quá nhiều¹³.

Cần chú ý rằng độ phức tạp của thuật toán Bron–Kerbosch không liên quan trực tiếp đến số tập độc lập cực đại, nên dùng kỳ vọng số tập độc lập cực đại để phân tích thời gian chạy kỳ vọng của thuật toán Bron–Kerbosch là không chính xác. Thực tế có những thuật toán tìm kiếm tập độc lập cực đại có độ phức tạp liên quan trực tiếp đến số tập độc lập cực đại, nhưng chúng vượt ra ngoài phạm vi bài này; bạn đọc có hứng thú có thể tự tìm hiểu.

Ứng dụng. Sau khi giới thiệu tính chất và thuật toán của tập độc lập cực đại, ta hãy xem nó có những ứng dụng nào.

Ví dụ 1 (Bài toán tô màu 3 cho đồ thị). Cho đồ thị vô hướng đơn $G = (V, E)$ bậc n . Dùng ba màu tô các đỉnh trong V sao cho hai đầu mút u, v của mỗi cạnh $(u, v) \in E$ có màu khác nhau. $n \leq 40$.

Bài toán tô màu 3 cho đồ thị cũng là một bài toán NP-hard nổi tiếng. Thuật toán ngây thơ mỗi lần liệt kê màu của một đỉnh kề với đỉnh đã xác định màu, cần liệt kê $\mathcal{O}(2^n)$ trường hợp, không thể qua dữ liệu $n = 40$. Làm thế nào để giải hiệu quả hơn?

Trước hết đưa ra một định lý:

Định lý 3.5. Đồ thị vô hướng $G = (V, E)$ tô được bằng 3 màu khi và chỉ khi G tồn tại một tập độc lập cực đại I sao cho đồ thị $G - I$ là đồ thị hai phía¹⁴.

¹³Vì tác giả chưa nghiên cứu phương sai của x , không thể thu được mô tả chính xác hơn về phân bố của x .

¹⁴Đồ thị vô hướng $G = (V, E)$ là đồ thị hai phía (bipartite graph) nếu có thể chia V thành hai tập S và $V - S$ sao cho hai đầu mút của mỗi cạnh không nằm trong cùng một tập, tức với mọi $(u, v) \in E$, $u \in S, v \in V - S$ hoặc $u \in V - S, v \in S$.

Chứng minh. (Tính đủ) Giả sử I là một tập độc lập cực đại của G và $G - I$ là đồ thị hai phía. Theo tính chất của đồ thị hai phía, tồn tại tập đỉnh $X \subseteq V - I$. Ký hiệu $Y = V - I - X$, sao cho với mọi $u, v \in X$ hoặc $u, v \in Y$, ta có $(u, v) \notin E$.

Vì I là tập độc lập của G , nên với mọi $u, v \in I$, ta có $(u, v) \notin E$.

Do đó tô các đỉnh trong I bằng màu 1, các đỉnh trong X bằng màu 2, các đỉnh trong Y bằng màu 3 là một phương án hợp lệ.

(Tính cần) Giả sử $G = (V, E)$ tô được bằng 3 màu. Ký hiệu X, Y, Z lần lượt là tập các đỉnh được tô màu 1, 2, 3.

Theo định nghĩa, với mọi $(u, v) \in E$, hai đỉnh u, v không thuộc cùng một tập trong ba tập này, nên X, Y, Z đều là tập độc lập.

Nếu X không phải là tập độc lập cực đại, thì tồn tại $v \in V - X$ sao cho $X + \{v\}$ là tập độc lập. Thêm v vào tập X ; đồng thời, nếu $v \in Y$ thì xóa v khỏi Y , ngược lại $v \in Z$ và xóa v khỏi Z . Lặp quá trình này cho đến khi X là tập độc lập cực đại.

Hiển nhiên lúc này Y, Z vẫn là các tập độc lập của G , tức với $u, v \in Y$ hoặc $u, v \in Z$, ta có $(u, v) \notin E$. Vì vậy $G[Y \cup Z]$ là đồ thị hai phía. Bài toán được chứng minh. ■

Kiểm tra một đồ thị có phải đồ thị hai phía hay không và tô màu nó có thể thực hiện trong $\mathcal{O}(m)$ thời gian. Vì vậy chỉ cần liệt kê mọi tập độc lập cực đại I của đồ thị G , rồi kiểm tra đồ thị $G - I$ có phải đồ thị hai phía hay không:

1. Nếu với mọi tập độc lập cực đại I , đồ thị $G - I$ đều không phải đồ thị hai phía, thì đồ thị G không tô được bằng 3 màu.
2. Nếu tồn tại một tập độc lập cực đại I sao cho đồ thị $G - I$ là đồ thị hai phía, thì đồ thị G tô được bằng 3 màu: tô các đỉnh trong I bằng màu 1, còn $G - I$ được tô hai phía bằng màu 2 và 3.

Trong bài này, vì G là đồ thị đơn, $m = \mathcal{O}(n^2)$, nên độ phức tạp thời gian của thuật toán là $\mathcal{O}(3^{n/3}n^2)$.

Ví dụ 2 (Mùa thể thao của Tiểu Q, test 10). Cho một hệ m phương trình đồng dư tuyến tính n ẩn

$$\begin{cases} a_{0,0}x_0 + a_{0,1}x_1 + \cdots + a_{0,n-1}x_{n-1} \equiv c_0 \pmod{b_0}, \\ a_{1,0}x_0 + a_{1,1}x_1 + \cdots + a_{1,n-1}x_{n-1} \equiv c_1 \pmod{b_1}, \\ \dots \\ a_{m-1,0}x_0 + a_{m-1,1}x_1 + \cdots + a_{m-1,n-1}x_{n-1} \equiv c_{m-1} \pmod{b_{m-1}}. \end{cases}$$

Hãy tìm một nghiệm $(x_1, x_2, \dots, x_n)$ thỏa mãn càng nhiều phương trình càng tốt. Đây là bài nộp đáp án.¹⁵

Trong ví dụ này chỉ thảo luận test 10. Ở test này, bằng cách lập mô hình đồ thị, xem mỗi phương trình là một đỉnh và nối cạnh giữa hai phương trình xung đột nhau, ta có thể chuyển thành bài toán tập độc lập lớn nhất trên một đồ thị vô hướng có số đỉnh $n = 90$, số cạnh $m = 223$. Vì quá trình chuyển đổi cụ thể vượt khỏi phạm vi bài viết, nên lược bỏ.

Dùng thuật toán Bron–Kerbosch để tìm mọi tập độc lập cực đại, rồi in ra tập lớn nhất trong số đó. Cách làm này hiệu quả ra sao?

Tác giả so sánh thuật toán tìm kiếm ngây thơ Simple-Search và thuật toán tìm kiếm dựa trên tập độc lập cực đại Maximal-Search. Hai thuật toán chỉ dùng phép cắt nhánh cơ bản nhất: nếu ngay cả khi thêm tất cả các đỉnh còn lại vào I , kích thước tập thu được cũng không vượt quá tập độc lập lớn nhất hiện đã tìm được, thì cắt nhánh. Vì mục đích kiểm thử chỉ là xem Maximal-Search có hiệu suất chạy tốt hơn

¹⁵Nguồn bài: bài 3 của WC2013.

Simple-Search hay không, ở đây không thêm các phép cắt nhánh khác phụ thuộc vào tính chất của bài toán.

Với Simple-Search, chương trình của tác giả chạy vài giờ vẫn chỉ tìm được tập độc lập kích thước 33, và chương trình chưa kết thúc. Còn với Maximal-Search, chương trình của tác giả chỉ mất chưa đến 1 phút để tìm được một tập độc lập kích thước 34, và chỉ mất 3 phút để chứng minh số độc lập của đồ thị thật sự bằng 34. Có thể thấy dùng tập độc lập cực đại để tìm kiếm quả thật có thể nâng cao hiệu suất chạy rất nhiều.

Bằng cách thêm nhiều tối ưu cắt nhánh hơn, có thể rút ngắn thêm thời gian chạy của thuật toán; bạn đọc có hứng thú có thể thử.

6.3.2 Thuật toán tập độc lập dựa trên quy hoạch động

Quy hoạch động là một phương pháp xử lý bài toán hiệu quả và linh hoạt. Trong bài toán tập độc lập, quy hoạch động không chỉ giải được các bài toán tối ưu như tập độc lập lớn nhất, tập độc lập trọng số lớn nhất, mà còn giải được bài toán đếm như đếm tập độc lập. Dưới đây vẫn lấy bài toán tập độc lập lớn nhất làm ví dụ, nhưng để thể hiện tính phổ dụng của quy hoạch động, phần thảo luận tiếp theo sẽ không thêm các phép cắt nhánh chỉ dành cho bài toán tối ưu, như cắt nhánh theo tính tối ưu.

Thuật toán. Nếu muốn dùng quy hoạch động để giải bài toán tập độc lập, ta cần chuyển bài toán thành các bài toán con có quy mô nhỏ hơn. Với tập độc lập, ta có hai định lý sau:

Định lý 3.6. Với đồ thị vô hướng $G = (V, E)$ và $V' \subseteq V$, với mọi $I \subseteq V'$, I là tập độc lập của G khi và chỉ khi I là tập độc lập của $G[V']$.

Chứng minh. (Tính đủ) Khi I là tập độc lập của $G[V']$, với mọi $(u, v) \in E$, nếu $u, v \in V'$ thì hiển nhiên u, v không đồng thời thuộc I ; nếu một trong u, v không thuộc V' , giả sử $u \notin V'$, thì $u \notin I$. Vì vậy I là tập độc lập của G .

(Tính cần) Khi I là tập độc lập của G , vì mỗi cạnh của $G[V']$ đều thuộc G , mỗi cạnh của $G[V']$ có ít nhất một đầu mút không thuộc I . Do đó I là tập độc lập của $G[V']$. ■

Định lý 3.7. Với đồ thị vô hướng $G = (V, E)$ và $v \in V$, nếu $I \subseteq V$ và $v \in I$, thì I là tập độc lập của G khi và chỉ khi $I - \{v\}$ là tập độc lập của $G[V - \{v\} - N(v)]$.

Chứng minh. Ký hiệu $T = V - \{v\} - N(v)$.

(Tính đủ) Khi I là tập độc lập của G , $N(v) \cap I = \emptyset$, nên $I - \{v\} \subseteq T$. Vì mọi cặp đỉnh trong I không kề nhau trong G , $I - \{v\} \subseteq I$, và $G[T] \subseteq G$, nên $I - \{v\}$ là tập độc lập của $G[T]$.

(Tính cần) Khi $I - \{v\}$ là tập độc lập của $G[T]$, vì $I - \{v\} \subseteq V - \{v\} - N(v)$, nên $N(v) \cap I = \emptyset$. Mặt khác, các cạnh mà G có thêm so với $G[T]$ đều kề với v hoặc với các đỉnh trong $N(v)$, nên I là tập độc lập của G . ■

Theo hai định lý trên, ta có thể dùng quy hoạch động nén trạng thái để tìm số độc lập $\alpha(G)$ của đồ thị vô hướng bất kỳ $G = (V, E)$.

Với tập đỉnh $S \subseteq V$, định nghĩa $f(S)$ là số độc lập của đồ thị con cảm sinh bởi S trên G , tức $f(S) = \alpha(G[S])$; hiển nhiên $f(\emptyset) = 0$.

Xét trường hợp $S \neq \emptyset$. Lấy tùy ý $v \in S$, xét một tập $I \subseteq S$. Nếu $v \notin I$, thì I là tập độc lập của $G[S]$ khi và chỉ khi I là tập độc lập của $G[S - \{v\}]$. Nếu $v \in I$, thì I là tập độc lập của $G[S]$ khi và chỉ khi

$I - \{v\}$ là tập độc lập của $G[S - \{v\} - N(v)]$. Từ đó có:

$$f(S) = \begin{cases} 0, & S = \emptyset, \\ \max\{f(S - \{v\}), f(S - \{v\} - N(v)) + 1\}, & \forall v \in S, S \neq \emptyset. \end{cases} \quad (3.1)$$

Khi cài đặt, đánh số các đỉnh trong đồ thị là $0, 1, \dots, n-1$; đỉnh v có thể chọn là đỉnh có số hiệu lớn nhất trong S . Tương tự, có thể dùng kỹ thuật nén bit để lưu tập S . Ngoài ra, có thể dùng tìm kiếm có nhớ để tính f , trạng thái được lưu bằng Hash Table.

Thuật toán này không chỉ tìm được số độc lập, mà còn tìm được một tập độc lập lớn nhất. Xem mã giả sau, trong đó hàm f là hàm tìm kiếm có nhớ được định nghĩa theo công thức (3.1):

Thuật toán 2 Subset-Dynamic-Programming

```

1:   $S = V$ 
2:   $I = \emptyset$ 
3:  while  $S \neq \emptyset$  do
4:    Cho  $v$  là đỉnh có số hiệu lớn nhất trong  $S$ 
5:    if  $f(S - \{v\}) > f(S - \{v\} - N(v)) + 1$  then
6:       $S \leftarrow S - \{v\}$ 
7:    else
8:       $S \leftarrow S - \{v\} - N(v)$ 
9:       $I \leftarrow I + \{v\}$ 
10:   end if
11: end while
12: return  $I = 0$ 
```

Nếu cài đặt trực tiếp, độ phức tạp là $\mathcal{O}(2^{n/2})$, giả sử mỗi thao tác với Hash Table tốn $\mathcal{O}(1)$ thời gian. Lý do là trong $n/2$ tầng đệ quy đầu, mỗi tầng nhiều nhất có 2 nhánh; sau khi đệ quy quá $n/2$ tầng, S chỉ còn chứa các đỉnh trong nửa đầu theo số hiệu, nên tổng độ phức tạp là $\mathcal{O}(2^{n/2})$.

Với n bất kỳ, đều có thể dựng đồ thị G khiến thuật toán này có độ phức tạp $\Theta(2^{n/2})$: nối đỉnh 0 với $\lfloor n/2 \rfloor$, đỉnh 1 với $\lfloor n/2 \rfloor + 1, \dots$, đỉnh $\lfloor n/2 \rfloor - 1$ với $2\lfloor n/2 \rfloor - 1$. Khi đó $\lfloor n/2 \rfloor$ tầng đệ quy đầu đều có 2 nhánh, và mọi nhánh đều khác nhau.

Ví dụ 3 (Đếm clique). Cho đồ thị vô hướng đơn $G = (V, E)$. Hỏi G có bao nhiêu clique. Một clique được định nghĩa là một tập đỉnh $S \subseteq V$ sao cho hai đỉnh bất kỳ trong S đều có cạnh nối. $n \leq 50$.

Ký hiệu $\overline{G}$ là đồ thị bù của G . Không khó thấy S là clique của G khi và chỉ khi S là tập độc lập của $\overline{G}$.

Điều này là vì nếu S là clique của G , các đỉnh trong S đôi một kề nhau trong G , nên đôi một không kề nhau trong $\overline{G}$, tức S là tập độc lập của $\overline{G}$. Ngược lại, nếu S không phải clique của G , tức tồn tại hai đỉnh u, v không kề nhau trong G , thì u, v kề nhau trong $\overline{G}$. Nói cách khác, S không phải tập độc lập của $\overline{G}$.

Vì vậy bài toán chuyển thành bài toán đếm tập độc lập: tìm số tập độc lập của $\overline{G}$. Hiển nhiên dạng bài toán đếm này không thể dùng chiến lược tối ưu tìm kiếm, nhưng dùng thuật toán Subset-Dynamic-Programming nêu trên thì có thể giải trong thời gian $\mathcal{O}(2^{n/2})$.

Tối ưu hiệu suất. Qua kiểm thử thực tế, hiệu suất của thuật toán Subset-Dynamic-Programming không bằng thuật toán tìm kiếm đã tối ưu. Vì sao? Xét bài toán tập độc lập lớn nhất: độ phức tạp tệ nhất của thuật toán này là $\mathcal{O}(2^{n/2})$, nhưng khi cắt nhánh tìm kiếm, đỉnh bậc 1 có thể được xử lý trực tiếp, nên chỉ cần $\mathcal{O}(n)$ thời gian. Có thể tương tự cách tối ưu thuật toán tìm kiếm để dùng một số phép “cắt nhánh” tối ưu DP nói trên không?

Câu trả lời là có. Tuy nhiên tác giả không định lặp lại những tối ưu trước đó: đã dùng thuật toán dựa trên DP thì nên nghiên cứu các tối ưu phổ dụng hơn. Ví dụ, khi tìm tập độc lập trọng số lớn nhất, xử lý

trực tiếp đỉnh bậc 1 là sai. Sau khi nghiên cứu, tác giả thấy các tối ưu sau có thể nâng cao hiệu suất DP rất nhiều:

1. Trong công thức chuyển trạng thái, đỉnh v không lấy đỉnh có số hiệu lớn nhất trong S , mà lấy đỉnh bậc lớn nhất trong $G[S]$, chú ý là đồ thị con cảm sinh chứ không phải đồ thị gốc.
2. Khi đồ thị $G[S]$ không liên thông, ký hiệu tập đỉnh của các thành phần liên thông lần lượt là $S_1, S_2, \dots, S_k$. Vì mỗi thành phần liên thông độc lập với nhau, có thể chuyển thành các bài toán con nhỏ hơn:

$$f(S) = \sum_{i=1}^k f(S_i).$$

3. Khi đồ thị $G[S]$ không chứa chu trình, có thể đổi sang quy hoạch động trên cây để giải.

Thuật toán DP sau tối ưu được ký hiệu là Optimized-Subset-Dynamic-Programming. Tác giả không thể phân tích độ phức tạp của thuật toán này trong trường hợp xấu nhất, nhưng qua kiểm thử trên đồ thị ngẫu nhiên, Optimized-Subset-Dynamic-Programming nhanh hơn Subset-Dynamic-Programming trước đó rất nhiều. Trong đó tối ưu 1 và 2 có hiệu quả rõ rệt, nhất là với đồ thị khá thưa. Lý do là khi liên tục xóa các đỉnh bậc lớn trong đồ thị thưa, đồ thị con cảm sinh $G[S]$ rất dễ trở nên không liên thông.

Ví dụ 4 (Mùa thể thao của Tiểu Q, test 10). Đề bài xem Ví dụ 2.

Phần trên đã nhắc đến thời gian cần thiết để dùng Maximal-Search tìm nghiệm tối ưu của bài toán này. Bây giờ ta thử dùng thuật toán tập độc lập dựa trên quy hoạch động Subset-Dynamic-Programming. Đáng tiếc là kiểm thử cho thấy hiệu suất chạy của thuật toán này không cao, và do số trạng thái quá nhiều, mức tiêu thụ bộ nhớ cũng không thể chấp nhận.

Ta thử tiếp Optimized-Subset-Dynamic-Programming. Điều đáng ngạc nhiên là hiệu suất chạy của thuật toán này rất cao: qua kiểm thử của tác giả, thuật toán chỉ mất 1 giây để thu được nghiệm tối ưu, tức một tập độc lập kích thước 34. Hơn nữa trong quá trình chạy, thuật toán không dùng bất kỳ tối ưu nào phụ thuộc vào tính chất riêng của bài toán, như cắt nhánh theo tính tối ưu. Có thể thấy trên đồ thị thưa, Optimized-Subset-Dynamic-Programming quả thật là một thuật toán xuất sắc.

Liên hệ với thuật toán tìm kiếm. Thực ra, nếu bỏ phần ghi nhớ khỏi thuật toán này, tức không dùng Hash Table để lưu giá trị f , ta thu được một thuật toán tìm kiếm có tối ưu. Thuật toán tìm kiếm này, ký hiệu Optimized-Search, vẫn nhanh hơn thuật toán tìm kiếm ngây thơ Simple-Search, nhưng chậm hơn Optimized-Subset-Dynamic-Programming.

Optimized-Subset-Dynamic-Programming kết hợp các tư tưởng tối ưu của tìm kiếm và quy hoạch động; nó không chỉ có tính phổ dụng khá mạnh mà độ khó cài đặt cũng nhỏ.

Tuy nhiên Optimized-Subset-Dynamic-Programming có một nhược điểm: độ phức tạp không gian khá lớn. Còn thuật toán tìm kiếm Optimized-Search có không gian cấp đa thức và có thể chạy trong thời gian dài hơn. Vì vậy có thể chỉ ghi nhớ giá trị $f(S)$ của các S nhỏ, phần còn lại dùng phương pháp tìm kiếm; như thế có thể giải bài toán với yêu cầu không gian thấp hơn.

Kiểm thử và so sánh. Sau khi nghiên cứu thuật toán quy hoạch động và các tối ưu nói trên, tác giả đã cài đặt thuật toán này (cùng phiên bản đã tối ưu) và các thuật toán dựa trên tìm kiếm trước đó, rồi dùng đồ thị ngẫu nhiên để kiểm thử hiệu suất chạy kỳ vọng của chúng. Trong hàng đầu của bảng sau, n, m biểu thị đồ thị ngẫu nhiên n đỉnh m cạnh; cột cuối 90, 223 biểu thị đồ thị tương ứng với test 10 của bài *Mùa thể thao của Tiểu Q* trong WC2013. Thời gian trong bảng biểu thị ước lượng thời gian chạy kỳ vọng của thuật toán trên đồ thị tương ứng; dấu “-” nghĩa là thời gian chạy quá dài, chưa đo được.

Thuật toán	40, 60	50, 85	60, 120	90, 223
Simple-Search	2s	-	-	-
Maximal-Search	< 0.01s	< 0.1s	1s	-
Subset-Dynamic-Programming	< 0.01s	< 0.1s	1s	-
Optimized-Subset-Dynamic-Programming-1	< 0.01s	< 0.01s	< 0.01s	1s
Optimized-Subset-Dynamic-Programming-2	< 0.01s	< 0.01s	< 0.01s	1s

Maximal-Search và Subset-Dynamic-Programming lần lượt dùng các tối ưu “cắt nhánh tìm kiếm” và “ghi nhớ”, hiệu quả của chúng khá gần nhau. Optimized-Subset-Dynamic-Programming kết hợp ưu điểm của cả hai, nên hiệu suất nghiêm ngặt cao hơn hai thuật toán này. Điều đáng nói là khác biệt giữa Optimized-Subset-Dynamic-Programming-1 và Optimized-Subset-Dynamic-Programming-2 nằm ở việc thuật toán sau thêm tối ưu 3, tức đổi sang DP trên cây. Dù tối ưu 3 trông rất hiệu quả vì biến bài toán cấp mũ thành bài toán thời gian tuyến tính, qua kiểm thử, hiệu suất chạy của hai thuật toán gần như không khác biệt.

6.4 Bài toán tập độc lập trên đồ thị đặc biệt

Phần trên giới thiệu tư tưởng cơ bản để giải tập độc lập trên đồ thị tổng quát: tối ưu tìm kiếm và quy hoạch động. Các phương pháp này đều có độ phức tạp cấp mũ. Trong mục này, ta sẽ bàn tiếp bài toán tập độc lập trên đồ thị đặc biệt: khi bản thân đồ thị có một tính chất đặc biệt nào đó, liệu có thể giải cùng bài toán bằng độ phức tạp đa thức hay không?

6.4.1 Thuật toán tập độc lập lớn nhất dựa trên tư tưởng matching đồ thị

Tập độc lập lớn nhất của đồ thị hai phía. Tập độc lập lớn nhất của đồ thị hai phía là một bài toán kinh điển. Ta có định lý sau:

Định lý 4.1. Với đồ thị hai phía n đỉnh G , $\alpha(G) = n - \nu(G)$, trong đó $\alpha(G)$ và $\nu(G)$ lần lượt là số độc lập và số matching của đồ thị G .

Chứng minh của định lý này có thể tìm thấy trong nhiều tài liệu, nên ở đây lược bỏ.

Dùng thuật toán Hungary hoặc luồng mạng để tìm một số matching $\nu(G)$ của đồ thị hai phía $G = (X, Y, E)$, ta thu được số độc lập $\alpha(G)$ của G . Ngoài ra, sau khi dựng mạng luồng, tìm lát cắt nhỏ nhất có thể thu được một tập độc lập lớn nhất I .

Thuật toán này chỉ giải được bài toán tập độc lập dạng tối ưu, không giải được các bài toán phức tạp hơn như đếm hoặc có thêm ràng buộc khác.

Tập độc lập lớn nhất của đồ thị không móng. Tập độc lập lớn nhất của đồ thị hai phía cho ta một ý tưởng: khi tìm matching lớn nhất của đồ thị, ta có thể liên tục tăng kích thước matching bằng cách tìm đường tăng. Vậy khi tìm tập độc lập lớn nhất của các đồ thị khác, có thể dùng cách tăng tương tự không? Đáng tiếc là trên đồ thị bất kỳ, đồ thị con cảm sinh bởi hiệu đối xứng¹⁶ của hai tập độc lập không nhất thiết là một số đường đi hoặc chu trình, nên không thể dùng phương pháp tìm đường tăng để tìm tập độc lập lớn nhất.

Tuy nhiên, trên một loại đồ thị đặc biệt, thuật toán này là khả thi.

Định nghĩa 4.1. Đồ thị không móng (claw-free graph) được định nghĩa là đồ thị vô hướng mà mọi đồ thị con cảm sinh đều không phải $K_{1,3}$. Trong đó $K_{1,3}$ được gọi là **móng** (claw), tức đồ thị hai phía đầy đủ có một phần chứa 1 đỉnh và phần kia chứa 3 đỉnh.

¹⁶Hiệu đối xứng của hai tập A, B là $A \triangle B = \{x \mid [x \in A] \neq [x \in B]\}$.

Tập độc lập lớn nhất của đồ thị không móng có thể được giải bằng thuật toán tương tự matching trên đồ thị tổng quát. Thuật toán này dựa trên định lý sau:

Định lý 4.2. Giả sử I_1, I_2 là hai tập độc lập của đồ thị không móng G . Khi đó mỗi thành phần liên thông của $G[I_1 \triangle I_2]$ đều là một đường đi đơn hoặc một chu trình đơn.

Chứng minh. Giả sử $v \in I_1$, khi đó $N(v) \cap I_1 = \emptyset$. Trong $G[I_1 \triangle I_2]$, bậc của v là $|N(v) \cap I_2|$.

Giả sử tồn tại ba đỉnh khác nhau $v_1, v_2, v_3 \in N(v) \cap I_2$. Vì I_2 là tập độc lập, nên v_1, v_2, v_3 đôi một không kề nhau. Do đó $G[\{v, v_1, v_2, v_3\}]$ là một móng, mâu thuẫn.

Vì vậy $|N(v) \cap I_2| \leq 2$, tức mọi đỉnh của $G[I_1 \triangle I_2]$ đều có bậc không vượt quá 2. Định lý được chứng minh. ■

Chú ý rằng nếu $|I_1| < |I_2|$, thì $G[I_1 \triangle I_2]$ nhất định tồn tại một thành phần liên thông C sao cho số đỉnh thuộc I_2 trong thành phần này nhiều hơn số đỉnh thuộc I_1 . Vì các đỉnh thuộc I_1 và I_2 trong C xuất hiện xen kẽ nhau, nếu C là chu trình đơn thì số đỉnh thuộc I_1 và I_2 trong C bằng nhau; còn nếu C là đường đi đơn thì số đỉnh thuộc I_1 và I_2 trong C chênh nhau 1. Vì vậy nhất định tồn tại một đường đi có số đỉnh thuộc I_2 nhiều hơn số đỉnh thuộc I_1 đúng 1. Ta gọi C là đường tăng (augment path) của I_1 .

Như vậy, có thể tương tự thuật toán matching lớn nhất trên đồ thị tổng quát để dùng thuật toán đường tăng tìm tập độc lập lớn nhất của đồ thị không móng $G = (V, E)$. Ban đầu đặt $I = \emptyset$. Mỗi lần xuất phát từ một đỉnh để tìm một đường tăng, rồi đảo trạng thái các đỉnh trên đường tăng: đỉnh trước đó không thuộc tập độc lập thì thêm vào tập độc lập, đỉnh trước đó thuộc tập độc lập thì xóa khỏi tập độc lập. Cài đặt thuật toán này tương tự thuật toán hoa của Edmonds; chi tiết cài đặt và chứng minh tính đúng đắn xem Tài liệu tham khảo [4], ở đây không trình bày thêm.

6.4.2 Thuật toán tập độc lập lớn nhất dựa trên tư tưởng chia giai đoạn trên đồ thị

6.4.3 Quy hoạch động trên đồ thị phân tầng

Với đồ thị $G = (V, E)$, chia tập đỉnh V thành k tập rời nhau $V_1, V_2, \dots, V_k$ sao cho với mọi $u \in V_i, v \in V_j$, nếu $|i - j| > 1$ thì $(u, v) \notin E$. Khi đó gọi dãy tập $\langle V_1, V_2, \dots, V_k \rangle$ là một phân tầng của G . Nếu mỗi V_i có không nhiều đỉnh, ta có thể quyết định theo thứ tự $V_1, V_2, \dots, V_k$, và ở mỗi giai đoạn chỉ cần nén trạng thái cách chọn của một tầng, hiệu quả cao hơn nhiều so với DP nén trạng thái trên toàn bộ đồ thị tổng quát.

Ký hiệu $f(i, S)$ là tập độc lập lớn nhất trong $G[V_1 \cup V_2 \cup \dots \cup V_i]$ có chứa S làm tập con. Công thức chuyển trạng thái như sau:

$$f(i, S) = \begin{cases} -\infty, & S \text{ không độc lập,} \\ 0, & i = 0, \\ \max\{f(i-1, S') \mid S' \subseteq V_{i-1}, S \cup S' \text{ độc lập}\} + |S'|, & \text{trường hợp còn lại.} \end{cases}$$

Số độc lập của G là giá trị lớn nhất của $f(k, I)$ trên mọi tập độc lập I của $G[V_k]$. Quá trình này cũng tìm được một tập độc lập lớn nhất của G , phương pháp tương tự như trước, nên ở đây không trình bày thêm.

Độ phức tạp thời gian của thuật toán này là $\mathcal{O}\left(\sum_{i=1}^{k-1} 2^{|V_i|+|V_{i+1}|}\right)$. Tuy nhiên trong đa số trường hợp, số tập độc lập trong mỗi V_i không nhiều, nên hiệu suất thời gian thực tế cao hơn cận trên lý thuyết rất nhiều.

6.4.4 Quy hoạch động trên “đồ thị k -xương rồng”

Ví dụ 5. Cho đồ thị vô hướng đơn $G = (V, E)$, bảo đảm mỗi cạnh thuộc đúng một chu trình đơn. Hãy tìm số độc lập của G . $|V| \leq 50\,000$, $|E| \leq 60\,000$.¹⁷

Nếu G không liên thông, chỉ cần tìm số độc lập của từng thành phần liên thông của G rồi cộng lại. Dưới đây giả sử G là đồ thị liên thông. Đồ thị đơn liên thông mà mỗi cạnh thuộc nhiều nhất một chu trình đơn được gọi là “xương rồng”. Nó có tính chất đặc biệt gì?

Ta có thể thử mạnh dạn: chọn tùy ý $r \in V$, lấy r làm gốc và tìm kiếm theo chiều sâu (depth-first search, DFS) trên đồ thị G , thu được một cây tìm kiếm theo chiều sâu (cây DFS) $T = (V, E_T)$. Hiển nhiên T là một cây khung của G . Định nghĩa cạnh cây là cạnh thuộc E_T , cạnh không cây là cạnh không thuộc E_T , tức thuộc $E - E_T$.

Cây DFS có một tính chất rất quan trọng:

Định lý 4.3. Với mọi cạnh không cây $e = (u, v)$, trong T , hoặc u là tổ tiên của v , hoặc v là tổ tiên của u .

Chứng minh. Giả sử $e = (u, v)$ là cạnh không cây và u được thăm trước v . Khi thăm u , nếu v không nằm trong cây con gốc u , thì khi liệt kê cạnh ra e của u , v chưa được thăm, nên bước tiếp theo sẽ đi theo cạnh e để thăm v . Từ đó $e \in E_T$, mâu thuẫn với giả thiết. Vậy u là tổ tiên của v . Tương tự, nếu v được thăm trước u , thì v là tổ tiên của u . ■

Với cạnh không cây $e = (u, v)$, nếu cạnh cây e' nằm trên đường đi đơn từ u đến v trong cây T , thì gọi cạnh cây e' bị cạnh không cây e **phủ** (cover). Với xương rồng, ta có:

Định lý 4.4. Mỗi cạnh cây bị nhiều nhất một cạnh không cây phủ.

Chứng minh. Giả sử một cạnh cây e bị hai cạnh không cây (u_1, v_1) , (u_2, v_2) phủ. Khi đó (u_1, v_1) cùng đường đi đơn từ v_1 đến u_1 trong T tạo thành một chu trình đơn C_1 ; (u_2, v_2) cùng đường đi đơn từ v_2 đến u_2 trong T tạo thành một chu trình đơn C_2 . Cạnh e đồng thời thuộc C_1 và C_2 , mâu thuẫn với định nghĩa xương rồng. Vì vậy e bị nhiều nhất một cạnh không cây phủ. ■

Tính chất này cho phép ta làm quy hoạch động trên cây T . Ý tưởng ban đầu là: ký hiệu $f(i, 0)$ là kích thước tập độc lập lớn nhất trong cây con gốc đỉnh $i \in V$ của T , $f(i, 1)$ là kích thước tập độc lập lớn nhất trong cây con i khi đỉnh cha của i thuộc tập độc lập. Tuy nhiên cách này không bảo đảm hai đầu mút của cạnh không cây không đồng thời thuộc tập độc lập.

Xét thêm một chiều trạng thái. Ký hiệu $f(i, j, k)$ là kích thước tập độc lập lớn nhất trong cây con gốc i , trong trường hợp đỉnh cha của i thuộc ($j = 1$) hoặc không thuộc ($j = 0$) tập độc lập, và đỉnh đầu trên u_i của cạnh không cây $e'_i = (u_i, v_i)$ phủ cạnh e_i nối i với cha của nó thuộc ($k = 1$) hoặc không thuộc ($k = 0$) tập độc lập. Khi e_i không thuộc chu trình, $k = 0$. Khi chuyển, liệt kê i có thuộc tập độc lập hay không; chú ý khi i là đỉnh đầu dưới v_i của e'_i và $k = 1$, không thể chọn i . Công thức chuyển trạng thái như sau, trong đó Chi_i biểu thị tập con của đỉnh i :

- Khi $j = 1$ hoặc $k = 1 \wedge i = v_i$:

$$f(i, j, k) = \sum_{c \in \text{Chi}_i} f(c, 0, [e'_c = e'_i]k).$$

- Ngược lại:

$$f(i, j, k) = \max \left\{ \sum_{c \in \text{Chi}_i} f(c, 0, [e'_c = e'_i]k), 1 + \sum_{c \in \text{Chi}_i} f(c, 1, [e'_c = e'_i]k + [u_c = i]) \right\}.$$

¹⁷Nguồn bài: LYDSY 4316.

Sau khi tiền xử lý mọi u_i, v_i trong $\mathcal{O}(m)$ thời gian, ta có thể dùng quy hoạch động $\mathcal{O}(n)$ nói trên để tìm tập độc lập lớn nhất. Độ phức tạp thời gian là $\mathcal{O}(n + m)$.

Sau khi nghiên cứu thuật toán trên, tác giả suy nghĩ liệu thuật toán này có thể mở rộng hay không. Qua phân tích, tác giả nhận thấy sau khi sửa thuật toán một cách đơn giản, nó không chỉ xử lý được xương rồng mà còn xử lý được đồ thị trong đó mỗi cạnh thuộc không nhiều chu trình đơn.

Ta gọi các đồ thị như vậy là “đồ thị k -xương rồng”¹⁸, tức đồ thị mà mỗi cạnh thuộc không nhiều nhất k chu trình đơn, trong đó k tương đối nhỏ.

Khi giải bài toán tập độc lập trên đồ thị k -xương rồng $G = (V, E)$, ta cũng chọn một đỉnh làm gốc và DFS trên đồ thị để thu được cây DFS T . Tiếp theo với mỗi đỉnh i , ký hiệu C_i là tập các cạnh không cây phủ cạnh e_i nối i với cha của nó. Khi đó có một tính chất:

Định lý 4.5. Trong đồ thị k -xương rồng, với mọi $i \in V$, $|C_i| \leq k$.

Chứng minh. Với mỗi $(u, v) \in C_i$, cạnh (u, v) và đường đi từ u đến v trong T đều tương ứng với một chu trình đơn chứa e_i . Vì vậy nhất định tồn tại $|C_i|$ chu trình đơn chứa e_i .

Do số chu trình đơn chứa e_i không vượt quá k , nên $|C_i| \leq k$. ■

Tiếp theo, ta lợi dụng tính chất này để thiết kế quy hoạch động. Trong trạng thái, cần ghi lại đỉnh đầu trên của mọi cạnh trong C_i có thuộc tập độc lập hay không; cách chuyển tương tự. Cụ thể, ký hiệu $f(i, j, S)$ là kích thước tập độc lập lớn nhất trong cây con gốc i của T , trong trường hợp đỉnh cha của i thuộc ($j = 1$) hoặc không thuộc ($j = 0$) tập độc lập, và trong tập U_i gồm các đỉnh đầu trên của các cạnh trong C_i , những đỉnh thuộc tập độc lập tạo thành tập S . Công thức chuyển trạng thái như sau:

- Khi $j = 1$ hoặc $S \cup \{i\}$ không phải là tập độc lập:

$$f(i, j, S) = \sum_{c \in \text{Chi}_i} f(c, 0, S \cap U_i).$$

- Ngược lại:

$$f(i, j, S) = \max \left\{ \sum_{c \in \text{Chi}_i} f(c, 0, S \cap U_i), 1 + \sum_{c \in \text{Chi}_i} f(c, 1, (S \cup \{i\}) \cap U_i) \right\}.$$

Tương tự thuật toán trước, khi xem k là hằng số, độ phức tạp của thuật toán này là $\mathcal{O}(n)$.

Thực ra điều kiện “đồ thị k -xương rồng” là quá rộng. Chỉ cần số cạnh không cây phủ mỗi cạnh cây đều không vượt quá k , thậm chí chỉ cần $\sum_{v \in V} 2^{|C_v|}$ không lớn, thuật toán này đều có hiệu suất rất cao.

Thuật toán này có thể mở rộng sang các bài toán phức tạp hơn, như đếm tập độc lập.

Ví dụ 6 (Bài toán đếm tập con, test 7 và 8). Cho đồ thị vô hướng $G = (V, E)$, $|V| = n$, $|E| = m$. Hỏi có bao nhiêu tập con $V' \subseteq V$ thỏa $|V'| = k$ và với mọi $(u, v) \in E$, $u \notin V' \vee v \notin V'$. Vì đáp án có thể rất lớn, chỉ cần in đáp án lấy modulo p . Đây là bài nộp đáp án.¹⁹

Trong ví dụ này chỉ thảo luận test 7 và 8. Hai test này thỏa G là đồ thị liên thông, quy mô dữ liệu như bảng sau:

Mã test	$n =$	$m =$	$k =$	$p =$
7	4998	5002	666	1000000009
8	11986	12011	1098	1000000007

¹⁸Tên gọi bắt nguồn từ LYDSY 4863.

¹⁹Nguồn bài: tự sáng tác, UOJ #259.

Bài toán yêu cầu số tập độc lập kích thước k trong G . Vì G là đồ thị liên thông và $m - n$ rất nhỏ, có thể thấy G nhận được bằng cách thêm $m - n + 1$ cạnh vào một cây.

Nếu sau khi dựng cây DFS của G , ta vét cạn một đầu mút của mỗi cạnh không cây có được chọn hay không, rồi dùng quy hoạch động trên cây để thống kê số tập độc lập, thì có thể qua test 7 khá nhanh. Nhưng test 8 cần chạy quá lâu và cần tối ưu. Theo tư tưởng đã nêu trong mục này, số chu trình đơn mà mỗi cạnh thuộc về không nhiều, nên có thể dùng thuật toán trên để giải.

Ký hiệu:

- C_i là tập các u_e thỏa $e \in E'$, u_e là tổ tiên của i , không tính i , và v_e nằm trong cây con gốc i .
- $f(i, j, S, s)$ là số tập độc lập kích thước s trong j cây con đầu tiên của i , với điều kiện tập các đỉnh trong C_i thuộc tập độc lập là S .
- $g(i, j, S, s)$ là số tập độc lập kích thước s trong i và j cây con đầu tiên của i , với điều kiện tập các đỉnh trong C_i thuộc tập độc lập là S .
- $F(i, S, s) = f(i, t_i, S, s)$, $G(i, S, s) = g(i, t_i, S, s)$, trong đó t_i là số con của i .

Với $f(i, j, S, s)$, giả sử trong i và $j - 1$ cây con đầu tiên của i có tổng cộng s_1 đỉnh, trong cây con thứ j có s_2 đỉnh, và con thứ j là c_j . Khi đó

$$f(i, j, S, s) = \sum_{x=\max\{s-s_1, 0\}}^{\min\{s, s_2\}} f(i, j-1, S, s-x) G(c_j, S \cap C_{c_j}, x), \quad 0 \leq s \leq s_1 + s_2.$$

Với $g(i, j, S, s)$, nếu tồn tại cạnh $e \in E'$ sao cho $u_e \in S$ và $v_e = i$, thì i không thể thuộc tập độc lập, nên

$$g(i, j, S, s) = f(i, j, S, s).$$

Ngược lại, tồn tại tập độc lập chứa i ; số tập độc lập như vậy được ký hiệu $g'(i, j, S, s)$. Khi đó

$$g'(i, j, S, s) = \sum_{x=\max\{s-1, 0\}}^{\min\{s-1, s_2\}} g'(i, j-1, S, s-x) F(c_j, (S \cup \{i\}) \cap C_{c_j}, x), \quad 0 \leq s \leq s_1 + s_2.$$

Do đó

$$g(i, j, S, s) = g'(i, j, S, s) + f(i, j, S, s), \quad 0 \leq s \leq s_1 + s_2.$$

Biên là $f(i, 0, S, 0) = g'(i, 0, S, 1) = 1$; mọi trạng thái chưa định nghĩa có giá trị 0. Đáp án chính là $G(r, \emptyset, k)$.

Chỉ cần tính các trạng thái thỏa $s \leq k$. Độ phức tạp là $\mathcal{O}(k \sum_{v \in V} 2^{|C_v|})$, trong đó $|C_v|$ thường không quá 12. Toàn bộ tính toán được thực hiện theo modulo p ; bộ nhớ có thể cấp phát động.

Cần chú ý rằng chọn các đỉnh $r \in V$ khác nhau làm gốc, cũng như dùng các thứ tự khác nhau để DFS, sẽ cho hiệu suất chạy khác nhau. Có thể chọn một gốc r để DFS sao cho

$$\sum_{v \in V} 2^{|C_v|}$$

nhỏ nhất, rồi mới thực hiện thuật toán trên để giảm độ phức tạp thời gian. Qua thực nghiệm, số trạng thái của test 7 xấp xỉ 5×10^5 , có thể qua test này trong 1 giây; số trạng thái của test 8 xấp xỉ 10^8 , có thể qua test này trong 2 phút.

6.5 Tổng kết

Các phương pháp tối ưu thuật toán cho bài toán NP-hard nhiều không kể xiết. Bài viết này chỉ nhắc đến vài thuật toán tối ưu cho bài toán tập độc lập. Các phương pháp này giải những bài toán tương tự nhau, nhưng tư tưởng lại khác nhau: với bài toán tối ưu thông thường, như tập độc lập lớn nhất, có thể dùng thuật toán tìm kiếm có cắt nhánh theo tính tối ưu để giảm lượng liệt kê; với bài toán đếm hoặc có ràng buộc bổ sung, như đếm tập độc lập, có thể dùng quy hoạch động nén trạng thái và nâng cao hiệu suất chạy bằng cách tối ưu số trạng thái; với đồ thị có thể “tăng” và đồ thị có tính giai đoạn rõ rệt, lại có thể dùng thuật toán độ phức tạp đa thức để hoàn thành hiệu quả.

Đồng thời, bài viết phân tích và so sánh thời gian chạy của vài thuật toán trong trường hợp ngẫu nhiên, giúp mọi người có nhận thức sâu hơn về hiệu suất giải bài toán tập độc lập.

Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi.

Cảm ơn cô Chen Ying của Trường Trung học số 1 Phúc Châu đã quan tâm và chỉ dẫn.

Cảm ơn các anh chị khóa trên của Trường Trung học số 1 Phúc Châu đã đem lại nhiều gợi mở và chỉ dẫn.

Cảm ơn tổ huấn luyện OI đã giúp đỡ tôi.

Cảm ơn bạn Chen Junkun đã cung cấp cho tôi không ít kỹ thuật tối ưu thuật toán tập độc lập, giúp tôi rất nhiều trong nghiên cứu DP trên cây DFS.

Cảm ơn mọi người đã cung cấp các tài liệu mà bài viết này từng tham khảo.

Cảm ơn quý vị đã dành thời gian quý báu trong lúc bận rộn để đọc bài.

Tài liệu tham khảo

- [1] Wikipedia, Bron–Kerbosch algorithm.
- [2] Robson, J. M. (2001), Finding a maximum independent set in time $\mathcal{O}(2^{n/4})$.
- [3] Tomita, Etsuji; Tanaka, Akira; Takahashi, Haruhisa (2006), The worst-case time complexity for generating all maximal cliques and computational experiments.
- [4] Minty, George J. (1980), “On maximal independent sets of vertices in claw-free graphs”, *Journal of Combinatorial Theory. Series B*, 28 (3): 284–304, doi:10.1016/0095-8956(80)90074-X, MR 579076.

7 Báo cáo ra đề và mở rộng của *Đồ thị con kỳ diệu*

Chen Junkun, Trường Trung học số 1 Phúc Châu, tỉnh Phúc Kiến

Tóm tắt

Đồ thị con kỳ diệu là một bài truyền thống do tôi ra trong đợt kiểm tra chéo đội tuyển tập huấn năm 2017. Bằng cách làm yếu các điều kiện khác nhau của bài toán, ta có thể chuyển nó thành các bài toán đồ thị thỏa những điều kiện đặc thù khác nhau, rồi giải bằng nhiều thuật toán. Bài viết này tập trung nghiên cứu một lớp bài toán DP có cập nhật trên đồ thị dưới một điều kiện đặc biệt, đề xuất “cây tròn–vuông” và thuật toán dựa trên tư tưởng phân trị theo chuỗi, đồng thời mở rộng ứng dụng của hai thuật toán đó. Tôi hy vọng thông qua bài *Đồ thị con kỳ diệu*, có thể chia sẻ với mọi người một phương pháp xử lý chung cho một lớp thành phần song liên thông đỉnh và phương pháp giải các bài toán DP trên cây có cập nhật.

7.1 Đề bài

7.1.1 Mô tả tóm tắt

Cho một đồ thị vô hướng đơn $G = (V, E)$. Đặt $n = |V|$, $m = |E|$; mỗi đỉnh có một trọng số, lần lượt là $v_1, v_2, \dots, v_n$. Đồ thị G thỏa một điều kiện đặc biệt: với mọi tập V' gồm 7 đỉnh bất kỳ trong V , luôn tồn tại ba đỉnh đôi một khác nhau p, q, x sao cho $p, q \in V'$, $x \in V - \{p, q\}$, và sau khi xóa đỉnh x khỏi đồ thị thì p và q không liên thông.

Bây giờ, cho hằng số k . Cần tìm một đồ thị con liên thông $G' = (V', E')$ của G sao cho với mọi $p \in V'$, bậc của p trong G' không vượt quá k . Hãy cực đại hóa tổng trọng số các đỉnh trong đồ thị con, tức $\sum_{p \in V'} v_p$ (chỉ cần in giá trị lớn nhất), đồng thời tính số đồ thị con khác nhau đạt được giá trị lớn nhất đó, lấy modulo 64123. Hai đồ thị con $G_1 = (V_1, E_1)$ và $G_2 = (V_2, E_2)$ được coi là khác nhau khi và chỉ khi $V_1 \neq V_2$ hoặc $E_1 \neq E_2$.

Bài còn cần hỗ trợ q lần cập nhật; mỗi lần sửa trọng số v_p của một đỉnh p . Hãy trả lời hai câu hỏi trên trước mọi cập nhật và sau mỗi lần cập nhật.

7.1.2 Dữ liệu vào

Dòng đầu chứa ba số nguyên n, m, k , lần lượt là số đỉnh, số cạnh của G và hằng số giới hạn bậc k .

Dòng thứ hai chứa n số nguyên $v_1, v_2, \dots, v_n$, là trọng số ban đầu của các đỉnh.

Trong m dòng tiếp theo, dòng thứ i chứa hai số nguyên $x_i, y_i \in [1, n]$, biểu thị cạnh thứ i nối hai đỉnh x_i và y_i .

Dòng tiếp theo chứa một số nguyên q , là số lần sửa trọng số đỉnh.

Trong q dòng tiếp theo, dòng thứ i chứa hai số nguyên p_i, w_i , biểu thị trong lần sửa thứ i , đặt $v_{p_i} \leftarrow w_i$.

7.1.3 Dữ liệu ra

Ban đầu và sau mỗi lần sửa, in một dòng đáp án, tổng cộng $q + 1$ dòng. Mỗi dòng chứa hai số, lần lượt là “tổng trọng số của đồ thị con có tổng trọng số lớn nhất” và “số đồ thị con hợp lệ khác nhau có tổng trọng số đạt giá trị lớn nhất, lấy phần dư khi chia cho 64123”.

7.1.4 Ví dụ

Dữ liệu vào

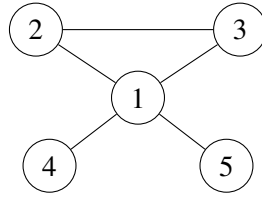
```
5 5 2
1 2 1 2 2
2 3
2 1
1 4
1 3
1 5
1
2 1
```

Dữ liệu ra

```
6 4
5 5
```

7.1.5 Giải thích ví dụ

Trong ví dụ, đồ thị G như sau:



Ban đầu có 4 phương án có tổng trọng số đỉnh đạt giá trị lớn nhất 6:

Số	V'	E'
1	$\{1, 2, 3, 4\}$	$\{(2, 3), (1, 2), (1, 4)\}$
2	$\{1, 2, 3, 4\}$	$\{(2, 3), (1, 3), (1, 4)\}$
3	$\{1, 2, 3, 5\}$	$\{(2, 3), (1, 2), (1, 5)\}$
4	$\{1, 2, 3, 5\}$	$\{(2, 3), (1, 3), (1, 5)\}$

Sau khi sửa trọng số đỉnh 2 thành 1, có 5 phương án có tổng trọng số đỉnh đạt giá trị lớn nhất 5:

Số	V'	E'
1	$\{1, 2, 3, 4\}$	$\{(2, 3), (1, 2), (1, 4)\}$
2	$\{1, 2, 3, 4\}$	$\{(2, 3), (1, 3), (1, 4)\}$
3	$\{1, 2, 3, 5\}$	$\{(2, 3), (1, 2), (1, 5)\}$
4	$\{1, 2, 3, 5\}$	$\{(2, 3), (1, 3), (1, 5)\}$
5	$\{1, 4, 5\}$	$\{(1, 4), (1, 5)\}$

7.1.6 Giới hạn và quy ước

Với mọi dữ liệu, bảo đảm $n \geq 2$, $m, q \geq 0$, $2 \leq k \leq 3$, $1 \leq v_i, w_i \leq 5000$. Thông tin từng subtasks như sau (chấm gộp):

Subtask	Điểm	Test	n	m	k	q	Tính chất
1	7	1	≤ 10	≤ 20	$= 3$	≤ 100	Không
2	18	2,3	≤ 10000	$= n - 1$	$= 2$	≤ 20000	1
3	7	4,5	≤ 50000	$= n - 1$	$= 2$	≤ 50000	1
4	15	6,7,8	≤ 100000	$= n - 1$	$= 2$	≤ 200000	1
5	12	9,10	≤ 100	≤ 300	$= 2$	$= 0$	2,3
6	9	11,12	≤ 1000	≤ 3000	$= 3$	$= 0$	3
7	5	13	≤ 30000	≤ 100000	$= 3$	$= 0$	Không
8	14	14,15	≤ 100000	≤ 300000	$= 3$	$= 0$	Không
9	3	16	≤ 30000	≤ 55000	$= 3$	≤ 10000	Không
10	10	17–20	≤ 30000	≤ 100000	$= 3$	≤ 10000	Không

Tính chất 1: G là một cây vô căn có n đỉnh.

Tính chất 2: mọi v_i, w_i đều bằng 1.

Tính chất 3: với mọi tập V' gồm 5 đỉnh bất kỳ trong V , luôn tồn tại ba đỉnh đôi một khác nhau p, q, x sao cho $p, q \in V'$, $x \in V - \{p, q\}$, và sau khi xóa x khỏi đồ thị thì p và q không liên thông.

Với mỗi test, nếu trong đáp án của thí sinh, số thứ nhất của mọi dòng đều đúng nhưng một số dòng có số thứ hai sai, vẫn được 60% số điểm của test đó (làm tròn đến số nguyên).

7.1.7 Hạn chế

Giới hạn thời gian: 4 giây trên TUOJ.

Giới hạn bộ nhớ: 1 GB.

7.2 Phân tích sơ bộ

7.2.1 Ký hiệu và quy ước

Dùng ký hiệu $[x]$: nếu mệnh đề x đúng thì $[x] = 1$, ngược lại $[x] = 0$.

Dùng ký hiệu $\mathcal{O}(A) + \mathcal{O}(B) + \mathcal{O}(C)$ để mô tả thuật toán có độ phức tạp tiền xử lý $\mathcal{O}(A)$, độ phức tạp mỗi lần sửa $\mathcal{O}(B)$, và độ phức tạp bộ nhớ $\mathcal{O}(C)$. Khi đó tổng thời gian là $\mathcal{O}(A + qB)$.

Dùng ký hiệu đại diện $*$. Nếu một biến xuất hiện có chứa $*$, nó biểu thị tập mọi biến thỏa điều kiện. Ví dụ, nếu với mỗi $p \in V$ định nghĩa $f(p)$, thì $f(*)$ là tập $\{f(p) \mid p \in V\}$; nếu với mỗi $p \in V, 1 \leq d \leq k$ định nghĩa $f(p, d)$, thì $f(a, *)$ là tập $\{f(a, b) \mid 1 \leq b \leq k\}$.

Đồ thị gốc có thể không liên thông. Nếu vậy, ta có thể tính nghiệm tối ưu cho từng thành phần liên thông. Vì thế các phân tích sau mặc định đang xét một đồ thị liên thông.

Bài này cần tính số phương án tối ưu. Có thể dùng thuật toán chung sau để đếm: định nghĩa thông tin (v, c) , trong đó v là giá trị tối ưu, c là số phương án sau khi lấy modulo; định nghĩa phép cộng

$$(v_1, c_1) + (v_2, c_2) = (\max(v_1, v_2), c_1[v_1 \geq v_2] + c_2[v_2 \geq v_1] \bmod 64123),$$

và phép nhân

$$(v_1, c_1) \times (v_2, c_2) = (v_1 + v_2, c_1 c_2 \bmod 64123).$$

Dùng thông tin này là có thể đồng thời duy trì nghiệm tối ưu và số phương án; phần sau sẽ không nhắc lại.

7.2.2 Thuật toán một: vét cạn

Trong subtask 1, dữ liệu rất nhỏ, $m \leq 20$, nên có thể xét mọi đồ thị con.

Chú ý mục tiêu tối ưu và các lần sửa chỉ liên quan đến V' , còn E' chỉ ảnh hưởng đến số phương án. Vì vậy có thể tiền xử lý, với mỗi V' , số cách chọn E' .

Khi tiền xử lý, liệt kê tập cạnh E' , kiểm tra đồ thị con sinh ra có thỏa giới hạn bậc và liên thông hay không; nếu có thì cập nhật số phương án cho tập đỉnh của đồ thị con đó. Cần xử lý riêng trường hợp một đỉnh.

Sau mỗi lần sửa, chỉ cần tính lại tổng trọng số của mỗi V' , rồi cộng số phương án của các V' đạt tối ưu.

Độ phức tạp là $\mathcal{O}(m2^m) + \mathcal{O}(2^n) + \mathcal{O}(2^n + m)$. Thuật toán qua được subtask 1, kỳ vọng 7 điểm. Thuật toán này chưa dùng bất kỳ tính chất nào của đề, nên độ phức tạp rất tệ. Trước hết, ta xét một lớp dữ liệu đặc biệt: dữ liệu là cây.

7.3 Dữ liệu trên cây với $k = 2$

Subtask 2 đến 4, tổng 40 điểm, thỏa G là một cây và $k = 2$. Không khó thấy các test này yêu cầu tìm đường đi có tổng trọng số lớn nhất trên cây, có hỗ trợ sửa trọng số một đỉnh. Sau đây gọi “chuỗi dài nhất” là chuỗi có tổng trọng số lớn nhất.

7.3.1 Thuật toán hai: vét cạn trên cây

Trong subtask 2, dữ liệu cho phép thuật toán $\mathcal{O}(n)$ cho mỗi lần sửa.

Bài toán này có cấu trúc con tối ưu, nên có thể dùng quy hoạch động. Chọn tùy ý một đỉnh gốc r để biến cây thành cây có gốc; ký hiệu $\text{Ch}(i)$ là tập con của đỉnh i . Đặt $f(i)$ là khoảng cách từ i đến đỉnh xa

nhất trong cây con của i ²⁰, và $g(i)$ là tổng trọng số của chuỗi dài nhất nằm hoàn toàn trong cây con của i . Khi đó

$$f(i) = v_i + \max_{p \in \text{Ch}(i)} f(p),$$

$$g(i) = \max \left(f(i), \max_{p \in \text{Ch}(i)} g(p), v_i + \max_{\substack{p, q \in \text{Ch}(i) \\ p \neq q}} (f(p) + f(q)) \right).$$

Chỉ cần ghi thêm số phương án trong $f(i)$ và $g(i)$ là tính được số phương án tối ưu. Đáp án là $f(r)$ và số phương án tương ứng.

Mỗi lần sửa, chỉ cần cập nhật các giá trị DP trên đường từ p đến gốc r , nên thời gian mỗi lần sửa là $\mathcal{O}(n)$.

Độ phức tạp là $\mathcal{O}(n) + \mathcal{O}(n) + \mathcal{O}(n)$. Thuật toán qua được subtask 2, có thể qua subtask 3, kỳ vọng 18 đến 25 điểm.

7.3.2 Thuật toán ba: phân trị theo đỉnh

Subtask 3 và 4 có số lần sửa và kích thước cây lớn hơn, nên cần tối ưu độ phức tạp mỗi lần sửa.

Đây là một bài toán thống kê đường đi, do đó có thể dùng phân trị theo đỉnh; nội dung và cách cài đặt cụ thể có thể xem [1]. Trong quá trình phân trị, chỉ cần xét các đường đi qua trọng tâm hiện tại g và số phương án của chúng. Gọi tập nhánh của trọng tâm g là Br (chính là tập cạnh đi ra từ g trong thành phần liên thông hiện tại). Đặt $f(i)$ là khoảng cách từ g đến đỉnh xa nhất trong cây con i , với thành phần hiện tại được gốc ở g . Khi đó chuỗi dài nhất qua trọng tâm g là

$$\max \left(f(g), \max_{\substack{p, q \in \text{Br} \\ p \neq q}} (f(p) + f(q)) \right) - v_g.$$

Mỗi lần sửa đỉnh i , xét mọi thành phần trong các tầng phân trị chứa i . Số thành phần như vậy chỉ là $\mathcal{O}(\log n)$, nên có thể lần lượt sửa vết cạn. Trong mỗi thành phần, phần bị sửa là khoảng cách từ mọi đỉnh trong cây con của i , với gốc hiện tại g , đến g ; mọi giá trị đó cùng cộng thêm hiệu giữa v_i mới và v_i cũ.

Dễ nghĩ đến việc lấy thứ tự DFS của thành phần khi gốc là g , rồi dùng cây đoạn duy trì giá trị nhỏ nhất của khoảng cách đến g trên một đoạn; khi đó mỗi lần sửa là sửa một đoạn.

Sau khi sửa độ sâu, cần duy trì giá trị đường đi qua trọng tâm. Giá trị $f(g)$ chỉ cần truy vấn gốc cây đoạn. Để duy trì

$$\max_{\substack{p, q \in \text{Br} \\ p \neq q}} f(p) + f(q),$$

có thể dùng heap để duy trì $f(p)$ của từng nhánh; khi đó chỉ cần lấy giá trị lớn nhất và lớn nhì. Mỗi $f(p)$ của một nhánh cũng có thể truy vấn đoạn trên cây đoạn.

Độ phức tạp là $\mathcal{O}(n \log n) + \mathcal{O}(\log^2 n) + \mathcal{O}(n \log n)$. Thuật toán qua được subtask 2, 3, 4, kỳ vọng 40 điểm.

7.4 Dữ liệu trên cây với $k = 3$

Xét một lớp dữ liệu đặc biệt: dữ liệu trên cây với $k = 3$. Lớp này không xuất hiện riêng trong subtasks, nhưng là hướng rất gần với lời giải chính. Ta sẽ suy nghĩ về lớp bài toán này và thử mở rộng lời giải của nó sang đồ thị tổng quát.

²⁰Ở đây khoảng cách được định nghĩa là tổng trọng số đỉnh trên đường đi đơn duy nhất giữa hai đỉnh.

7.4.1 Thuật toán bốn: DP trên cây

Vẫn có thể thiết kế một thuật toán quy hoạch động. Ghi $f(i, d)$ là tổng trọng số lớn nhất của một đồ thị con liên thông mà mọi đỉnh đều nằm trong cây con của i , có chứa i , và bậc của i đúng bằng d . Đặt $g(i) = \max f(i, *)$. Khi đó chuyển $f(i, *)$ bằng cách với mỗi $p \in \text{Ch}(i)$, thực hiện một DP ba lô với trọng lượng 1, giá trị $g(p)$, rồi cuối cùng cộng thêm a_i .

Độ phức tạp của thuật toán là $\mathcal{O}(nk^2) + \mathcal{O}(nk^2) + \mathcal{O}(nk)$; điểm kỳ vọng và các test qua được giống thuật toán hai.

7.4.2 Hướng tối ưu?

Trong thuật toán ba, ta dùng phân trị theo đỉnh, chia cây thành $\mathcal{O}(\log n)$ cấu trúc phân trị. Khi sửa, chỉ cần lần lượt sửa các cấu trúc ấy rồi dùng cấu trúc dữ liệu (cây đoạn) để duy trì mỗi cấu trúc phân trị, nhờ đó hỗ trợ sửa hiệu quả.

Tuy nhiên với bài $k = 3$, đối tượng cần tìm không còn là đường đi có tổng trọng số lớn nhất mà là một thành phần liên thông. Trong phân trị theo đỉnh, không chỉ khó xử lý “thành phần liên thông hợp lệ đi qua trọng tâm”, mà còn khó hỗ trợ sửa hiệu quả.

Dưới đây là một bài ví dụ gợi mở cho việc suy nghĩ về bài gốc.

Ví dụ 1. Tập độc lập trọng số lớn nhất trên cây, phiên bản có cập nhật. Cho một cây vô căn có n đỉnh, mỗi đỉnh có trọng số a_i . Có q thao tác; mỗi thao tác sửa a_i của một đỉnh. Ban đầu và sau mỗi lần sửa, hãy in tổng trọng số của tập độc lập có tổng trọng số lớn nhất trên cây. Dữ liệu: $n, q \leq 1000000$, $1 \leq a_i \leq 10^9$. Giới hạn thời gian 3 giây, bộ nhớ 512 MB.²¹

Nếu không có cập nhật, có thể thiết kế DP như sau. Chọn tùy ý một đỉnh làm gốc. Đặt $f(i)$ là tổng trọng số lớn nhất của tập độc lập trọng số lớn nhất nằm trong cây con của i , trong đó i thuộc tập độc lập; đặt $g(i)$ là giá trị tương tự khi i không thuộc tập độc lập. Khi đó

$$g(i) = \sum_{p \in \text{Ch}(i)} \max(f(p), g(p)), \quad f(i) = a_i + \sum_{p \in \text{Ch}(i)} g(p).$$

Nhưng DP này không cho phép tính nhanh nghiệm tối ưu khi dữ liệu vào thay đổi.

Xét tiếp trường hợp đặc biệt: cây là một chuỗi. Bài toán trên chuỗi dễ duy trì bằng cây đoạn. Với mỗi nút cây đoạn, ghi thông tin $\text{ans}(a, b)$, trong đó $a, b \in \{0, 1\}$. Nếu đoạn mà nút đó duy trì là $[l, r]$, thì $\text{ans}(a, b)$ là tổng trọng số lớn nhất của tập độc lập có mọi đỉnh nằm trong $[l, r]$, thỏa $a = [l \text{ thuộc tập độc lập}]$, $b = [r \text{ thuộc tập độc lập}]$. Phép gộp thông tin đoạn là

$$\text{ans}(a, b) = \max_{c+d < 2} (\text{ans}_l(a, c) + \text{ans}_r(d, b)).$$

Trong cây đoạn, số nút chứa một vị trí chỉ là $\mathcal{O}(\log n)$, nên sau khi sửa cũng chỉ cần tính lại $\mathcal{O}(\log n)$ nút đó. Vì vậy thuật toán này hỗ trợ tính nghiệm tối ưu sau cập nhật trong $\mathcal{O}(\log n)$ thời gian.

Nhưng khi mở rộng lên cây, ta gặp không ít rắc rối. Thuật toán cây đoạn trên chuỗi thực chất ghi lại thông tin của mọi đỉnh trong thành phần liên thông hiện tại (trên chuỗi là một đoạn) nối với bên ngoài (các thành phần liên thông của cấu trúc phân trị khác), cụ thể là chúng có thuộc tập độc lập hay không. Trên chuỗi, một thành phần liên thông có nhiều nhất 2 điểm nối ra ngoài, nên ghi được. Trên cây, nếu phân trị theo đỉnh, một thành phần liên thông có thể nối với các thành phần liên thông khác qua $\mathcal{O}(\log n)$ đỉnh, rất phức tạp để duy trì, càng khó hỗ trợ cập nhật hiệu quả. Vì vậy phân trị động theo đỉnh không phù hợp.

²¹Nguồn: bài toán kinh điển.

Điểm mấu chốt khiến cây đoạn và phân trị theo đỉnh hiệu quả là tư tưởng phân trị: mỗi lần sửa chỉ ảnh hưởng đến $\mathcal{O}(\log n)$ cấu trúc phân trị, nhờ đó bảo đảm độ phức tạp. Ở đây, ta có thể xét phân trị dựa trên chuỗi (gọi tắt là phân trị theo chuỗi).

Phân trị theo chuỗi thực hiện sau khi phân rã nặng–nhẹ: lấy ra một chuỗi nặng, xóa chuỗi đó khỏi cây, rồi đệ quy xử lý từng cây con còn lại (gốc của những cây con này là các con nhẹ của một đỉnh trên chuỗi nặng). Sau khi đệ quy xong, xét đóng góp của chuỗi nặng vào đó.²²

Dùng phân trị theo chuỗi, xét tối ưu trực tiếp DP ở trên. Dễ thấy đỉnh đầu của mỗi chuỗi nặng hoặc là gốc cây, hoặc có cha thuộc một chuỗi nặng khác. Vì vậy các chuỗi nặng cũng tạo thành quan hệ cây có gốc: nếu chuỗi nặng A là cha của chuỗi nặng B , thì cha của đỉnh đầu chuỗi B nằm trong A , còn đỉnh đầu của B là một con nhẹ của một đỉnh trong A . Gọi cây này là cây chuỗi nặng. Ta thay đổi thứ tự DP theo thứ tự của cây chuỗi nặng: trước đây xử lý mọi đỉnh trong cây con, còn bây giờ xử lý mọi chuỗi nặng trong cây con của cây chuỗi nặng hiện tại.

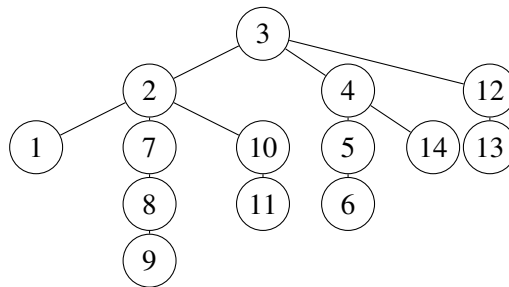
Với mỗi chuỗi nặng, sau khi đã đệ quy xử lý mọi chuỗi nặng trong các cây con theo cạnh nhẹ của từng đỉnh và đã xét ảnh hưởng của các giá trị DP đó lên đáp án, bài toán trên chuỗi nặng gần như trở thành bài tập độc lập động trên dãy, nên có thể dùng cây đoạn trên dãy để duy trì. Ta dùng lại thiết kế trạng thái $\text{ans}(a, b)$ của dãy, có sửa đổi nhỏ. Giả sử các đỉnh trên chuỗi nặng theo thứ tự khoảng cách đến gốc tăng dần là $p_1, p_2, \dots, p_c$, và nút cây đoạn hiện tại duy trì đoạn $[l, r]$. Ghi $\text{ans}(a, b)$ là tổng trọng số lớn nhất của tập độc lập có mọi đỉnh nằm trong các đỉnh $p_l, p_{l+1}, \dots, p_r$ và trong các cây con của con nhẹ của chúng, thỏa $a = [p_l \text{ thuộc tập độc lập}]$, $b = [p_r \text{ thuộc tập độc lập}]$. Phép gộp thông tin giống trên dãy.

Còn một vấn đề: làm sao tính ảnh hưởng của mọi cây con theo cạnh nhẹ của một đỉnh, tức làm sao tính $\text{ans}(a, b)$ cho đoạn dài 1 trên chuỗi. Chỉ có $\text{ans}(0, 0)$ và $\text{ans}(1, 1)$ có ý nghĩa. Gọi đỉnh đơn đó là i , và $\text{Ch}_0(i)$ là tập con nhẹ của i . Từ DP ở trên, dễ có

$$\text{ans}(0, 0) = \sum_{p \in \text{Ch}_0(i)} \max(f(p), g(p)), \quad \text{ans}(1, 1) = a_i + \sum_{p \in \text{Ch}_0(i)} g(p).$$

Chỉ cần thống kê hai tổng này.

Quan sát hình sau:



Đừng quên rằng con nhẹ của một đỉnh chính là p_1 của chuỗi nặng chứa con nhẹ ấy. Vì vậy có thể mượn thông tin của chuỗi nặng chứa con nhẹ để tính giá trị DP. Với con nhẹ p ,

$$f(p) = \max(\text{ans}(1, 0), \text{ans}(1, 1)), \quad g(p) = \max(\text{ans}(0, 0), \text{ans}(0, 1)).$$

Do đó chỉ cần mở hai biến cho mỗi đỉnh để duy trì hai tổng nói trên của mọi con nhẹ.

Mỗi lần sửa, trước hết sửa chuỗi nặng chứa đỉnh bị sửa, rồi nhảy tới chuỗi nặng chứa cha của đỉnh đầu chuỗi hiện tại, thực hiện cập nhật một điểm; cứ thế nhảy đến gốc. Khi nhảy giữa các chuỗi nặng, không được tính lại hai tổng bằng $\mathcal{O}(|\text{Ch}_0(p_i)|)$, mà phải trực tiếp cộng trừ vào hai biến duy trì tổng.

Như vậy bài toán được giải với độ phức tạp $\mathcal{O}(n) + \mathcal{O}(\log^2 n) + \mathcal{O}(n)$.

²²Cài đặt cụ thể và chứng minh chi tiết về phân trị theo chuỗi có thể xem [1].

Nhìn lại thuật toán này, nó đổi thứ tự DP thành thứ tự của cây chuỗi nặng, từ đó chuyển bài toán thành bài toán trên chuỗi rồi dùng thuật toán trên dây để giải hiệu quả. Vì thuật toán không trực tiếp khai thác tính chất riêng của bài toán trên cây, mà chia cây thành nhiều chuỗi và lần lượt giải bài toán trên dây; trong khi bài toán tương tự trên dây thường có nhiều tính chất hơn và dễ duy trì hơn bài toán trên cây, thuật toán có khả năng mở rộng rất mạnh. Bây giờ xét mở rộng thuật toán này sang dữ liệu trên cây với $k = 3$.

7.4.3 Thuật toán năm: phân trị theo chuỗi

Xét dùng phân trị theo chuỗi để giải bài trên cây với $k = 3$ có cập nhật. Vẫn tối ưu trực tiếp DP của thuật toán bốn. Trước hết, sau khi xử lý xong mọi chuỗi nặng trong cây con, với mỗi chuỗi nặng, điều cần xét là mọi cây con liên thông có đỉnh cao nhất nằm trên chuỗi nặng này. Giao của cây con ấy với chuỗi nặng là một đoạn trên chuỗi. Vì vậy, sau khi đã xét đóng góp của các cây con theo cạnh nhẹ, bài toán trên mỗi chuỗi nặng giống một bài tổng đoạn con lớn nhất có giới hạn bậc đỉnh, và cũng có thể duy trì bằng cây đoạn.

Gọi các đỉnh trên chuỗi nặng theo thứ tự khoảng cách đến gốc tăng dần là $p_1, p_2, \dots, p_c$. Với mỗi nút cây đoạn, giả sử đoạn là $[l, r]$, ghi các thông tin sau, với tiền đề chung là “mọi đỉnh đều nằm trong cây con của p_l ”:

- $\text{cro}(a, b)$: tổng trọng số lớn nhất của đồ thị con liên thông chứa p_l và p_r , trong đó bậc của p_l là a , bậc của p_r là b ;
- $\text{lef}(a)$: tổng trọng số lớn nhất của đồ thị con liên thông chứa p_l nhưng không chứa p_r , trong đó bậc của p_l là a ;
- $\text{rig}(b)$: tổng trọng số lớn nhất của đồ thị con liên thông chứa p_r nhưng không chứa p_l , trong đó bậc của p_r là b ;
- mid : tổng trọng số lớn nhất của đồ thị con liên thông không chứa cả p_l lẫn p_r .

Cách duy trì thông tin giống bài tổng đoạn con lớn nhất. Khi gộp hai đoạn, liệt kê trạng thái của nút trái và nút phải, kiểm tra bậc của đỉnh giữa có thể gộp hay không (để gộp cần thêm cạnh giữa hai đỉnh đó, nên bậc của cả hai đều phải nhỏ hơn k); nếu gộp được thì cập nhật trạng thái sau khi gộp. Độ phức tạp gộp thông tin là $\mathcal{O}(k^4)$, có thể tối ưu bằng tổng tiền tố xuống $\mathcal{O}(k^2)$.

Vì cần đếm số phương án, thông tin không được dư thừa: phải phân loại nghiêm ngặt để mỗi đồ thị con thuộc đúng một lớp. Một số chi tiết cần chú ý là: $\text{cro}(a, *)$ của nút trái có thể cập nhật $\text{lef}(a)$ của nút hiện tại; $\text{cro}(*, b)$ của nút phải có thể cập nhật $\text{rig}(b)$; $\text{rig}(*)$ của nút trái và $\text{lef}(*)$ của nút phải đều có thể cập nhật mid . Trong trạng thái sau khi gộp, nếu cả hai nút con đều không phải đoạn đơn thì bậc của p_l sau gộp là bậc của p_l trong trạng thái nút trái, bậc của p_r sau gộp là bậc của p_r trong trạng thái nút phải. Nhưng nếu một nút con là đoạn đơn, ví dụ nút trái là đoạn đơn, thì bậc của p_l sau gộp sẽ thay đổi (trong bài này chỉ tăng 1), nên cũng cần duy trì.

Cuối cùng còn phải tính thông tin $\text{cro}(*, *)$, $\text{lef}(*)$, $\text{rig}(*)$, mid cho từng điểm trên chuỗi. Nhờ phân rẽ nặng–nhẹ, con nhẹ của một đỉnh chính là đầu trái của chuỗi nặng chứa con nhẹ đó, nên lại có thể mượn thông tin cây đoạn của chuỗi chứa con nhẹ. Với một đỉnh đơn, chỉ có mid và $\text{cro}(a, a)$ có ý nghĩa.

Để tiện, với mỗi chuỗi nặng, ghi một “thông tin chuỗi nặng”: $\text{val}(d)$ là tổng trọng số lớn nhất của đồ thị con nằm hoàn toàn trong cây con của đỉnh đầu chuỗi, chứa đỉnh đầu chuỗi và bậc của đỉnh đầu chuỗi là d ; ans là tổng trọng số lớn nhất nằm hoàn toàn trong cây con của đỉnh đầu chuỗi nhưng không chứa đỉnh đầu chuỗi. Có thể lấy $\text{val}(d)$ từ $\text{lef}(d)$ và $\text{cro}(d, *)$ của gốc cây đoạn của chuỗi; lấy ans từ mid và $\text{rig}(*)$.

mid của một đỉnh đơn là giá trị lớn nhất trong ans của thông tin chuỗi nặng của các con nhẹ. Còn $\text{cro}(a, a)$ của đỉnh đơn là kết quả DP ba lô trên các mảng $\text{val}(\ast)$ của thông tin chuỗi nặng của từng con nhẹ. Để hỗ trợ sửa, với mỗi đỉnh dùng một cây đoạn hoặc cây cân bằng để duy trì phép gộp ba lô các $\text{val}(\ast)$ của mọi chuỗi nặng con nhẹ và giá trị ans lớn nhất. Như vậy có thể tính thông tin đỉnh đơn trên chuỗi và hỗ trợ sửa.

Mỗi lần sửa, cũng trước hết sửa chuỗi nặng chứa đỉnh bị sửa, rồi nhảy tới chuỗi nặng chứa cha của đỉnh đầu chuỗi hiện tại để cập nhật một điểm; cứ thế đến gốc. Tương tự, khi nhảy giữa các chuỗi nặng, cần sửa trong cây đoạn hoặc cây cân bằng duy trì thông tin chuỗi nặng con nhẹ.

Độ phức tạp là $\mathcal{O}(n) + \mathcal{O}(\log^2 n k^2) + \mathcal{O}(n)$. Các subtasks qua được và điểm kỳ vọng giống thuật toán ba.

Đến đây, ta đã giải thành công mọi dữ liệu trên cây. Tiếp theo xét dữ liệu trên đồ thị tổng quát.

7.5 Dữ liệu đồ thị tổng quát

Trong subtask 5 đến 8, tổng 40 điểm, không có cập nhật; chỉ cần giải một lần.

7.5.1 Tính chất đặc biệt

Chú ý điều kiện đặc biệt của đề: với mọi tập V' gồm $t \in \{5, 7\}$ đỉnh bất kỳ trong V , luôn tồn tại ba đỉnh đôi một khác nhau p, q, x sao cho $p, q \in V', x \in V - \{p, q\}$, và sau khi xóa x khỏi đồ thị thì p và q không liên thông. Xét ý nghĩa của điều kiện này.

Nhắc lại kiến thức về thành phần song liên thông đỉnh.²³ Ta biết hai đỉnh $p, q \in V$ song liên thông đỉnh khi và chỉ khi không tồn tại $x \in V - \{p, q\}$ sao cho sau khi xóa x , p và q không liên thông. Trong một đồ thị vô hướng, có thể dùng thuật toán Tarjan để tìm từng thành phần song liên thông đỉnh. Nếu hai đỉnh p, q cùng nằm trong một thành phần song liên thông đỉnh C , thì p, q song liên thông đỉnh; ngược lại thì không. Hai thành phần song liên thông đỉnh có không quá một đỉnh chung. Nếu hai thành phần A, B có đỉnh chung x , thì với mọi $p \in A - \{x\}, q \in B - \{x\}$, sau khi xóa x , p và q không liên thông. Ta gọi x là điểm khớp (giữa A và B).

Từ các tính chất trên, đoán điều kiện đặc biệt tương đương với mệnh đề: đồ thị không có thành phần song liên thông đỉnh có kích thước ít nhất t , tức mọi thành phần song liên thông đỉnh đều có kích thước nhỏ hơn t . Thử chứng minh:

Chứng minh. (Đủ) Phản chứng. Nếu tồn tại một thành phần song liên thông đỉnh có kích thước không nhỏ hơn t , chỉ cần lấy t đỉnh bất kỳ trong thành phần đó để tạo V' . Khi đó các cặp đỉnh trong V' đều song liên thông đỉnh, tức không tồn tại $x \in V - \{p, q\}$ sao cho sau khi xóa x , p và q không liên thông, mâu thuẫn đề bài.

(Cần) Vẫn phản chứng. Nếu tồn tại t đỉnh tạo thành tập V' sao cho không tồn tại ba đỉnh đôi một khác nhau $p, q \in V', x \in V - \{p, q\}$ làm cho sau khi xóa x , p và q không liên thông, thì theo định nghĩa, mọi $p, q \in V'$ đều song liên thông đỉnh. Vì vậy V' là một thành phần song liên thông đỉnh hoặc là tập con của một thành phần song liên thông đỉnh; do đó nhất định tồn tại một thành phần song liên thông đỉnh có kích thước ít nhất t , mâu thuẫn giả thiết. ■

Điều kiện đặc biệt trong đề có $t = 7$, nên mỗi thành phần song liên thông đỉnh có kích thước không vượt quá 6. Có thể thiết kế thuật toán dựa trên tính chất này. Trong các thuật toán sau, nếu không gây nhầm lẫn, ta gọi tất thành phần song liên thông đỉnh là “khối song liên thông”.

²³Cài đặt cụ thể của thành phần song liên thông đỉnh và chứng minh các định lý liên quan có thể xem [2].

7.5.2 Thuật toán sáu: vét cạn trên các khối song liên thông

Subtask 5 và 6 thỏa tính chất 3, nên mỗi khối song liên thông có kích thước không vượt quá 4.

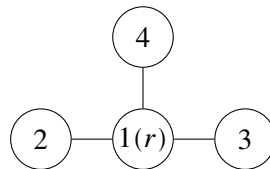
Quá trình Tarjan tìm khối song liên thông như sau. Trong DFS, duy trì một ngăn xếp. Mỗi khi DFS tới đỉnh i , đẩy i vào ngăn xếp, tăng bộ đếm thời gian, rồi đặt $dfn(i)$ và $low(i)$ bằng giá trị bộ đếm hiện tại. Với mỗi cạnh (i, p) , nếu đó là cạnh ngược lên tổ tiên, dùng $dfn(p)$ cập nhật $low(i)$; ngược lại DFS vào p , sau khi quay về dùng $low(p)$ cập nhật $low(i)$. Nếu $low(p) \geq dfn(i)$, ta tìm được một khối song liên thông C : pop ngăn xếp đến khi pop p . Nếu tập đỉnh được pop là S , thì tập đỉnh của khối vừa tìm là $V_C = S \cup \{i\}$, và gọi i là gốc của C .

Nếu một đỉnh p nằm trong một khối C và không phải gốc của C , ta nói p thật sự thuộc C . Khi đó, trừ đỉnh đầu tiên gọi Tarjan (gốc của cây DFS), mỗi đỉnh thật sự thuộc đúng một khối. Mỗi khối có đúng một gốc, và gốc của nó thật sự thuộc một khối khác; vì vậy các khối tạo thành một rừng cây có gốc, trong đó cha của mỗi khối là khối chứa gốc của nó (nếu có). Gọi rừng cây có gốc này là cây khối song liên thông. Kết hợp với quá trình Tarjan, có thể thấy Tarjan tương đương với duyệt cây khối theo thứ tự DFS.

Bây giờ xét DP trên cấu trúc cây. Thiết kế trạng thái $f(i, d)$: tổng trọng số lớn nhất của một đồ thị con liên thông có mọi đỉnh nằm trong khối song liên thông gốc i (và mọi khối trong cây con của các khối ấy), đồng thời bậc của i trong đồ thị con là d . Vì Tarjan duyệt cây khối theo DFS, với mọi $p \in S$, toàn bộ khối có gốc p và mọi khối dưới nó đã được xét xong (khi tìm khối có gốc i , đỉnh i chắc chắn còn trên ngăn xếp), nên $f(p, *)$ cũng đã được tính.

Xét xử lý trong khối C để cập nhật đáp án và $f(i, *)$. Vì $|V_C| \leq 4$, số cạnh không vượt quá $\binom{4}{2} = 6$, nên có thể giống thuật toán một: vét cạn tập cạnh để liệt kê một đồ thị con của C , tức liệt kê phần cạnh của đồ thị con cần tìm bên trong khối C .

Tiếp theo, còn phải xét cách nối đồ thị con này với các cây con trong cây khối. Hình sau là một đồ thị con đang được liệt kê (giả sử $k = 3$):



Ví dụ, trong phần cạnh hiện tại của khối, bậc của đỉnh 2 là 1; ta còn có thể thêm vào 2 nhiều nhất 2 bậc từ phía dưới. Do đó cách nối trong cây con chỉ cần thỏa bậc của 2 không vượt quá 2, tức có thể ghép ²⁴ đồ thị con hiện tại với một đồ thị con “mọi đỉnh đều nằm trong khối có gốc 2 (và mọi khối trong cây con dưới nó), đồng thời bậc của đỉnh 2 không vượt quá 2”. Vậy trọng số tốt nhất của đồ thị con hiện tại có thể cộng thêm $\max(f(2, 0), f(2, 1), f(2, 2))$.

Thực hiện phép ghép như vậy cho mỗi đỉnh trong đồ thị con G' đang xét, ta tính được tổng trọng số lớn nhất khi phần nằm trong C là G' và đã xét mọi khối trong cây con của C ; gọi giá trị đó là $g(G')$. Nếu G' chứa gốc i , dùng $g(G')$ cập nhật $f(i, *)$ (đây là một bài toán ba lô); ngược lại dùng $g(G')$ cập nhật đáp án toàn cục ans, vì khi đó đồ thị con hiện tại không còn liên thông với i và các đỉnh phía trên, nên cần cập nhật đáp án tại đây. Cần chú ý: khi tính tổng trọng số lớn nhất của G' , chỉ cộng trọng số các đỉnh không phải gốc trong tập đỉnh của G' ; trọng số của gốc được xét trong khối chứa thật sự đỉnh gốc, tránh tính sai tổng trọng số đồ thị con.²⁵ Sau khi Tarjan chạy xong, ta thu được đáp án. Số phương án có thể được duy trì đồng thời trong quá trình này.

²⁴Ở đây “ghép” hai đồ thị con nghĩa là lấy hợp tập đỉnh và hợp tập cạnh của chúng.

²⁵Nếu cộng trọng số của gốc, thì trong mọi khối có cùng gốc đó trọng số gốc đều bị cộng một lần, làm $f(i, *)$ tính lặp trọng số gốc.

Gọi c là kích thước khối song liên thông lớn nhất trong đồ thị. Độ phức tạp xấu nhất là

$$\mathcal{O}\left(c2^{\binom{c}{2}}n + m\right) + \mathcal{O}\left(c2^{\binom{c}{2}}n + m\right) + \mathcal{O}(n + m).$$

Thuật toán qua được subtask 1, 2, 5, 6; có thể qua subtask 3, kỳ vọng 46 đến 53 điểm.²⁶

7.5.3 Thuật toán bảy: tiền xử lý đồ thị con

Subtask 7 và 8 không còn thỏa tính chất 3; một khối song liên thông có thể có $\binom{6}{2} = 15$ cạnh, và tổng số đỉnh n rất lớn, nên độ phức tạp của thuật toán sáu không qua được.

Nút thắt của thuật toán sáu nằm ở việc liệt kê mọi đồ thị con của mỗi khối. Xét tối ưu việc liệt kê. Trong thuật toán một, ta gộp các trạng thái tương đương và không đổi (tiền xử lý số cách nối cạnh cho mỗi tập đỉnh), từ đó tối ưu độ phức tạp; ở đây cũng có thể làm tương tự. Nhưng bây giờ còn phải xét giới hạn bậc, nên cần ghi trạng thái: một số tứ phân 6 chữ số deg²⁷ nén trạng thái bậc của từng đỉnh (gọi tắt là “nén trạng thái bậc”), trong đó chữ số thứ i , từ thấp đến cao, biểu thị bậc của đỉnh thứ i trong đồ thị. Hai trạng thái có cùng deg có cùng tổng trọng số và cách chuyển như nhau, nên chỉ cần tiền xử lý số cách nối cạnh cho từng trạng thái deg. Có một điểm tiện lợi: với một đồ thị có ít nhất một cạnh, mọi đỉnh trong nó đều có bậc ít nhất 1, nên ghi deg cũng biểu thị được tập đỉnh. Đặt $f(\text{deg})$ là số đồ thị con khác nhau có nén trạng thái bậc là deg. Khi đó chỉ cần tính từng $f(\text{deg})$ là có thể thống kê nghiệm tối ưu và số phương án, không cần liệt kê từng đồ thị con.

Xét cách tính $f(\text{deg})$. Có thể ngay từ đầu tiền xử lý mọi đồ thị có không quá 6 đỉnh thỏa giới hạn bậc (không quá 20000 đồ thị), rồi sắp theo số đỉnh. Khi xử lý mỗi khối, chỉ cần liệt kê mọi đồ thị có số đỉnh không vượt quá kích thước khối hiện tại, dùng phép bit để kiểm tra nhanh tập cạnh của đồ thị đó có phải là tập con của khối hiện tại hay không. Như vậy mỗi khối chỉ cần chưa đến 20000 phép tính. Số trạng thái khác nhau về lý thuyết chỉ là 4^6 , thực tế chưa đến 1600, nên có thể qua dữ liệu subtask 7, 8.

Để tăng tốc chương trình nhằm qua subtask 3, có thể tối ưu thêm. Thuật toán này thực chất là DP trên cây theo giai đoạn là cây con; sau khi sửa đỉnh p , chỉ các giá trị DP trên đường từ p đến gốc có thể thay đổi. Vì vậy mỗi lần sửa cũng chỉ cần sửa các đỉnh trên đường này. Chú ý trong DP ban đầu có thao tác “cập nhật đáp án toàn cục”; có thể không ghi biến toàn cục ans, mà ghi $g(i)$ là tổng trọng số lớn nhất sau khi xét mọi khối có gốc i và mọi khối trong cây con của chúng. Khi đó mỗi lần cũng chỉ cần cập nhật $g(i)$ trên đường từ đỉnh bị sửa đến gốc. Thực tế, sau các tối ưu này, thuật toán có thể qua một phần test của subtask 10.

Gọi c là kích thước khối lớn nhất; $S(s, k)$ là số đồ thị con có không quá s đỉnh và mọi đỉnh có bậc không vượt quá k ; ²⁸ $G(s, k)$ là số nén trạng thái bậc của các đồ thị con có không quá s đỉnh và mọi đỉnh có bậc không vượt quá k .²⁹ Độ phức tạp của thuật toán là

$$\mathcal{O}\left(c2^{\binom{c}{2}} + (S(c, k) + G(c, k)c)n + m\right) + \mathcal{O}(G(c, k)cn) + \mathcal{O}(S(c, k) + G(c, k)n + m).$$

Thuật toán qua được subtask 1, 2, 5, 6, 7, 8; có thể qua subtask 3, kỳ vọng 65 đến 72 điểm.

7.6 Cách chuyển hóa tốt hơn

Hai subtasks cuối có nhiều cập nhật, cần hỗ trợ sửa hiệu quả hơn.

²⁶Với dữ liệu cây, $c = 2$.

²⁷Cơ số là lũy thừa của 2 cho phép dùng phép bit để truy vấn, sửa từng chữ số nhanh; vì vậy dù k có giá trị nào, vẫn dùng cơ số 4.

²⁸Theo trên, $S(6, 3) \leq 20000$.

²⁹Theo trên, $G(6, 3) \leq 1600$.

7.6.1 Cây tròn–vuông: cải tiến cây khối song liên thông

Ở trên ta đã nhắc đến cấu trúc cây khối song liên thông. Nhưng cấu trúc này tương đương với việc gom mọi đỉnh của một khối lại với nhau, gây nhiều bất tiện: trong chuyển trạng thái vừa phải liệt kê đồ thị con, vừa phải thống kê trọng số, lại phải xử lý riêng nhiều chi tiết. Vì vậy cấu trúc cây này khó được duy trì trực tiếp bằng phân trị theo chuỗi.

Bây giờ xét sửa cây khối song liên thông để DP gọn hơn và dễ tối ưu hơn.

Ví dụ 2. Thương nhân. Cho một đồ thị vô hướng liên thông có n đỉnh, m cạnh, mỗi đỉnh có trọng số a_i . Có q truy vấn; mỗi truy vấn cho hai đỉnh x, y , yêu cầu tìm một đường đi đơn từ x đến y sao cho trọng số nhỏ nhất trên đường đi là nhỏ nhất. Dữ liệu: $n, m, q \leq 5000000$, $1 \leq a_i \leq 10^9$. Giới hạn thời gian 2 giây, bộ nhớ 512 MB.³⁰

Nếu $x = y$, đáp án là a_x . Sau đây mặc định $x \neq y$.

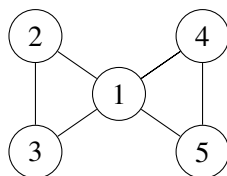
Trước hết, nếu tồn tại một khối song liên thông C chứa đồng thời x, y , thì đáp án là trọng số nhỏ nhất trong khối đó. Chứng minh như sau.

Chứng minh. Trước hết chứng minh với mọi $q \in C$, tồn tại một đường đi đơn từ x đến y đi qua q . Tạo đỉnh phụ p , nối $(p, x), (p, y)$. Vì x, y liên thông, tồn tại một đường đi từ y đến x không qua p . Khi đó từ p đến x có hai đường đi: $p \leftrightarrow x$ và $p \leftrightarrow y \rightarrow x$; do đó p và x song liên thông đỉnh. Tương tự, p và y song liên thông đỉnh, nên x, p, y cùng thuộc một khối. Vì hai khối có nhiều nhất một đỉnh chung, p nhất định thuộc khối $C \cup \{p\}$. Vì vậy với mọi $q \in C$, tồn tại hai đường đi không giao nhau về đỉnh giữa p và q ; trong đó một đường đi qua x , một đường đi qua y . Nhờ đó tồn tại đường đi đơn $x \rightarrow q \rightarrow y$. Vậy mọi đỉnh q trong khối đều có thể nằm trên một đường đi đơn từ x đến y .

Tiếp theo chứng minh với mọi $q \notin C$, không tồn tại đường đi đơn từ x đến y đi qua q . Với điểm q ngoài khối, lấy một khối bất kỳ D chứa nó. Giả sử trên đường đi trong cây khối từ C đến D , dãy khối là $C, P_1, P_2, \dots, P_c, D$. Khi đó từ x đến q phải đi qua điểm chung của C và P_1 ; từ q đến y cũng phải đi qua điểm chung đó. Điểm này bị đi qua hai lần, nên q không nằm trên bất kỳ đường đi đơn nào từ x đến y . ■

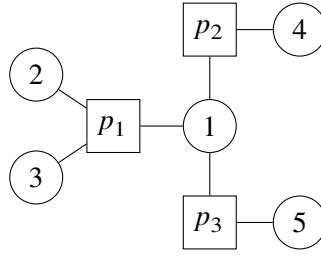
Tiếp tục xét trên cây khối. Có thể đoán tính chất: với mọi truy vấn x, y , đáp án là giá trị nhỏ nhất của các đỉnh trong mọi khối trên đường đi cây khối từ khối chứa x đến khối chứa y . Nhưng mô tả này chưa đủ cụ thể: x, y có thể nằm trong nhiều khối, vậy cặp khối nào mới đúng? Chọn tùy ý thì không đúng; phản ví dụ là đồ thị ghép từ ba tam giác. Một mô tả đúng là chọn cặp gần nhất. Nhưng mô tả ấy vẫn chưa đủ cụ thể và cũng khó cài đặt. Hạn chế của cây khối lộ rõ; ta cần một cách chuyển đồ thị thành cây tốt hơn.

Xét cách chuyển sau: với mỗi khối song liên thông C , tạo một đỉnh phụ p_C , nối p_C với mọi đỉnh trong C , và xóa mọi cạnh gốc trong khối (vì hai khối có nhiều nhất một đỉnh chung, nên không cạnh nào bị xóa lặp). Gọi các đỉnh của đồ thị gốc là đỉnh tròn, các đỉnh phụ mới là đỉnh vuông. Ví dụ đồ thị sau:



Chuyển thành cây tròn–vuông như sau (một cạnh cũng là một khối):

³⁰Nguồn: giản lược từ UOJ 30, đồng thời mở rộng dữ liệu.



Mỗi đỉnh vuông trong hình tương ứng với một khối. Để thấy đồ thị sau chuyển hóa tương đương với việc sửa cây khối như sau: 1. trong cây khối, mỗi khối nối trực tiếp với khối cha; sau chuyển hóa, đỉnh vuông của mỗi khối trước hết nối với đỉnh tròn là gốc của khối đó, rồi từ đỉnh tròn gốc nối với đỉnh vuông của khối cha; 2. các đỉnh không trở thành gốc của khối nào được giữ lại trên cây dưới dạng lá. Từ các sửa đổi này có thể thấy đồ thị sau chuyển hóa vẫn là một cây. Ta gọi cây này là cây tròn–vuông của đồ thị gốc.

Vì mỗi đỉnh phụ nối ra một “cây hoa cúc”, hình dạng cây tròn–vuông không phụ thuộc vào hình dạng cây khối. Nói cách khác, dù bắt đầu thuật toán Tarjan từ đỉnh nào, cây tròn–vuông dựng được đều giống nhau. Hơn nữa, vì cây giữ lại mọi đỉnh của đồ thị gốc, nó xử lý quan hệ giữa các khối thuận tiện hơn: nếu hai khối A, B của đồ thị gốc có điểm chung q , thì trên cây tròn–vuông có các cạnh (p_A, q) và (q, p_B) . Thay vì xem cây tròn–vuông là kết quả sửa cây khối, nên xem nó như một cách hoàn toàn mới để tổ chức thông tin giữa các khối.

Xét giải ví dụ trên cây tròn–vuông. Đặt trọng số của mỗi đỉnh vuông bằng trọng số nhỏ nhất trong khối tương ứng. Ta bất ngờ có: đáp án chính là trọng số nhỏ nhất trên đường đi từ x đến y trong cây tròn–vuông. Chứng minh tính chất này như sau. Mỗi khối nằm trên đường đi tròn–vuông từ x đến y đều có thể được đi vào từ một đỉnh tròn và đi ra từ một đỉnh tròn khác, do đó có thể đi qua bất kỳ đỉnh nào trong khối. Còn muốn đến một khối không nằm trên đường đi đó thì phải đi qua lặp lại một đỉnh tròn nào đó (trên đường giữa hai đỉnh vuông nhất định có ít nhất một đỉnh tròn), nên phải lặp đỉnh.

Bài toán còn lại là hỏi giá trị nhỏ nhất trên đường đi x đến y của một cây. Bài này rất giống bài kinh điển *Vận chuyển hàng hóa* của NOIP2013; dùng thuật toán trong mục 3.2.1 của [4] có thể đạt $\mathcal{O}(m + na(n)) + \mathcal{O}(q \log \log n) + \mathcal{O}(n \log \log n + m)$. Vì thuật toán không cần xử lý tình huống đặc biệt, mã cài đặt khá đẹp.

Ví dụ này thể hiện sức mạnh và sự tiện lợi của cây tròn–vuông. Bây giờ xét áp dụng cây tròn–vuông vào bài gốc.

7.6.2 Thuật toán tám: dùng cây tròn–vuông giải subtask 7 và 8

Ta đã có công cụ mạnh là cây tròn–vuông. Thử dùng nó giải subtask 7, 8 để tìm một thuật toán gọn hơn.

Trước hết dựng cây tròn–vuông của đồ thị gốc. Trong cây tròn–vuông, mọi cạnh của đồ thị gốc đều không được giữ lại, nên mọi cách nối cạnh trong đồ thị con phải được xử lý ở đỉnh vuông tương ứng. Bài này khá giống bài trên cây với $k = 3$, nên có thể mượn thuật toán bốn. Vẫn ghi trạng thái $f(i, d)$: nếu i là đỉnh tròn, nó biểu thị tổng trọng số lớn nhất của đồ thị con có mọi đỉnh nằm trong cây con tròn–vuông của i , có chứa i , và bậc của i trong đồ thị con đúng bằng d ; nếu i là đỉnh vuông, nó biểu thị tổng trọng số lớn nhất của đồ thị con có mọi đỉnh nằm trong cây con tròn–vuông của i , có chứa gốc r_i của khối tương ứng với i , và bậc của r_i trong đồ thị con đúng bằng d .

Xét chuyển trạng thái. DFS trên cây. Nếu i là đỉnh tròn thì mọi con của nó đều là đỉnh vuông, và gốc của các khối tương ứng đều là i . Chỉ cần làm DP ba lô trên các $f(p, *)$ của mọi con $p \in \text{Ch}(i)$, rồi cuối cùng cộng v_i .

Nếu i là đỉnh vuông, cần liệt kê cách nối cạnh trong khối tương ứng (dùng cách của thuật toán bảy để tối ưu liệt kê). Với mỗi đồ thị con được liệt kê, ta cũng xét mỗi đỉnh không phải gốc p trong đồ thị con

còn có thể thêm bao nhiêu bậc từ phía dưới. Nếu có thể thêm d bậc, trọng số đồ thị con hiện tại cộng thêm $\max_{k=0}^d f(p, k)$. Trên cây tròn–vuông, các con của một đỉnh vuông chính là mọi đỉnh không phải gốc trong khối tương ứng, nên các $f(p, *)$ của chúng đều đã tính xong. Cuối cùng, nếu đồ thị con này chứa gốc r_i , gọi bậc của r_i trong đồ thị con là d , thì dùng trọng số đồ thị con cập nhật $f(i, d)$; ngược lại dùng trọng số đó cập nhật đáp án toàn cục ans.

Thuật toán này nhìn qua không khác thuật toán bầy nhiều. Nhưng nó có một đặc điểm: mọi cách nối cạnh được xử lý tại đỉnh vuông, mọi phép tính trọng số và giới hạn bậc được xử lý tại đỉnh tròn. Việc phân công rõ ràng giữa đỉnh tròn và đỉnh vuông không chỉ khiến mô tả thuật toán rõ hơn, giảm độ phức tạp mã, mà còn rất thuận lợi cho tối ưu thuật toán sau này.

Độ phức tạp, subtasks qua được và điểm kỳ vọng đều giống thuật toán bầy.

7.6.3 Thuật toán chín: lời giải trọn điểm

Bây giờ mọi thứ đã sẵn sàng. Kết hợp phân trị theo chuỗi và cây tròn–vuông, ta đã rất gần lời giải trọn điểm. DP trên cây tròn–vuông gọn hơn DP trên cây khối trước đó, nên dễ dùng phân trị theo chuỗi để tối ưu. Trên mỗi chuỗi nặng, bài toán vẫn là tổng đoạn con lớn nhất có giới hạn bậc đỉnh, nên cách giải tương tự phần trước.

Trong cây đoạn trên chuỗi nặng, vẫn duy trì bốn thông tin $\text{cro}(*, *)$, $\text{lef}(*, *)$, $\text{rig}(*, *)$, mid . Định nghĩa ở đây gần giống trước nhưng có khác biệt nhỏ. Với mỗi đoạn, định nghĩa “điểm thật bên trái” p'_l và “điểm thật bên phải” p'_r : nếu p_l là đỉnh tròn thì $p'_l = p_l$, ngược lại p'_l là cha của p_l trên cây tròn–vuông, tức gốc của khối tương ứng với p_l (khi $l \neq 1$, đỉnh này chính là p_{l-1}); nếu p_r là đỉnh tròn thì $p'_r = p_r$, ngược lại $p'_r = p_{r+1}$ (ở đây p_{r+1} chắc chắn tồn tại vì cuối chuỗi nặng nhất định là lá). Nói cách khác, ở đây ngoài việc ghi thông tin bậc của gốc khối (đỉnh tròn cha) cho đỉnh vuông, trạng thái còn ghi thông tin của đỉnh tròn kế tiếp trong chuỗi nặng. Khi đó $\text{cro}(a, b)$ có nghĩa là “đồ thị con có trọng số lớn nhất khi bậc của p'_l đúng bằng a , bậc của p'_r đúng bằng b ”. Các trạng thái khác cũng sửa tương ứng: bậc được ghi là bậc của p'_l và p'_r .

Mỗi lần gộp thông tin, tại trung điểm $m = \lfloor (l + r) / 2 \rfloor$, ta đang xét bậc của cùng một đỉnh, tức đỉnh tròn trên cạnh (p_m, p_{m+1}) (mỗi cạnh có một đầu tròn và một đầu vuông). Đỉnh này vừa là điểm thật bên phải của nút trái, vừa là điểm thật bên trái của nút phải; do đó điều kiện hợp lệ khi gộp là tổng bậc của đỉnh đó trong hai trạng thái con không vượt quá k . Ở đây vẫn cần chú ý các chi tiết đã nói trong thuật toán năm, chẳng hạn trường hợp một nút con là đoạn đơn.

Còn vấn đề cuối cùng: tính thông tin đỉnh đơn trên mỗi chuỗi nặng. Nếu đỉnh đó là đỉnh tròn, vẫn dùng cách trong thuật toán năm: dùng cây đoạn duy trì thông tin của các chuỗi nặng đi ra, và cuối cùng mọi $\text{cro}(a, a)$ đều cộng thêm v_i . Nếu đỉnh đó là đỉnh vuông, cần liệt kê mọi trạng thái đồ thị con trong khối tương ứng, tức trạng thái dạng (set, deg) như trên. Giả sử đỉnh đơn cây đoạn là i ; trong trạng thái này, với các đỉnh tròn không phải điểm thật trái cũng không phải điểm thật phải, chúng nhất định là con nhẹ, nên dùng thông tin chuỗi nặng của chúng để ghép. Sau khi tính nghiệm tối ưu của trạng thái này, dựa vào việc trạng thái có chứa hai điểm thật hay không và bậc của chúng để quyết định cập nhật loại thông tin nào trong bốn loại thông tin của đỉnh đơn; số phương án phải nhân với số đồ thị con khác nhau của (set, deg) đã tiền xử lý. Một cách cài đặt tiện lợi là trong nén trạng thái, đặt điểm thật trái của đỉnh đơn là đỉnh số 0, điểm thật phải là đỉnh số 1; cách này giảm nhiều trường hợp riêng. Đừng quên ans trong thông tin chuỗi nặng của con nhẹ cũng phải cập nhật mid.

Mỗi lần sửa, trong quá trình nhảy lên theo phân rã cây, cần cập nhật thông tin đỉnh đơn của cha đỉnh đầu mọi chuỗi nặng đi qua. Nếu đỉnh này là đỉnh vuông, vẫn cần 1600×4 phép tính (số 4 là số đỉnh trong khối sau khi bỏ p_{i-1} và p_{i+1}) để liệt kê trạng thái và duy trì thông tin bằng vét cạn. Khi khởi tạo, ta tiền xử lý và lưu từng nén trạng thái đồ thị con cùng số phương án của nó, không cần mỗi lần đều duyệt mọi đồ thị con.

Kế thừa các ký hiệu $c, S(s, k), G(s, k)$ của thuật toán bầy, độ phức tạp là

$$\mathcal{O}\left(c2^{\binom{s}{2}} + (S(c, k) + G(c, k)c)n + m\right) + \mathcal{O}\left((G(c, k)c + k^4 \log n) \log n\right) + \mathcal{O}(S(c, k) + G(c, k)n + m).$$

Thuật toán qua được mọi subtask, kỳ vọng 100 điểm.

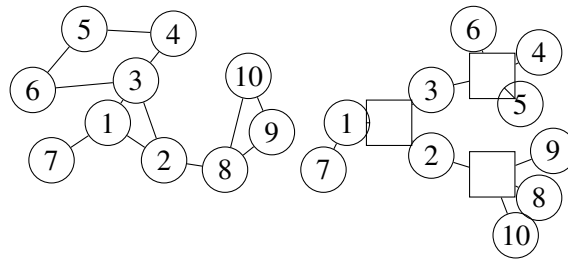
7.7 Mở rộng

Trong bài này, ta dựng cây tròn–vuông của đồ thị gốc, từ đó chuyển bài toán về cây và giải bằng phân trị theo chuỗi. Trên thực tế, cả cây tròn–vuông lẫn phân trị theo chuỗi đều có khả năng mở rộng lớn. Sau đây giới thiệu một số ứng dụng của chúng.

7.7.1 Bài toán trên xương rồng

Một xương rồng được định nghĩa là một đồ thị vô hướng liên thông không có khuyên, trong đó mỗi cạnh thuộc không quá một chu trình đơn.

Dễ thấy mỗi chu trình trong xương rồng là một khối song liên thông, nên tự nhiên cũng có thể dùng cây tròn–vuông để giải. Vì cần xử lý thông tin trên chu trình, ta còn phải duy trì thứ tự các đỉnh tròn kề với từng đỉnh vuông. Đồng thời, để phân biệt chu trình độ dài 2 (cạnh song song) với cạnh cây thông thường, ta có thể không tạo đỉnh vuông cho cạnh cây thông thường, mà chỉ tạo đỉnh vuông cho chu trình. Sau đây là một xương rồng và cây tròn–vuông của nó:



Thực ra, cây tròn–vuông ban đầu được đề xuất chính là để giải các bài toán tính trên xương rồng hiệu quả và tiện lợi hơn. Cách xử lý thành phần song liên thông đỉnh được cải tiến từ cách xử lý xương rồng.

Sau nghiên cứu, gần như mọi bài toán xương rồng tính như DP trên xương rồng,³¹ đường đi ngắn nhất trên xương rồng,³² phân rã chuỗi trên xương rồng,³³ xương rồng ảo,³⁴ và phân trị theo đỉnh trên xương rồng³⁵ đều có thể giải trực tiếp bằng cây tròn–vuông. Hơn nữa, theo nghiên cứu của các bạn khác,³⁶ ngay cả xương rồng động và k -xương rồng động³⁷ cũng có thể được giải bằng tư tưởng tương tự cây tròn–vuông.

Ứng dụng của cây tròn–vuông trên xương rồng không liên quan đến chủ đề chính của bài này và khá dài, nên được lược bỏ. Thuật toán cụ thể có trong blog UOJ của tác giả³⁸ và trong slide trao đổi học viên WC2017 của tôi.³⁹

³¹<http://www.lydsy.com/JudgeOnline/problem.php?id=4316>

³²<http://www.lydsy.com/JudgeOnline/problem.php?id=2125>

³³<http://uoj.ac/problem/158>

³⁴<http://uoj.ac/problem/189>

³⁵<http://uoj.ac/problem/23>

³⁶<http://awd.blog.uoj.ac/blog/2319>

³⁷Đồ thị vô hướng liên thông trong đó mỗi cạnh thuộc nhiều nhất k chu trình đơn. Xương rồng thường nói là 1-xương rồng.

³⁸<http://immortalco.blog.uoj.ac/blog/1955>

³⁹Xem tài liệu tham khảo [3], có thể tải từ blog UOJ của tôi.

7.7.2 DP cây con có hỗ trợ sửa dữ liệu vào

Trong bài này, ta dùng phân trị theo chuỗi để hỗ trợ hiệu quả thao tác sửa một đỉnh và duy trì giá trị DP toàn cục. Thực tế, thuật toán này có thể mở rộng sang một lớp bài tổng quát hơn: sửa dữ liệu vào tại một đỉnh và hỏi giá trị DP của một cây con trong cây có gốc.

Ví dụ 3. Bài toán khổng chế dịch bệnh. Cho một cây có gốc gồm n đỉnh, mỗi đỉnh có trọng số a_i . Với mỗi đỉnh, bạn có thể trả chi phí a_i để chọn toàn bộ lá trong cây con của i . Có m thao tác, mỗi thao tác thuộc một trong hai loại:

1. sửa a_i của một đỉnh;
2. nhập i , hỏi chi phí nhỏ nhất để chọn mọi lá trong cây con của i .

Dữ liệu $n, m \leq 1000000$. Giới hạn thời gian 1 giây, bộ nhớ 256 MB. ⁴⁰

Bài gốc có tính chất đặc biệt: sau cập nhật, a_i không giảm. Lời giải chuẩn của bài gốc xuất phát từ tính chất này và thiết kế một thuật toán trả góp dựa trên sự thay đổi của điểm quyết định. Ở đây không có bảo đảm ấy, nên thuật toán cũ không dùng được.

Nếu mỗi truy vấn đều làm một DP vét cạn riêng, có thể ghi $f(i)$ là chi phí nhỏ nhất để chọn mọi lá trong cây con của i . Nếu i là lá thì $f(i) = a_i$, ngược lại

$$f(i) = \min \left(a_i, \sum_{p \in \text{Ch}(i)} f(p) \right).$$

Thời gian là tuyến tính.

Bây giờ xét dữ liệu có cập nhật. Cây trong bài là cây có gốc, và việc tính giá trị DP có thứ tự nghiêm ngặt; nếu dùng phân trị theo đỉnh thì khó duy trì thứ tự tính DP, càng khó hỗ trợ truy vấn.

Tiếp tục xét phân trị theo chuỗi. Mượn tư tưởng ở trên, tính DP theo thứ tự của cây chuỗi nặng. Vì cây chuỗi nặng cũng là cây có gốc, tính DP theo thứ tự của nó chỉ tương đương với việc chọn thứ tự chuyển trạng thái giữa các con của mỗi đỉnh, không ảnh hưởng tính đúng đắn của chuyển trạng thái. Do đó dùng phân trị theo chuỗi ở đây là hoàn toàn khả thi.

Vấn đề đầu tiên là chuyển trên một chuỗi. Giả sử chuỗi nặng hiện tại có độ dài m ; ghi p_i là đỉnh thứ i từ trên xuống trên chuỗi, c_i là tổng $f(x)$ của mọi con nhẹ x của đỉnh i (để tiện, nếu i là lá thì đặt $c_i = \infty$).

Vì tính DP theo thứ tự cây chuỗi nặng, ta cũng chỉ cần dùng một biến duy trì c_i , giống thuật toán tập độc lập trọng số lớn nhất trên cây. Khi sửa, chỉ cần cộng vào c_i hiệu giá trị $f(x)$ của con nhẹ đó.

Để tính $f(i)$ của từng đỉnh trên chuỗi nặng, có thể DP như sau. Trước hết $f(p_m) = a_{p_m}$; tiếp theo với mọi $1 \leq i < m$,

$$f(p_i) = \min(a_{p_i}, f(p_{i+1}) + c_{p_i}).$$

Phân tích công thức này. Xem một vị trí (a_{p_i}, c_{p_i}) là một phép biến đổi (a, b) thỏa

$$\text{trans}_{(a,b)}(x) = \min(a, x + b).$$

Khi đó $f(p_i)$ là kết quả của các phép biến đổi $(a_{p_m}, c_{p_m}), (a_{p_{m-1}}, c_{p_{m-1}}), \dots, (a_{p_i}, c_{p_i})$ lần lượt tác động lên 0. Hai phép biến đổi như vậy gộp lại vẫn có dạng cũ:

$$\begin{aligned} \text{trans}_{(c,d)}(\text{trans}_{(a,b)}(x)) &= \text{trans}_{(c,d)}(\min(a, b + x)) \\ &= \min(c, d + \min(a, b + x)) \\ &= \min(\min(c, a + d), b + d + x). \end{aligned}$$

⁴⁰Nguồn: cải biên từ BZOJ 4712, đồng thời mở rộng dữ liệu.

Tức là trước hết thực hiện (a, b) , rồi thực hiện (c, d) , tương đương với thực hiện $(\min(c, a + d), b + d)$.

Vì vậy có thể dùng cây đoạn trực tiếp duy trì phép biến đổi hợp của một đoạn. Khi truy vấn, hỏi hậu tố của chuỗi nặng chứa đỉnh đó, độ phức tạp $\mathcal{O}(\log n)$. Khi sửa a_p , trước hết sửa vị trí p trong cây đoạn của chuỗi nặng chứa p , sau đó tính $f_{\text{top}(p)}$ mới, cập nhật $c_{\text{fa}(\text{top}(p))}$ và sửa cây đoạn tương ứng; tiếp tục cập nhật lên đến gốc. Độ phức tạp $\mathcal{O}(\log^2 n)$.

Như vậy bài toán được phân trị theo chuỗi giải với tiền xử lý $\mathcal{O}(n \log n)$, mỗi lần sửa $\mathcal{O}(\log^2 n)$, mỗi truy vấn $\mathcal{O}(\log n)$, và bộ nhớ $\mathcal{O}(n)$.

Bài toán này cũng trực tiếp dùng phân trị theo chuỗi để tối ưu bản thân DP, đồng thời khéo léo khai thác tính chất chuyển trạng thái trên chuỗi, cuối cùng thu được một thuật toán rất hiệu quả và mã cài đặt đẹp. Vì vậy, phân trị theo chuỗi quả thật là một công cụ mạnh cho một lớp bài toán DP trên cây có cập nhật.

7.7.3 DP cây con hỗ trợ Link/Cut và sửa dữ liệu vào

Dễ thấy trong các ví dụ trên, cài đặt cụ thể của phân trị theo chuỗi không quá khác phân rã cây nặng–nhẹ thông thường: đều dùng cây đoạn để duy trì một số thông tin có thể gộp hiệu quả. Vậy các bài này có thể mở rộng sang phân rã chuỗi động, tức LCT, để đồng thời hỗ trợ Link/Cut và sửa dữ liệu vào hay không?

Câu trả lời là có. Lấy ví dụ 1: chỉ cần với mỗi đỉnh dùng hai biến duy trì hai tổng của mọi cạnh nhẹ, phần còn lại giống hệt phân trị theo chuỗi. Không chỉ vậy, vì LCT hỗ trợ đổi gốc, dùng LCT còn có thể tính giá trị DP cho cây con bất kỳ khi lấy một đỉnh tùy ý làm gốc. Độ phức tạp của LCT là $\mathcal{O}(\log n)$ trả góp cho mỗi thao tác, thậm chí tốt hơn thuật toán phân trị theo chuỗi.

Dĩ nhiên, phân trị theo chuỗi dùng LCT cũng có hạn chế. Một là LCT có hằng số lớn, thời gian chạy thực tế chưa chắc tốt hơn thuật toán phân trị theo chuỗi có hằng số rất nhỏ. Hai là nếu thông tin duy trì trên cạnh nhẹ không có phép trừ (ví dụ duy trì giá trị nhỏ nhất của mọi cạnh nhẹ), thì cần dùng Splay để bảo đảm độ phức tạp $\mathcal{O}(\log n)$, tăng độ khó cài đặt. Ba là với một số DP thuộc lớp này, duy trì bằng LCT sẽ tạo ra rất nhiều chi tiết; chẳng hạn nếu ví dụ 3 cần hỗ trợ Link/Cut và truy vấn cây con bất kỳ, thì phải xử lý việc đỉnh lá thay đổi, và để hỗ trợ đảo khi đổi gốc còn phải duy trì hai loại nhãn xuôi và ngược.

7.7.4 Truy vấn đường đi đơn có hỗ trợ sửa

Trên đồ thị tổng quát, cũng có thể có bài toán yêu cầu hỗ trợ sửa và truy vấn thông tin của một đường đi đơn giữa hai đỉnh. Khi đó cây tròn–vuông và phân rã cây nặng–nhẹ cùng phát huy tác dụng.

Ví dụ 4. Tourists. Cho một đồ thị vô hướng liên thông có n đỉnh, m cạnh, mỗi đỉnh có trọng số a_i . Có q thao tác:

1. cho hai đỉnh x, y , yêu cầu tìm một đường đi đơn từ x đến y sao cho trọng số nhỏ nhất trên đường đi là nhỏ nhất;
2. nhập p, v , đặt $a_p \leftarrow v$.

Dữ liệu $n, m, q \leq 100000$, $1 \leq a_i \leq 10^9$. Giới hạn thời gian 1 giây, bộ nhớ 512 MB.⁴¹

Bài này là phiên bản mạnh hơn của ví dụ 2, yêu cầu hỗ trợ sửa. Nếu giống ví dụ 2, đặt trọng số của mỗi đỉnh vuông bằng trọng số nhỏ nhất trong khối tương ứng, rồi dùng phân rã cây nặng–nhẹ duy trì giá trị nhỏ nhất trên đường đi, thì mỗi lần sửa trọng số một đỉnh sẽ phải duyệt mọi khối chứa đỉnh đó, độ phức tạp không chấp nhận được.

⁴¹Nguồn: UOJ 30.

Đừng quên thuật toán năm của *Đồ thị con kỳ diệu*. Trong thuật toán năm, đỉnh tròn và đỉnh vuông phân công rõ ràng: đỉnh tròn chỉ duy trì trọng số, đỉnh vuông chỉ duy trì cách nối cạnh. Nhờ đó hai loại đỉnh được tách rời; khi sửa đỉnh tròn, không cần duyệt mọi đỉnh vuông kề nó. Tư tưởng ở đây là: không để thông tin có thể thay đổi bị quá nhiều cấu trúc đồng thời duy trì. Trong bài này, cho mỗi đỉnh vuông chỉ duy trì thông tin của các đỉnh tròn con của nó, không duy trì thông tin của đỉnh tròn cha. Khi trọng số một đỉnh thay đổi, chỉ cần cập nhật trọng số đỉnh vuông cha của nó. Thao tác sửa là $\mathcal{O}(\log n)$.

Xét truy vấn. Tách đường đi x đến y thành hai chuỗi $x \rightarrow z \rightarrow y$, trong đó z là LCA của x, y trên cây tròn–vuông. Để thấy mọi đỉnh vuông nằm hoàn toàn trong $x \rightarrow z$ hoặc $z \rightarrow y$ và có cha là z thì đỉnh tròn cha của nó cũng nằm trên đường $x \rightarrow z \rightarrow y$, nên không cần xét riêng. Nhưng nếu z là đỉnh vuông, đỉnh tròn cha của nó chưa được xét; chỉ cần xét vết cận điểm này. Các thao tác còn lại đều là bài kinh điển phân rã cây nặng–nhẹ kết hợp cây đoạn. Độ phức tạp $\mathcal{O}(n + m) + \mathcal{O}(\log^2 n) + \mathcal{O}(n + m)$.

Thực ra bài này cũng có lời giải không dùng cây tròn–vuông, nhưng tối ưu độ phức tạp của lời giải đó không dễ nghĩ ra. Nhìn từ góc độ cây tròn–vuông sẽ dễ nghĩ cách tối ưu hơn, giảm phần nào độ khó tư duy.

Ví dụ 5. Đường đi đơn kỳ diệu. Cho một đồ thị vô hướng liên thông có n đỉnh, m cạnh, mỗi đỉnh có trọng số a_i . Bảo đảm mỗi thành phần song liên thông đỉnh có kích thước không vượt quá $c = 8$. Có q thao tác:

1. nhập x, y , tìm một đường đi đơn từ x đến y sao cho tổng trọng số các đỉnh trên đường đi lớn nhất;
2. nhập p, v , đặt $a_p \leftarrow v$;
3. hỏi đường đi đơn dài nhất của cả đồ thị.

Dữ liệu $n \leq 100000$, $m \leq 500000$, $q \leq 2000000$, $-10^9 \leq a_i \leq 10^9$, bảo đảm số thao tác loại 2 không vượt quá 20000. Giới hạn thời gian 6 giây, bộ nhớ 2 GB.⁴²

Bài này có ràng buộc giống *Đồ thị con kỳ diệu*: giới hạn kích thước khối song liên thông, nên có thể dùng lời giải tương tự.

Nếu chỉ có thao tác 2 và 3, bài toán giống *Đồ thị con kỳ diệu* (vì ở đây không cần đếm, và với cùng tập đỉnh, đáp án của đường đi đơn và chu trình đơn là như nhau, nên hai bài tương đương), dùng thuật toán chín của *Đồ thị con kỳ diệu* là được. Tại mỗi đỉnh vuông, tiền xử lý mọi đường đi đơn; cần ghi tập đỉnh đi qua và hai đầu mút, nên tổng trạng thái là $\mathcal{O}(2^{c-2} \binom{c}{2})$. Cũng có thể ghi bậc đỉnh như thuật toán năm của *Đồ thị con kỳ diệu*; nếu không lưu chu trình, số trạng thái là như nhau. Độ phức tạp thao tác 2 là $\mathcal{O}(2^{c-2} \binom{c}{2} + \log n \log n)$ mỗi lần, thao tác 3 là $\mathcal{O}(1)$.

Bây giờ xét hỗ trợ thao tác 1. Chú ý mọi đỉnh tròn trên đường đi từ x đến y trong cây tròn–vuông đều bắt buộc đi qua, nên đáp án có thể trực tiếp cộng trọng số các đỉnh đó. Còn mỗi đỉnh vuông tương đối độc lập: chỉ cần trong mỗi đỉnh vuông lấy giá trị đường đi lớn nhất rồi cộng lại. Nói cách khác, nếu các đỉnh trên đường đi cây tròn–vuông từ x đến y lần lượt là $p_0, s_1, p_1, s_2, \dots, p_{m-1}, s_m, p_m$, trong đó $p_0 = x, p_m = y$, thì với mỗi $1 \leq i \leq m$, đóng góp của s_i là đường đi đơn trọng số lớn nhất từ p_{i-1} đến p_i (đường đi này chắc chắn nằm hoàn toàn trong khối tương ứng với s_i và độc lập với các khối khác).

Với mỗi chuỗi nặng, giả sử các đỉnh trên chuỗi theo thứ tự là $p_0, s_1, p_1, s_2, \dots, p_{m-1}, s_m, p_m$ (nếu đầu chuỗi nặng là đỉnh vuông thì không có p_0 , không cần duy trì thông tin của s_1). Chỉ cần đặt đóng góp của mỗi p_i là a_{p_i} , đóng góp của mỗi s_i là độ dài đường đi đơn dài nhất từ p_{i-1} đến p_i trừ $a_{p_{i-1}} + a_{p_i}$. Khi đó trên chuỗi nặng chỉ cần hỏi tổng đóng góp trên đoạn để hoàn thành truy vấn. Thực chất thông tin được duy trì ở đây là $\text{cro}(1, 1)$ của đoạn.

Một truy vấn có thể đi qua nhiều chuỗi nặng, nên còn phải xét đường đi được chọn trong khối của

⁴²Nguồn: tự sáng tác.

đỉnh vuông tại vị trí vượt chuỗi. Ở đây hai đầu mút s và t của đường đi đều đã biết. Có thể khi khởi tạo hoặc sửa, tiền xử lý trực tiếp đường đi trọng số lớn nhất cho từng cặp đầu mút; khi đó truy vấn ở đây là $\mathcal{O}(1)$. Vì vậy mỗi thao tác 1 có độ phức tạp $\mathcal{O}(\log^2 n)$.

Như vậy bài toán được giải với tiền xử lý $\mathcal{O}(2^{c-2} \binom{c}{2} n + m)$, thao tác 1 có độ phức tạp $\mathcal{O}(\log^2 n)$, thao tác 2 có độ phức tạp $\mathcal{O}(2^{c-2} \binom{c}{2} + \log n \log n)$, thao tác 3 có độ phức tạp $\mathcal{O}(1)$, và bộ nhớ $\mathcal{O}(2^{c-2} \binom{c}{2} n + m)$.

Bài này mở rộng một trường hợp đặc biệt của *Đồ thị con kỳ diệu*, đồng thời dùng hai tác dụng của phân rã nặng–nhẹ: phân trị và truy vấn trên chuỗi. Nó giải được cả hai dạng vấn đề “duy trì thông tin toàn cục” và “hỗ trợ truy vấn giữa hai đỉnh”, thể hiện sức mạnh của việc phối hợp cây tròn–vuông với phân rã nặng–nhẹ.

7.8 Ý tưởng ra đề và tổng kết

7.8.1 Ý tưởng ra đề

Những năm gần đây, trong một số cuộc thi trực tuyến hoặc hệ thống chấm trực tuyến, đã xuất hiện các bài cần hỗ trợ hỏi giá trị DP toàn cục hoặc của một cấu trúc con sau khi sửa một điểm, như BZOJ 4712, Codeforces 480E, Codeforces 750E. Tôi nghiên cứu sâu lớp bài này và suy ra một số thuật toán hiệu quả dựa trên tư tưởng phân trị.

Trong đó, loại DP cây theo giai đoạn là cây con, cần sửa dữ liệu vào một đỉnh, như BZOJ 4712, là thú vị nhất. Nhưng thuật toán do người ra đề BZOJ 4712 đưa ra là một thuật toán trả góp dựa trên tính chất đặc biệt của phép sửa. Tôi nghiên cứu bài này và cuối cùng thiết kế được một thuật toán dùng phân trị theo chuỗi. Thuật toán ấy không chỉ chạy nhanh hơn mà còn có phạm vi áp dụng rộng hơn, nên tôi mở rộng nó sang phạm vi rộng hơn và cải biên thành ví dụ 3. Lời giải này rất đẹp và khá rộng dụng, nên khiến tôi rất hứng thú; vì vậy tôi muốn mở rộng nó sang ứng dụng tổng quát hơn.

Với thành phần song liên thông đỉnh, một bài toán kinh điển, trước đây tôi đã nghiên cứu ra một cấu trúc có thể xử lý quan hệ giữa các khối hiệu quả và gọn: cây tròn–vuông. Nó có thể chuyển đồ thị thành cây, rồi dùng lời giải của bài toán trên cây. Kết hợp cây tròn–vuông và phân trị theo chuỗi, tôi ra bài toán này.

Để tránh thuật toán vét cạn hoặc thuật toán sai qua được nhờ dữ liệu yếu, tôi thiết kế thông tin cần tính là nghiệm tối ưu và số phương án tối ưu. Thông tin này vừa không có phép trừ, vừa không được đóng góp lặp hoặc bỏ sót, nên dễ dựng dữ liệu mạnh hơn.

Một mục đích khác của việc ra bài là hy vọng có thể chia sẻ với mọi người kết quả nghiên cứu của tôi về việc dùng cây tròn–vuông và phân trị theo chuỗi để giải một lớp bài toán, mong khơi gợi thêm suy nghĩ về bài toán DP động trên cây và bài toán thành phần song liên thông đỉnh.

Đồ thị con kỳ diệu là một bài tương đối khó. Subtask của nó chia thành hai mảng: dữ liệu là cây và dữ liệu $q = 0$. Chúng lần lượt làm yếu một điều kiện của đề, đồng thời gợi mở một phần tư tưởng của lời giải chính: phân trị trên cây và cây tròn–vuông. Thí sinh có thể suy nghĩ từ nhiều góc độ và chọn nhiều thuật toán khác nhau. Để cài đúng lời giải chính của bài, cần tư duy đủ chặt chẽ và khả năng xử lý chi tiết. Ngoài ra lời giải chính có lượng mã và nhiều chi tiết cài đặt, nên đây là một bài khó.

7.8.2 Tổng kết mở rộng

Cây tròn–vuông có thể xử lý các bài toán trên xương rỗng rất hiệu quả, gọn và tổng quát; nó cũng có thể giải một số bài truy vấn đường đi đơn trên đồ thị tổng quát. Phân trị theo chuỗi có thể giải một lớp bài DP theo cây con có hỗ trợ sửa, và còn có thể kết hợp với LCT để hỗ trợ Link/Cut và truy vấn cây con bất kỳ. Dùng phân rã nặng–nhẹ để duy trì cây tròn–vuông có thể giải hiệu quả một lớp bài truy vấn đường đi đơn giữa hai đỉnh có hỗ trợ sửa một đỉnh.

Ngày nay, thành phần song liên thông đỉnh và bài toán DP có cập nhật vẫn có độ khó nhất định đối với nhiều thí sinh. Tôi hy vọng bài này có thể khiến thí sinh suy nghĩ thêm về ứng dụng của phân trị theo chuỗi trong DP trên cây có cập nhật, cũng như về cây tròn–vuông; từ đó có nhiều mở rộng hơn và giải được những bài toán rộng hơn. Nếu bạn nào có thu hoạch nghiên cứu ở hướng này, hoan nghênh đến trao đổi với tôi.

Lời cảm ơn

1. Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi.
2. Cảm ơn cha mẹ đã sinh thành và dưỡng dục.
3. Cảm ơn cô Chen Ying của Trường Trung học số 1 Phúc Châu đã quan tâm và chỉ dạy.
4. Cảm ơn các anh chị khóa trên của Trường Trung học số 1 Phúc Châu đã đem lại gợi mở và chỉ dẫn.
5. Cảm ơn các huấn luyện viên đội tuyển quốc gia đã giúp đỡ tôi.
6. Cảm ơn bạn Liu Yifan của Trường Trung học số 1 Phúc Châu đã giúp đỡ tôi rất nhiều trong nghiên cứu cây tròn–vuông và đã đặt tên cho cây tròn–vuông.
7. Cảm ơn bạn Zhong Zhixian đã kiểm đề bài này và thẩm định bản thảo bài viết.
8. Cảm ơn mọi người đã cung cấp các tài liệu mà bài viết này từng tham khảo.
9. Cảm ơn quý vị đã dành thời gian quý báu trong lúc bận rộn để đọc bài.

Tài liệu tham khảo

- [1] Qi Zichao. *Ứng dụng thuật toán phân trị trong các bài toán đường đi trên cây*.
- [2] Tarjan, R. E. (1972), “Depth-first search and linear graph algorithms”, *SIAM Journal on Computing*, 1 (2): 146–160, doi:10.1137/0201010.
- [3] Chen Junkun, Zhong Zhixian. “Making Graphs into Trees”, trao đổi học viên WC2017.
- [4] Ren Zhizhou. *Bàn sơ về tư tưởng heuristic trong thi tin học*.
- [5] Wu Zuofan. *Báo cáo ra đề “Tài xế tàu rời Qin Chuan”*.
- [6] Xu Yinzhan. *Cây đoạn trong một lớp bài toán phân trị*.

Ghi chú về trích xuất

Nội dung bài đầu được trích bằng pdftotext từ trang vật lý 3 đến 13 của PDF, tương ứng trang bài viết 1 đến 11; trang vật lý 14 là bài kế tiếp. Không cần OCR. Hai dòng kết thúc vòng lặp trong mã giả của nguồn in là **end for** = 0; bản dịch giữ nguyên ký hiệu này như một dị biệt của nguồn.

Bài thứ hai được trích bằng pdftotext từ trang vật lý 14 đến 26 của PDF, tương ứng trang bài viết 12 đến 26. Các hình minh họa nhỏ được dựng lại bằng tikz; ký hiệu ma trận con xóa hàng/cột $A^{I,J}$ và các phân số trong chứng minh Định lý 4.1 được kiểm tra lại bằng trang render vì lớp văn bản của PDF làm mất chi tiết này.

Bài thứ ba được trích bằng pdftotext từ trang vật lý 29 đến 33 của PDF, tương ứng trang bài viết 27 đến 31; trang vật lý 34 là bài kế tiếp. Bài này không có hình, chỉ có một bảng giới hạn dữ liệu được dựng lại trong LaTeX. Công thức khai triển $E(x)$ ở thuật toán sáu được giữ theo bản in: về phải dùng x^k trong

tổng, dù về mặt ký hiệu đây có vẻ là lỗi nguồn thay vì x^i . Dòng định nghĩa $P(x)$ cũng được giữ theo bản in, nơi không có thừa số lũy thừa của x trong tổng.

Bài thứ tư được trích bằng pdftotext từ trang vật lý 34 đến 51 của PDF, tương ứng trang bài viết 32 đến 49; trang vật lý 52 là bài kế tiếp. Bài này không có hình; hai bảng và hai mã giả được dựng lại bằng LaTeX. Các ký hiệu bị lớp văn bản trích thành $<$, $($ hoặc dấu phẩy đã được đối chiếu với trang render và chuyển về $\notin$, $\subset$, $\neq$ khi rõ nghĩa. Bản in ở điều kiện cực đại của S trên trang 36 ghi $V \cup \{v\}$; bản dịch dùng $S \cup \{v\}$ theo nội dung toán học ngay sau đó. Hai dòng mã giả **end for** = 0 và **return** $I = 0$ được giữ theo bản in.

Bài thứ năm được trích bằng pdftotext từ trang vật lý 52 đến 76 của PDF, tương ứng trang bài viết 50 đến 74; trang vật lý 77 là bài kế tiếp. Các bảng và hình minh họa nhỏ được dựng lại bằng LaTeX/tikz. Các hình được kiểm tra bằng trang render sau khi biên dịch; không dùng ảnh cắt từ PDF. Nguồn không có mẫu mã chương trình cần giữ nguyên.